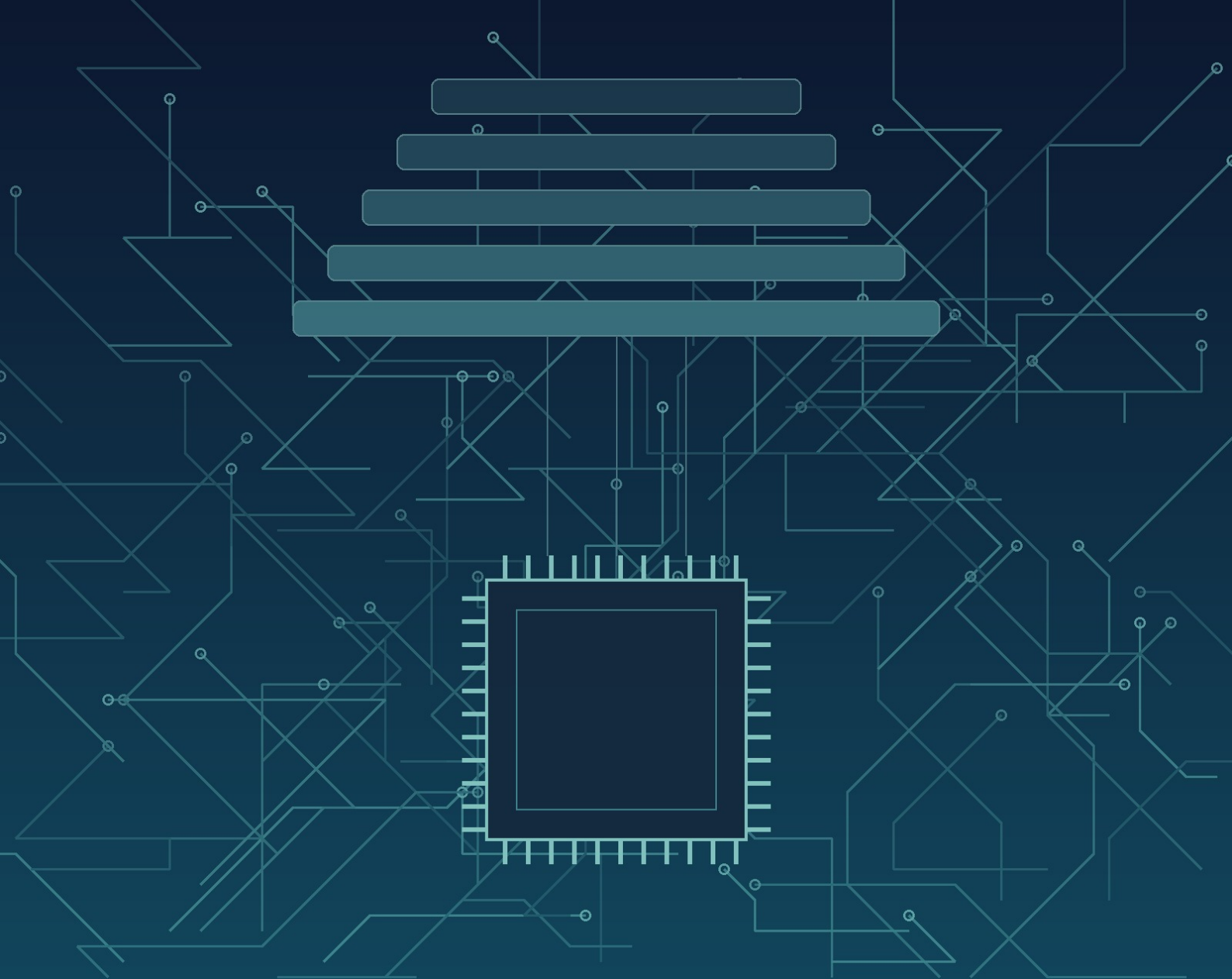




从裸机到操作系统

基于 xv6 的简单实现 —— Raspberry Pi 3 / AArch64

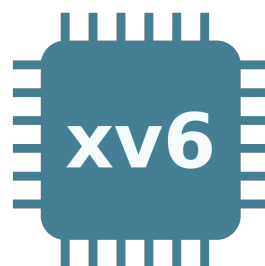
作者：Jiming Kang

组织：xv6-aarch64 项目

时间：2026 年 10 月

版本：0.1

源码：xv6-aarch64 (分支 rpi3-vfs)



从上电后的第一条指令，到能远程 SSH 登录、连 Wi-Fi、拍照的多核内核。

目录

第一章 序言	1
1.1 从一条指令开始	1
1.2 为什么是 xv6、AArch64 和树莓派	1
1.3 这个项目走过的路	1
1.4 本书的结构	2
1.5 阅读方法与约定	2
1.6 致谢	3
第二章 项目组织	4
2.1 目录结构	4
2.2 内核子系统	4
2.3 构建流程	5
2.3.1 编译选项	5
2.3.2 从目标文件到 kernel8.img	5
2.3.3 用户程序与文件系统镜像	6
2.4 安装、运行与调试	6
2.5 内核初始化顺序	7
第三章 基于汇编的裸机代码	8
3.1 xv6-aarch64 的启动代码	8
3.1.1 上电之后发生了什么	8
3.1.2 链接地址与装载地址	9
3.1.3 统一异常级别	9
3.1.4 不依赖任何环境的早期串口	12
3.2 异常和中断	14
3.2.1 中断与异常	14
3.2.2 异常向量	14
3.2.3 保存寄存器状态	16
3.2.4 设置向量表	17
3.2.5 屏蔽与解除屏蔽中断	18
3.2.6 配置中断控制器	19
3.2.7 通用 IRQ 处理程序	20
3.2.8 初始化定时器	23
3.2.9 处理定时器中断	24
3.2.10 小结	25
第四章 MMU	26
4.1 AArch64 的地址翻译	26
4.1.1 地址空间划分	26
4.1.2 三级页表	26
4.1.3 硬件如何遍历页表	27
4.1.4 描述符的属性位	27

4.1.5 内存属性间接表	27
4.2 启动页表	28
4.2.1 清 BSS 与早期栈	28
4.2.2 两棵临时页表	28
4.2.3 打开 MMU: TCR、MAIR 与 SCTLr	29
4.2.4 跳到高地址	30
4.3 多核启动	31
4.4 两阶段的物理内存初始化	32
4.5 最终内核页表	32
4.5.1 kvmmake()	32
4.5.2 walk() 与 mappages()	32
4.5.3 页表的规模	33
4.6 用户地址空间	34
4.6.1 内核空间与用户空间	34
4.6.2 exec() 之后的布局	34
4.6.3 第一个用户进程的页表	34
4.6.4 fork 时复制地址空间	35
4.6.5 TLB 与地址空间切换	35
4.6.6 缺页与按需分配	35
4.7 MMIO、总线地址与 DMA	36
4.7.1 MMIO	36
4.7.2 三种地址视图	36
4.8 缓存与 TLB: 三个真机故障	37
4.8.1 故障一: 停在 BPH0abde	37
4.8.2 故障二: ls 跳到了不属于它的代码	37
4.8.3 故障三: sbrk 之后的旧 TLB	38
第五章 进程创建: fork	39
5.1 进程由什么组成	39
5.1.1 struct proc	39
5.1.2 两种现场	40
5.2 动态进程表与内核栈	40
5.3 第一个用户进程	41
5.4 fork() 逐行解析	42
5.4.1 新进程的第一次运行	43
5.5 上下文切换与调度器	44
5.5.1 swtch()	44
5.5.2 调度循环	44
5.5.3 调度对进程是透明的	45
5.5.4 谁触发调度	45
5.5.5 关中断与禁止抢占	45
5.5.6 如何选择下一个进程	45
5.5.7 跟踪一次时间片轮转	46
5.6 exec、exit 与 wait	47
5.7 睡眠、唤醒与同步原语	47

5.7.1 sleep/wakeup	47
5.7.2 用户态同步原语	47
5.8 多核软实时调度	48
第六章 文件系统	49
6.1 分层结构	49
6.2 磁盘布局	49
6.3 块缓存、日志与 inode	50
6.4 块设备：从 ramdisk 到 SD 卡	50
6.4.1 三个阶段	50
6.4.2 SD 卡初始化	51
6.4.3 命令超时与恢复	51
6.5 根文件系统放在哪里	51
6.6 VFS 中间层	52
6.6.1 动机	52
6.6.2 挂载表与路径路由	52
6.6.3 三个后端	52
6.6.4 由用户态决定挂载	53
6.7 FAT32 与启动分区	53
6.7.1 bootfs 里有什么，谁来读	53
6.7.2 磁盘上的 FAT32	54
6.7.3 FAT 表与簇链	55
6.7.4 目录项与长文件名	55
6.7.5 函数一览与调用关系	56
6.7.6 写入：分配、追加与截断	57
6.8 xv6 根文件系统：superblock、inode 与目录	57
6.8.1 根文件系统怎样被认出来	57
6.8.2 superblock 与磁盘布局	58
6.8.3 inode：文件的元数据与块映射	59
6.8.4 目录与路径查找	60
6.8.5 从路径到 SD 扇区	61
6.8.6 mkfs 怎样生成 fs.img	61
6.8.7 在系统里更新根文件系统	62
6.9 烧录 SD 卡	62
6.9.1 分区布局	62
6.9.2 步骤	62
6.9.3 日常更新	63
第七章 驱动架构	64
7.1 设备模型	64
7.1.1 为什么不再只用 devsw[]	64
7.2 上半部与下半部	65
7.3 控制台、UART 与 TTY	66
7.3.1 Mini UART	66
7.3.2 控制台与行规程	66

7.3.3 TTY 层	66
7.4 USB 与 HID 键盘	66
7.4.1 三层软件与硬件路径	67
7.4.2 描述符：设备、配置、接口与端点	67
7.4.3 枚举：control 传输与请求序列	69
7.4.4 Split transaction：高速 Hub 后面的低速设备	69
7.4.5 URB 与中断驱动接收	70
7.4.6 HID 报告与输入子系统	71
7.4.7 真机排错：从 STALL 到第一份报告	71
7.5 I2C	73
7.6 网络协议栈	73
7.7 SDIO 无线网卡：BCM43430/43455	73
7.7.1 硬件路径	73
7.7.2 硬件结构：Arasan、SDIO 总线与 FullMAC 芯片	73
7.7.3 SDIO 枚举	75
7.7.4 固件装载	75
7.7.5 SDPCM 与 BCDC	75
7.7.6 WPA2 四次握手	76
7.7.7 数据流与并发	76
7.8 USB 无线网卡：MT7601U	77
7.9 CSI-2 摄像头：OV5647	78
7.9.1 路线选择	78
7.9.2 硬件结构：I2C、CSI-2、Unicam 与 DMA	78
7.9.3 sensor_ops：仿照 V4L2 子设备	79
7.9.4 板级资源	80
7.9.5 上电与探测	80
7.9.6 拍一帧	81
7.9.7 用户程序 camshot	82
7.9.8 一次成功拍照的日志解读	82
7.9.9 真机调试记录	84
7.9.10 当前状态	89
第八章 用户进程、线程与 libc	90
8.1 用户程序的形态	90
8.1.1 为什么需要系统调用	90
8.1.2 链接与装载	90
8.1.3 系统调用桩	90
8.1.4 内核如何分发系统调用	91
8.1.5 两个栈指针与返回级别	92
8.2 从 init 到 shell	92
8.3 用户程序一览	92
8.4 用户线程	93
8.4.1 设计	93
8.4.2 系统调用	93
8.4.3 clone 为什么“返回两次”	94

8.4.4 多核下的细节	94
8.5 libc 与 POSIX 源码兼容层	95
8.5.1 libc.a	95
8.5.2 POSIX 层	95
8.6 在 xv6 里编译 C 程序	95
8.7 一个大型用户程序: sshd	96
第九章 可选工作	97
9.1 ext2 读写与外置事务日志	97
9.1.1 动机与分区布局	97
9.1.2 ext2 的磁盘结构	98
9.1.3 从文件偏移到块号	99
9.1.4 分配与释放	100
9.1.5 接入 VFS: 按 inode 打开	100
9.1.6 持久化: <code>sdflush()</code> 写屏障	101
9.1.7 外置事务日志 <code>xjournal</code>	101
9.1.8 测试: WAL 程序与随机断电	103
9.1.9 在真机上使用	104
9.1.10 当前边界	104
参考文献	106

第一章 序言

本章内容

- ❑ 为什么要从裸机写起
- ❑ 本书的结构与阅读方法
- ❑ 为什么选 xv6、AArch64 和 Raspberry Pi 3
- ❑ 开发环境与约定
- ❑ 这个项目走过的路

1.1 从一条指令开始

一台树莓派插上电源后，GPU 固件把一段二进制文件拷到物理地址 0x80000，然后让 CPU0 从那里开始取指。此时没有栈，没有堆，没有 `printf`，没有页表，甚至不知道自己处于哪个异常级别。本书要回答的问题是：从这条指令出发，要依次搭起哪些东西，才能得到一个可以多核调度、读写 SD 卡、通过 Wi-Fi 被 SSH 远程登录、还能驱动摄像头的操作系统？

教科书通常按“进程、内存、文件、设备”分章讲操作系统原理，每一章都假设别的部分已经存在。真正动手时顺序恰好相反：要先让串口吐出一个字符，才能知道代码跑到了哪里；要先建好页表，内存分配器才有地方放空闲链表；要先有块设备驱动，文件系统才能读到 `superblock`。本书沿着这条“依赖链”组织内容，每一章都只依赖前面已经建好的东西。

1.2 为什么是 xv6、AArch64 和树莓派

xv6. xv6 是 MIT 为操作系统课程 6.S081/6.1810 编写的教学内核 [6]，用 C 重写了第六版 Unix 的核心思想。它只有一万行左右，却完整包含了进程、虚拟内存、系统调用、日志文件系统、管道和 shell。代码短，意味着每一个机制都能读到底；结构清楚，意味着加新功能时知道该放在哪一层。

AArch64. 原版 xv6 运行在 RISC-V 上。本项目的起点是一个面向 QEMU virt 机器的 AArch64 移植。ARMv8-A 的异常级别 (EL0 ~ EL3)、两套页表基址寄存器 (TTBR0_EL1/TTBR1_EL1)、内存属性间接表 (MAIR_EL1) 以及显式的缓存维护指令，都比 RISC-V 更接近手机和服务器的真实运行的系统，也更容易踩到“QEMU 正常、真机失败”的坑——而这些坑恰恰最有教学价值。

Raspberry Pi 3. 树莓派 3B/3B+ 使用 BCM2837 SoC (四核 Cortex-A53)，价格低、资料多，Linux 驱动全部开源，可以对照阅读。它的外设足够丰富：Mini UART、SD 卡控制器、DWC2 USB 主机、板载 SDIO Wi-Fi、I2C、CSI-2 摄像头接口……几乎每一类典型驱动都能在这块板子上找到一个真实的例子。同时 QEMU 提供了 `raspi3b` 机器模型，可以先在模拟器里调通流程，再上真机验证。

1.3 这个项目走过的路

项目的提交记录大致反映了“从裸机到操作系统”的顺序 (表 1.1)。最早的一个月只做了一件事：把 QEMU virt 上的 xv6 搬到 `raspi3b`；此后两周里，几乎每天都在真机上解决一个新问题。

这些功能都附有一篇专题文档 ([readme/](#) 目录)，记录了实现思路和调试过程。本书就是把这些文档按依赖顺序重新组织、补全前因后果之后得到的。

表 1.1: 主要里程碑 (取自 git 提交记录)

日期	内容
05-27	初始提交: QEMU virt 上的 xv6-aarch64
09-23	从 virt 转到 Raspberry Pi 3; SD 卡与 FAT32 分区
09-24	真机串口与内存属性问题; D/I-cache 一致性; SD 命令超时重试
09-27	进程间同步原语 (mutex、semaphore、event); 网络协议栈与 ping
09-28	TTY 中间层; Linux 风格的总线/设备/驱动模型; SDIO Wi-Fi
09-30	sshd 远程登录; SDIO 中断; workqueue 下半部
10-01	每设备私有 work、delayed work、NAPI; FAT32 启动分区挂载到 /boot; epoll; TFTP; 信号; USB 键盘
10-02	USB URB 接口; input 子系统与 evdev
10-03	多核软实时调度; 用户线程
10-04	POSIX 源码兼容层
10-05	CSI-2 摄像头 (OV5647 + Unicam)

1.4 本书的结构

- **第 2 章项目组织**: 目录结构、构建流程、镜像如何生成, 以及内核初始化顺序。这一章是全书的地图。
- **第 3 章基于汇编的启动代码**: 固件把控制权交给内核之后, `entry.S` 如何统一异常级别、点亮早期串口、建立临时页表、打开 MMU, 并唤醒其余三个核; 异常向量表和中断控制器也在这一章。
- **第 4 章 MMU**: 39 位虚拟地址、三级页表、内存属性; 内核页表与用户页表; MMIO 与 DMA 地址; 以及三个与缓存和 TLB 有关的真机故障。
- **第 5 章进程创建**: `struct proc`、第一个用户进程、`fork()` 的每一步、上下文切换和调度器, 以及在此基础上加入的同步原语和软实时调度。
- **第 6 章文件系统**: xv6 日志文件系统的分层, SD 卡驱动, 根文件系统的存放方式, VFS 中间层 (`procfs`、`ext2`、`FAT32`), 以及如何把系统烧录到 SD 卡。
- **第 7 章驱动架构**: 设备模型与上下半部; 控制台、UART 与 TTY; USB 与输入子系统; I2C; SDIO 与 USB 两种无线网卡; CSI-2 摄像头。
- **第 8 章用户进程、线程与 libc**: 系统调用桩、`init` 与 `shell`、用户线程、POSIX 兼容层和一个能在 xv6 里运行的小型 C 编译器。
- **第 9 章可选工作**: 主线之外的扩展。目前收录可读写的 `ext2`、`sdflush()` 写屏障和外置事务日志 `xjournal`, 以及用 WAL 程序做的断电测试。

1.5 阅读方法与约定

本书以源码为准。正文中的文件路径都相对于仓库根目录, 例如 `kernel/entry.S`。代码片段尽量保持原样, 只删去与讨论无关的行。描述某个函数时, 一般按“它解决什么问题 → 调用路径 → 关键代码 → 容易出错的地方”的顺序展开。

 **笔记** QEMU 的 `raspi3b` 模型与真机有不少差别: QEMU 对内存属性和缓存一致性非常宽容, 不模拟 SDIO Wi-Fi、摄像头和板载 USB 网卡, 固件 mailbox 也只实现了一部分。本书凡是只在 QEMU 上验证过的结论都会注明。一条经验贯穿全书: QEMU 能跑通不代表真机正确, 原子操作、缓存维护和设备时序必须在 Cortex-A53 上验证。

开发环境如下:

- 交叉工具链: `aarch64-linux-gnu-`、`aarch64-elf-` 或 `aarch64-unknown-elf-` 均可, Makefile 会自动探测;
- 模拟器: `qemu-system-aarch64 -machine raspi3b`;
- 真机: Raspberry Pi 3B 或 3B+, 一张 microSD 卡, 一根 3.3 V 的 USB-TTL 串口线 (GPIO14/15, 115200 8N1);
- 宿主机: macOS (文中的 `diskutil`、`/dev/rdiskN` 等命令以 macOS 为例, Linux 下换成 `fdisk/dd` 的等价写法即可)。

1.6 致谢

本项目从以下工作中获益良多：MIT 的 xv6[6]；Sergey Matyukevich 的 *raspberrypi-os*[7]，它对树莓派裸机异常与中断的讲解非常清楚；rockytriton 的 LLD 裸机教程 [12]，其中的 EMMC 驱动是本项目 SD 驱动的参考；Circle 裸机框架 [15]；以及 Raspberry Pi Linux 内核中的 `brcmfmac`、`mt7601u`、`bcm2835-unicam`、`ov5647` 等驱动 [11]。

第二章 项目组织

本章内容

- ❑ 仓库目录结构
- ❑ 内核子系统与源文件
- ❑ 从源码到镜像的构建流程
- ❑ 安装、运行与调试目标
- ❑ 内核初始化顺序：全书的地图

2.1 目录结构

仓库的顶层目录如表 2.1 所示。它保留了 xv6 的基本布局：`kernel/` 是内核，`user/` 是用户程序和用户库，`mkfs/` 是在宿主机上运行的文件系统镜像生成工具。在此之上，为了支撑 POSIX 源码兼容、真机部署和调试，又增加了 `include/`、`firmware/`、`tools/` 等目录。

表 2.1: 仓库顶层目录

路径	内容
<code>kernel/</code>	内核源码：启动汇编、异常、内存、进程、文件系统、VFS、网络栈、全部驱动
<code>user/</code>	用户程序（ <code>sh</code> 、 <code>ls</code> 、 <code>sshd</code> 、 <code>wifi</code> 、 <code>camshot</code> 等）和用户库（ <code>ulib</code> 、 <code>printf</code> 、 <code>umalloc</code> 、线程、POSIX 包装）
<code>include/</code>	面向 POSIX 源码的标准头文件： <code>stdio.h</code> 、 <code>unistd.h</code> 、 <code>pthread.h</code> 、 <code>sys/socket.h</code> 等
<code>mkfs/</code>	宿主机工具 <code>mkfs</code> ，把用户程序打包成 xv6 原生文件系统镜像 <code>fs.img</code>
<code>readme/</code>	30 余篇专题文档，记录每个功能的设计与调试过程
<code>firmware/</code>	无线网卡固件：BCM43430/43455 的 <code>.bin/.txt/.clm_blob</code> ，MT7601U 的 <code>mt7601u.bin</code>
<code>tools/</code>	调试脚本：串口跟踪的采集、回放和网页导出（ <code>trace_xv6.py</code> 等）
<code>ports/</code>	向其他开发板移植的实验（如全志 T113）
<code>config.txt</code>	树莓派固件配置文件，随内核一起拷到启动分区
<code>Makefile</code>	构建、打包、安装、QEMU 运行的全部规则

2.2 内核子系统

内核约 2.8 万行（含头文件），用户空间约 1.1 万行。按功能可以把内核源文件分成几组（表 2.2）。阅读时建议先看“启动与异常”和“内存与进程”两组，它们是其余一切的基础。

表 2.2: 内核源文件分组

子系统	主要文件
启动与异常	<code>entry.S</code> 、 <code>start.c</code> 、 <code>main.c</code> 、 <code>kernel.ld</code> 、 <code>trapasm.S</code> 、 <code>trap.c</code> 、 <code>timer.c</code> 、 <code>bcm2837.c</code>
内存	<code>kalloc.c</code> （物理页分配）、 <code>vm.c</code> （页表）、 <code>memlayout.h</code> 、 <code>aarch64.h</code> （系统寄存器与页表位）
进程与同步	<code>proc.c</code> 、 <code>proc.h</code> 、 <code>sched.h</code> 、 <code>swtch.S</code> 、 <code>spinlock.c</code> 、 <code>sleeplock.c</code> 、 <code>sync.c</code> 、 <code>workqueue.c</code> 、 <code>exec.c</code> 、 <code>syscall.c</code> 、 <code>sysproc.c</code>
文件系统	<code>bio.c</code> 、 <code>log.c</code> 、 <code>fs.c</code> 、 <code>file.c</code> 、 <code>sysfile.c</code> 、 <code>pipe.c</code> 、 <code>vfs.c</code> 、 <code>rootdev.c</code> 、 <code>fat32.c</code> 、 <code>ext2.c</code> 、 <code>xjournal.c</code>
设备核心	<code>device.c</code> 、 <code>device.h</code> （总线、设备、驱动、字符设备）
字符设备	<code>uart.c</code> 、 <code>console.c</code> 、 <code>tty.c</code> 、 <code>pty.c</code> 、 <code>input.c</code>
存储	<code>sdhost.c</code> （外置 SD 卡）、 <code>sd.c</code> （早期 Arasan SD 驱动）
USB	<code>dwc2.c</code> （主机控制器）、 <code>usb.c</code> （USB core 与 URB）、 <code>usbkbd.c</code> 、 <code>usbnet.c</code>
无线网络	<code>arasan_sdio.c</code> 、 <code>sdio.c</code> 、 <code>brcmfmac.c</code> 、 <code>wpa_crypto.c</code> 、 <code>mt7601u.c</code>
网络协议	<code>net.c</code> 、 <code>net.h</code> 、 <code>sysnet.c</code> 、 <code>epoll.c</code>
摄像头	<code>mbox.c</code> （固件 mailbox）、 <code>i2c.c</code> 、 <code>camsensor.h</code> 、 <code>ov5647.c</code> 、 <code>unicam.c</code>
其他	<code>rng.c</code> （硬件随机数）、 <code>printf.c</code> 、 <code>string.c</code>

2.3 构建流程

2.3.1 编译选项

内核是一个独立运行的程序：没有 C 运行库，没有动态链接器，也不能假设 CPU 已经打开浮点单元。关键的编译选项如下：

```
CFLAGS = -Wall -Werror -O0 -g -fno-omit-frame-pointer -mcpu=cortex-a53+nofp
CFLAGS += -ffreestanding -fno-common -nostdlib
CFLAGS += -I. -Iinclude -fno-stack-protector -fno-pie -no-pie
LDFLAGS = -z max-page-size=4096
```

- `-mcpu=cortex-a53+nofp`：生成 A53 指令，但禁止编译器使用浮点/SIMD 寄存器。内核上下文切换只保存通用寄存器，若编译器偷偷用 q0 做 memcpy，用户态的浮点状态就会被破坏。
- `-ffreestanding -nostdlib`：不假设标准库存在，memset、memcpy 等由 `kernel/string.c` 自己实现。
- `-fno-pie`：内核链接在固定的高虚拟地址，不需要位置无关代码。
- `-Werror`：所有警告视为错误，真机调试的成本很高，宁可在编译期多拦住一些问题。

2.3.2 从目标文件到 kernel8.img

图 2.1 是完整的构建流水线。

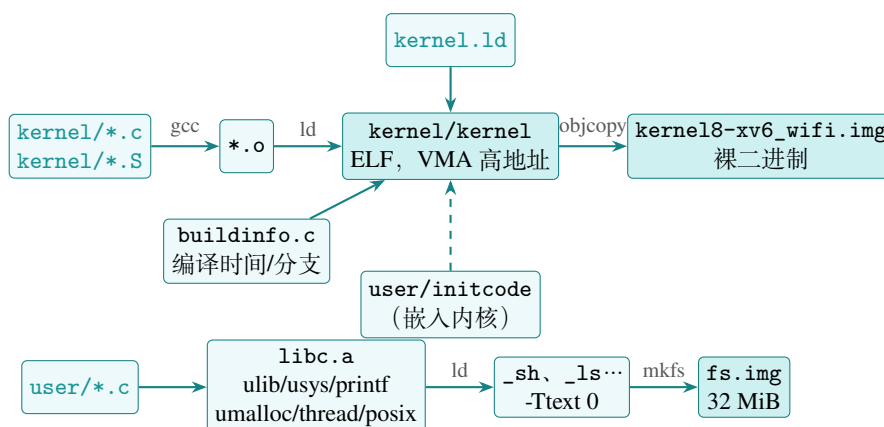


图 2.1: 构建流水线：上方生成内核镜像，下方生成根文件系统镜像

链接脚本 `kernel/kernel.ld` 同时给出两个地址：

```
. = 0xffffffff8000080000;      /* VMA: 运行时的虚拟地址 */
.text : AT(0x80000) { ... }    /* LMA: 镜像装入的物理地址 */
```

ELF 中的所有符号都按高虚拟地址解析，但固件并不解析 ELF，它只是把文件内容原样拷到物理地址 0x80000。因此需要 `objcopy -O binary` 去掉 ELF 头，得到“从第一个字节开始就是第一条指令”的裸镜像。在 MMU 打开之前，代码实际运行在物理地址上，这段代码必须只使用 PC 相对寻址（`adr/adrp`），这是第 3 章要处理的问题。

每次链接都会重新生成 `kernel/buildinfo.c`，写入当时的日期和 git 分支、提交号，启动时打印出来：

```
kernel image built: 2026-10-05 14:02:11 CDT (rpi3-vfs ebefc6c+dirty)
```

真机调试时经常要确认“SD 卡上的内核是不是刚编译的那个”，这一行能省去很多困惑。

2.3.3 用户程序与文件系统镜像

用户程序先编译成 `.o`，再与 `libc.a` 静态链接。每个程序都链接在虚拟地址 0、入口为 `main`：

```
LIBC_OBJS = $U/ulib.o $U/usys.o $U/printf.o $U/umalloc.o $U/thread.o \
            $U/posix.o $U/tweetnacl.o $U/ssh_crypto.o
_%: %.o $(ULIB)
$(LD) $(LDFLAGS) -N -e main -Ttext 0 -o $@ $^
```

`usys.S` 由 Perl 脚本 `user/usys.pl` 生成，每个系统调用对应三条指令（第 8 章）。最后 `mkfs` 把所有程序写进一个 32 MiB 的 `xv6` 原生文件系统镜像：

```
fs.img: mkfs/mkfs $(UPROGS)
mkfs/mkfs fs.img $(UPROGS)
truncate -s 32M fs.img
```

 **笔记** `mkfs` 是宿主机程序。如果仓库在 macOS 上编译过，`mkfs/mkfs` 就是一个 Mach-O 可执行文件，换到 Linux 构建时必须先删掉它重新编译，否则 `make fs.img` 会报 “Syntax error”。

2.4 安装、运行与调试

Makefile 提供了几组目标，覆盖从模拟器到真机的完整工作流（表 2.3）。

表 2.3: 常用 make 目标

目标	作用
<code>make</code>	构建内核与 <code>fs.img</code> ，并把内核同步到本机 TFTP 目录
<code>make qemu</code>	用 <code>raspi3b</code> 机器启动， <code>fs.img</code> 作为虚拟 SD 卡
<code>make qemu-net</code>	额外接入 QEMU 的 USB 网卡（CDC-ECM）和用户态 NAT
<code>make qemu-gdb</code>	启动后暂停，等待 GDB 连接
<code>make install-rpi3</code>	把内核、 <code>config.txt</code> 、Wi-Fi 固件拷到已挂载的启动分区；若给出 <code>RPI3_XV6_DEV</code> 则同时烧写根文件系统
<code>make install-rpi3-rawfs</code>	只把 <code>fs.img</code> 写入 SD 卡上的 <code>xv6</code> 原始分区
<code>make install-tftp</code>	把内核拷到宿主机 TFTP 目录，真机上用 <code>tftp</code> 命令下载到 <code>/boot</code>

QEMU 的 `raspi3b` 有两个串口：第一个是 PL011，第二个是 AUX Mini UART。本内核使用 Mini UART，因此 QEMU 参数要把第一个串口丢弃：

```
qemu-system-aarch64 -machine raspi3b -kernel kernel/kernel8-xv6_wifi.img \
    -display none -serial null -serial mon:stdio \
    -drive file=fs.img,if=sd,format=raw
```

真机通过启动分区里的 `config.txt` 指定内核文件名和几个关键参数：

```
arm_64bit=1           # 以 AArch64 状态启动 CPU
enable_uart=1         # 打开 GPIO14/15 上的串口
uart_2ndstage=1       # 固件自身的启动日志也输出到串口
core_freq=250          # 固定 VPU 时钟：Mini UART 波特率依赖它
core_freq_min=250
kernel=kernel8-xv6_wifi.img
camera_auto_detect=0   # 不让固件接管摄像头
```

其中 `core_freq=250` 很重要：Mini UART 的波特率由 VPU 核心时钟分频得到，如果固件动态调频，串口就会乱码（第 3.1.4 节）。

2.5 内核初始化顺序

`kernel/main.c` 是整个内核的入口函数，它的初始化顺序就是全书的目录。下面的代码去掉了注释中的细节，保留了调用顺序：

```
void main() {
    if(cpuid() == 0){
        kinit1(end, P2V(EARLYTOP)); // 只用启动页表覆盖的前 2 MiB    -> 第 4 章
        kvminit(); kvminithart(); // 建立并切换到最终内核页表    -> 第 4 章
        kinit2(P2V(EARLYTOP), P2V(PHYSTOP));
        device_init(); sdio_bus_init(); usb_bus_init(); // -> 第 7 章
        consoleinit(); ttyinit(); printfinit(); // 从这里起 printf 可用
        procinit(); workqueue_init(); // -> 第 5 章
        rnginit(); syncinit(); netinit();
        input_init(); usbkbd_driver_init(); dwc2_driver_init(); // USB 与键盘
        trapinit(); trapinithart(); gicv3init(); gicv3inithart(); // -> 第 3 章
        timerinit();
        binit(); iinit(); fileinit(); // -> 第 6 章
        sd_driver_init(); arasan_sdio_driver_init();
        fat32init(); rootdev_init(); // 找到根文件系统，挂起 bootfs
        brcmfmac_driver_init(); mt7601u_driver_init(); // 需要读 bootfs 里的固件
        ov5647_init(); camera_init(); // 摄像头
        ext2init(); vfsinit();
        userinit(); // 第一个用户进程
        start_secondary_cpus(); // 唤醒 CPU1~3
        started = 1;
    } else {
        while(started == 0) ;
        kvminithart(); trapinithart(); gicv3inithart(); timerinit();
    }
    scheduler();
}
```

几条顺序约束值得先记住，后面各章会解释原因：

- 页表必须在内存分配器之前就绪，但建页表本身又要分配页——所以分配器分两次初始化(`kinit1/kinit2`)。
- 设备核心 (`device_init`) 要在任何驱动注册之前初始化；控制台要尽早，因为之后的错误都要靠 `printf` 报告。
- 异常向量、中断控制器、定时器依次初始化，但真正打开 CPU 中断要等到 `scheduler()`。
- Wi-Fi 驱动需要从 FAT32 启动分区读取固件，所以必须排在 `rootdev_init()` 之后——它扫描 MBR 时顺带挂起 FAT32 启动分区。
- 其余三个 CPU 最后才被唤醒，它们只做每核相关的初始化，然后直接进入调度器。

第三章 基于汇编的裸机代码

本章内容

- ❑ 上电、链接地址与装载地址
- ❑ 异常级别：从 ARM9 的七种模式到 AArch64 的 EL0 ~ EL3
- ❑ 不依赖任何环境的早期串口
- ❑ 异常和中断：向量表、现场保存、中断控制器与定时器

3.1 节讲本项目的启动代码：从固件交出控制权、统一异常级别，到点亮早期串口。3.2 节讲异常和中断：向量表、现场保存、两级中断控制器和通用定时器。清 BSS、建启动页表、打开 MMU 和多核启动与页表关系密切，放在第 4 章的 4.2、4.3 节。这两节的代码全部来自 xv6-aarch64；它要兼容真机固件、QEMU 和多核，比教学内核复杂，所以每一步都先讲清楚背后的原理，这些原理主要参考了 Sergey Matyukevich 的 *raspberrypi-os*[7]（下文简称 RPi OS）第 1 ~ 3 课。RPi OS 后三课的原理也按同样的方式融入了后面各章：第 4 课（进程调度器）在第 5 章，第 5 课（用户进程与系统调用）在第 8 章，第 6 课（虚拟内存管理）在第 4 章，那些地方同样只用本项目的代码来讲。

3.1 xv6-aarch64 的启动代码

3.1.1 上电之后发生了什么

树莓派的启动与 PC 不同：最先运行的不是 ARM 核，而是 VideoCore GPU。

1. GPU 内部 ROM 从 SD 卡的 FAT32 分区读取 `bootcode.bin`，再加载 `start.elf`（GPU 固件）；
2. `start.elf` 读取 `config.txt`，初始化 SDRAM 和时钟，把内核镜像 `kernel8-xv6_wifi.img` 拷到物理地址 `0x80000`；
3. 固件还在物理地址 0 处放一段很小的 *armstub*。它让四个 Cortex-A53 核一起开始执行：CPU0 跳到 `0x80000`，CPU1 ~ 3 在 *spin-table* 上循环等待——各自读取 `0xe0`、`0xe8`、`0xf0` 处的 64 位地址，非零就跳过去；
4. 固件自带的 64 位 *armstub* 会先把核从 EL3 降到 EL2，因此内核通常在 EL2 拿到控制权；QEMU 也是如此。

项目早期曾使用自制的极简 *armstub* (`kernel/armstub.S`)，它只做一次跳转、保留在 EL3。后来发现这样做会让固件的 property mailbox 在真机上完全没有响应——而 SD 时钟查询、USB 上电、摄像头供电都依赖 mailbox。所以现在直接使用固件自带的 *armstub*，内核启动代码则要能同时处理 EL3、EL2 两种入口。

固件、`config.txt` 与镜像文件名。 树莓派的固件是闭源的，ARM 一侧能控制它的只有启动分区里的 `config.txt`。从一张空白 SD 卡启动一个裸机内核，至少需要四个文件：GPU 引导程序 `bootcode.bin`、GPU 固件 `start.elf`、`config.txt` 和内核镜像。固件根据镜像文件名决定 CPU 的执行状态：以 8 结尾的 `kernel8.img` 表示 ARMv8，固件会让四个核以 AArch64 状态启动；本项目用 `kernel=` 指定了自己的文件名，同时写了 `arm_64bit=1`。

另一种启动方式：`kernel_old`。 最简单的裸机教程（如 *raspberrypi-os*[7]）在 `config.txt` 中写 `kernel_old=1`。这时固件把镜像装在物理地址 0，并且不安装 *armstub*：四个核都从地址 0 开始执行同一段内核代码，而且停留在 EL3。内核必须自己读 `MPIDR_EL1` 的低 8 位区分核号，把非 0 号核挂进死循环，再自己从 EL3 降到 EL1。这种方式的好处是“什么都由自己掌控”，代价是失去了固件 *armstub* 提供的 *spin-table* 多核唤醒协议，也失去了 *armstub* 对 EL3 安全状态的初始化。本项目选择相反的路线：让固件做它擅长的事，内核只在 `_entry` 中兼容两种入口级别。

3.1.2 链接地址与装载地址

内核在高地址 0xfffff8000080000 链接，却被装到物理地址 0x80000。kernel/memlayout.h 用两个宏描述这种对应关系：

```
#define KERNBASE 0xfffff8000000000L    // 内核虚拟地址空间起点
#define KERNLINK (KERNBASE + KERNPA)    // 内核链接地址
#define V2P(a) (((uint64)(a)) - KERNBASE)
#define P2V(a) ((void *)(((char *) (a)) + KERNBASE))
```

KERNBASE 选在 39 位地址空间的高半部分起点：当 TCR_EL1.T1SZ= 25 时，TTBR1_EL1 负责的范围恰好是 0xfffff80000000000 ~ 0xffffffffffffffff。内核把全部物理内存线性映射到 KERNBASE + PA，所以物理地址与内核虚拟地址之间只差一个常数。

MMU 打开之前，CPU 用物理地址取指。此时如果执行 `ldr x0, =symbol` 得到的是高虚拟地址，访问它会立即出错；只有 PC 相对寻址的 `adr/adrp` 能得到正确的物理地址。这一限制决定了 `entry.S` 前半部分的写法。

独立环境下的编译。 内核不能依赖任何运行时环境，编译时要明确告诉编译器这一点：不链接 C 标准库（库中的大多数函数最终都要调用操作系统，而这里下面已经没有操作系统了）；不使用标准启动文件（设置栈、初始化静态数据、调用 `main` 这些事都由启动汇编自己完成）；声明为独立（freestanding）环境，使编译器不对 `memcpy` 之类的函数做任何假设；只使用通用寄存器，不让编译器生成 NEON/浮点指令，否则上下文切换就必须额外保存这些寄存器。本项目对应的选项是 `-ffreestanding -nostdlib -fno-common` 和 `-mcpu=cortex-a53+nofp`（第 2 章）。

为什么要把 ELF 变成裸镜像。 链接器输出的是 ELF 文件，它需要一个“装载器”去解析段表、把各段放到指定地址。固件不做这件事，它只是把文件按字节原样拷到内存，然后从第一个字节开始执行。所以要用 `objcopy -O binary` 抽出所有代码段和数据段，拼成一个裸镜像，并且保证启动代码位于镜像的最开头：常见的做法是把启动代码放进单独的 `.text.boot` 段并在链接脚本中排在第一位；本项目则让 `entry.o` 成为 OBJS 的第一个目标文件，`_entry` 自然落在 `.text` 的起点。

3.1.3 统一异常级别

AArch64 有四个异常级别：EL0 运行用户程序，EL1 运行操作系统内核，EL2 运行虚拟机监控器，EL3 运行安全监控固件。xv6 只需要 EL0 和 EL1。

为什么需要异常级别。 异常级别可以看成处理器的几种执行模式，每种模式只能使用一部分指令和寄存器。操作系统要实现进程隔离，就必须让用户程序运行在特权最低的 EL0：在这一级只能使用通用寄存器、栈指针和普通的访存指令，不能修改页表基址等系统寄存器，因此一个进程既看不到也改不了别的进程的内存。操作系统本身运行在 EL1，可以配置虚拟内存和大多数系统寄存器。EL2 留给虚拟机监控器：宿主系统在 EL2 隔离各个运行在 EL1 的客户系统，就像操作系统隔离用户进程一样。EL3 负责在“安全世界”和“非安全世界”之间切换，两边的隔离由硬件保证，非安全世界的软件无论如何都访问不到安全世界的指令和数据。

ARM9 时代：七种处理器模式。 四个异常级别是 ARMv8 才有的说法。在 ARM9（ARM920T、ARM926EJ-S 等 ARMv4T/v5 内核，S3C2440、AT91 这类板上最常见）时代，处理器用的是处理器模式（processor mode），当前模式记在 CPSR 的低 5 位 M[4:0] 里，一共七种（表 3.1）：用户模式（User）运行应用程序；系统模式（System）是“有特权的用户模式”；其余五种各对应一类异常——复位和 `swi` 指令进入管理模式（Supervisor, SVC），未定义指令进入未定义模式（Undefined），取指或数据访问出错进入中止模式（Abort），普通中断和快速中断分别进入 IRQ 模式和 FIQ 模式。

模式不等于特权级。七种模式里只有 User 是非特权的，其余六种的特权完全相同，都能改 CPSR、访问 CP15 协处理器（MMU、缓存的控制寄存器都在这里）。所以 ARM9 实际上只有两个特权级，模式之间的区别不在于“能做什么”，而在于用哪一组寄存器：每个异常模式有自己私有（banked）的 r13（SP）、r14（LR）和 SPSR，FIQ 还额外私有 r8~r12，这样快速中断处理程序不用保存任何通用寄存器就能开工。异常的类型直接决定进入哪个模式；向量表固定在地址 0x00000000（或 CP15 选择的高端 0xffff0000），每类异常占一个 4 字节的槽，只放得下一条跳转指令。

这种设计给操作系统带来不少麻烦。内核必须在启动时依次切进 IRQ、Abort、Undefined 各模式，为每个模式单独设置栈指针；而内核真正想在一个统一的上下文里处理所有陷入，所以 Linux 的 ARM 移植在每个异常入口只往该模式的小栈里存三个字（r0、lr、spsr），随即切换到 SVC 模式，在进程的内核栈上继续处理——七种模式在软件里又被折回成“用户态”和“内核态”两种。另外，特权代码可以直接用 msr cpsr_c, #0xd3 改写 CPSR 进入任意模式，模式切换并不一定经过异常。

TrustZone 与虚拟化：再加两种模式。ARMv6K 和 ARMv7 引入安全扩展（TrustZone），处理器多了一个“安全/非安全”状态位，以及负责在两个世界之间切换的监控模式（Monitor），由 smc 指令进入。ARMv7 的虚拟化扩展（Cortex-A15 起）又加了一个 Hyp 模式，供虚拟机监控器使用，由 hvc 进入。为了说清楚这些模式之间的高低，ARMv7 手册开始使用特权级（PL0: User; PL1: 六种原有特权模式加 Monitor; PL2: Hyp）。到这时已经有九种模式，而“特权高低”和“用哪组寄存器”这两件事仍然搅在一起。

AArch64：把模式折叠成异常级别。ARMv8 的 AArch64 状态把这两件事拆开了。异常级别只表示特权的高低：EL0 对应原来的 User，EL1 一次吸收了 SVC、IRQ、FIQ、Abort、Undefined、System 六种模式，EL2 对应 Hyp，EL3 对应 Monitor。异常的类型不再决定模式，而由两样东西表达：

- 异常发生时跳到目标级别向量表里的哪一项。VBAR_ELx 指向的表有 16 项，按“来自哪一级、用哪个栈 × 同步/IRQ/FIQ/SError”排列，每项 128 字节，能直接放下一段保存现场的代码（3.2 节）；
- 同步异常的具体原因写进 ESR_ELx 的 EC 字段：svc 是 0x15，来自 EL0 的数据中止是 0x24，未定义指令是 0x00……原来要靠“当前处于 Abort 模式还是 Undefined 模式”来区分的事情，现在读一个寄存器就知道。

私有寄存器也大大精简：31 个通用寄存器不再分组，每一级只保留自己的栈指针 SP_ELx、返回地址 ELR_ELx 和保存状态 SPSR_ELx，FIQ 的快速寄存器组被取消。原来的 System 模式（特权级、但用 User 的寄存器）也有对应物：EL1t 在 EL1 下借用 SP_EL0，EL1h 使用自己的 SP_EL1（见下文 SPSR 的模式字段）。最重要的一点是：AArch64 里没有任何指令能直接改写当前级别，升级只能靠异常，降级只能靠 eret。

表 3.1: ARM9/ARMv7 处理器模式与 AArch64 异常级别

模式	CPSR.M	出现于	AArch64	何时进入（旧架构）
User (usr)	10000	ARMv4	EL0	应用程序运行
FIQ (fiq)	10001	ARMv4	EL1	快速中断；私有 r8~r14
IRQ (irq)	10010	ARMv4	EL1	普通中断
Supervisor (svc)	10011	ARMv4	EL1	复位、swi/svc 系统调用
Abort (abt)	10111	ARMv4	EL1	预取中止、数据中止
Undefined (und)	11011	ARMv4	EL1	未定义指令
System (sys)	11111	ARMv4	EL1	无对应异常，由特权代码直接切入
Monitor (mon)	10110	ARMv6K/v7	EL3	smc，安全/非安全世界切换
Hyp (hyp)	11010	ARMv7VE	EL2	hvc、虚拟化陷入

这张对应关系并不只是类比。ARMv8 处理器仍能在 EL0、EL1 上运行 32 位代码（AArch32 状态），这时旧的模式依旧存在，而且严格按表 3.1 落在对应的异常级别上；那些私有寄存器也被映射到 64 位通用寄存器上（例如 IRQ 模式的 r13/r14 就是 X17/X16），所以 64 位内核切换到 32 位进程时不需要另外保存它们。对比早期面向树莓派 1（ARMv6）的 xv6 移植：它要在启动时为 IRQ、Abort、Undefined 各设一个栈，并在每个入口切回 SVC 模式；本项目在 AArch64 上只需要每个核一个 SP_EL1，所有异常都在同一个 EL1 里处理。

异常级别只能靠异常来改变。 没有更高特权软件的参与，程序无法提升自己的级别——否则隔离就形同虚设。级别只在异常发生和返回时改变。发生异常（非法访存、svc 指令、硬件中断等）并由 EL_n 处理时，硬件依次做三件事：把返回地址存入 ELR_ELn ，把当时的处理器状态存入 $SPSR_ELn$ ，跳到异常向量；处理程序结束时执行 `eret`，硬件再从这两个寄存器恢复状态和地址。关键在于这两个寄存器都是可写的，处理程序不一定要“回到原处”。启动时降级正是利用这一点。

调试启动代码时，一个简单而有用的手段是读出 `CURRENTEL`：当前级别保存在它的第 [3:2] 位，右移 2 位即得 0~3。用不同的固件或 `config.txt` 启动时，读到的值可能是 3 或 2，下面的代码对两者都要处理：

`kernel/entry.S` 的第一件事就是读 `CURRENTEL`，不论从 $EL3$ 还是 $EL2$ 进入，都降到 $EL1$ ：

```
_entry:
    mrs x15, mpidr_el1
    and x15, x15, #0xff           // 记下核号，后面区分主核与副核
    mrs x0, CurrentEL
    cmp x0, #0xc                  // CurrentEL[3:2] == 3 ?
    b.ne 2f
    mov x0, #(1 << 31)             // HCR_EL2.RW: EL1 使用 AArch64
    msr hcr_el2, x0
    msr cntvoff_el2, xzr           // 虚拟计数器偏移清零
    mov x0, #3
    msr cnthctl_el2, x0            // 允许 EL1 访问物理计数器/定时器
    mov x0, #0x431                // SCR_EL3: RES1 | RW | NS
    msr scr_el3, x0
    mov x0, #0x3c5                // 目标 EL1h，屏蔽 D/A/I/F
    msr spsr_el3, x0
    adr x0, 3f
    msr elr_el3, x0
    eret
2:  cmp x0, #0x8                  // EL2 路径与上面相同，只是写 *_el2
    ...
    eret
3:  ldr x0,=((3 << 28) | (3 << 22) | (1 << 20) | (1 << 11))
    msr sctlr_el1, x0              // 只保留 RES1 位：MMU、缓存先全部关闭
    isb
    cbnz x15, secondary_wait      // 副核去等待
    bl early_mini_uart            // 主核点亮串口
    b entry
```

“降级”的方式很有意思：处理器没有一条“切换到 $EL1$ ”的指令，只能伪造一次异常返回。把目标状态写进 $SPSR_ELx$ 、把返回地址写进 ELR_ELx ，然后执行 `eret`，CPU 就会“返回”到 $EL1$ 的标号 3 处。表 3.2 列出了涉及的寄存器。

表 3.2: 降级到 $EL1$ 时设置的系统寄存器

寄存器	值	含义
HCR_EL2	bit 31	RW=1：下一级 $EL1$ 以 AArch64 运行，否则会掉进 AArch32
CNTVOFF_EL2	0	虚拟计数器 = 物理计数器，内核用的是虚拟定时器
CNTHCTL_EL2	3	$EL1/EL0$ 可以访问物理计数器和物理定时器
SCR_EL3	0x431	非安全状态，下一级为 AArch64
SPSR_ELx	0x3c5	M[3:0]=0101 即 $EL1h$ （使用 SP_EL1 ）；DAIF 四位全部置 1
SCTLR_EL1	RES1 位	不继承固件留下的 MMU/缓存状态

SPSR 中的 D、A、I、F 四位分别屏蔽调试异常、SError、IRQ 和 FIQ。启动期间必须全部屏蔽：此时既没有异常向量表，也没有栈，任何一个中断都会让 CPU 跳到不可预知的地方。

SPSR 里保存的处理器状态。 写入 SPSR_ELx 的值就是 eret 之后的 PSTATE，它由三部分组成：条件标志 N、Z、C、V（上一次运算结果是否为负、为零、无符号溢出、有符号溢出，条件分支指令据此跳转）；中断屏蔽位 D、A、I、F；以及模式字段 M[3:0]，它决定返回到哪一级、使用哪个栈指针。EL1 有两种栈模式：EL1h 使用 EL1 自己的 SP_EL1，EL1t 借用 EL0 的 SP_EL0。内核总是使用 EL1h，这样陷入内核时自动换到内核栈，用户栈指针原样留在 SP_EL0 中。有的教学内核只屏蔽 A、I、F 三位（值 $7 < 6$ ），本项目连同调试异常 D 一起屏蔽（ $0xf < 6$ ）。

SCTLR_EL1 的初值。 SCTLR_EL1 控制 EL1 下的 MMU、数据缓存、指令缓存和字节序。手册把其中若干位标为 RES1（保留，必须写 1），即上面那个 $(3 < 28) | (3 < 22) | (1 < 20) | (1 < 11)$ ；其余位全写 0，含义是：EL1 和 EL0 的数据访问都是小端，指令缓存、数据缓存和 MMU 全部关闭。不继承固件留下的任何状态，后面的代码才有确定的起点。

HCR_EL2 与 SCR_EL3。 即使不实现虚拟机监控器，也必须设置 HCR_EL2 的 RW 位，因为它决定 EL1 运行在 AArch64 还是 AArch32；同理，SCR_EL3 的 RW 位决定 EL2 的执行状态，NS 位让所有更低的级别处于非安全状态。漏掉 RW 位，eret 之后处理器会以 32 位状态执行 64 位指令，表现为“毫无输出地死掉”。

3.1.4 不依赖任何环境的早期串口

普通的 printf 依赖一长串设施：有效的栈、清零的 BSS、页表、内存分配器、自旋锁和控制台驱动。如果内核在这些设施就绪之前挂掉，串口上什么也看不到，连“死在哪一步”都无从判断。项目在真机上调通的第一个关键是：用纯汇编编写一个不需要栈、不需要内存、不需要 C 运行环境的串口输出。

内存映射的设备寄存器。 BCM2837 上所有外设都通过内存映射寄存器访问：树莓派 3 把 $0x3F000000$ 以上的一段物理地址留给外设，每个寄存器是 32 位，读写某一位就是配置或启动设备的某项功能。Broadcom 手册中的地址以 $0x7E$ 开头，那是 VideoCore 总线上的地址，ARM 一侧对应的物理地址要换成 $0x3F$ 开头（第 4 章）。

树莓派 3 的 GPIO14/15 可以连到 PL011 UART0 或 AUX Mini UART。本内核使用 Mini UART，即 GPIO14/15 的 ALT5 功能。树莓派有两个 UART：PL011 功能完整，但在树莓派 3 上默认被板载蓝牙占用；Mini UART 结构简单，只需几个寄存器就能工作，代价是它的波特率依赖 VPU 核心时钟。初始化分三步：

第一步：选择 GPIO 复用功能。 大多数 GPIO 引脚可以连接不同的外设，每个引脚有 0~5 共六种“复用功能”，用 3 位编码，十个引脚共用一个 GPFSELn 寄存器。GPIO14、15 由 GPFSEL1 的第 14:12 位和 17:15 位控制；它们的 ALT5 分别是 Mini UART 的 TXD1 和 RXD1，ALT5 的编码是 010。所以要把这两个 3 位字段清零，再各写入 2。注意 GPIO 编号与排针脚号不同：GPIO14 是第 8 脚，GPIO15 是第 10 脚，第 6 脚是地。

第二步：关闭内部上下拉。 一个悬空的输入引脚读到的值是随机的；上拉或下拉电阻让它在悬空时稳定地读到 1 或 0。串口两根线始终接着对端，不需要上下拉，而且上下拉状态在重启后仍然保持，所以使用前要显式撤销。BCM2837 改变上下拉状态需要一个带时钟的握手过程：先在 GPPUD 中写入目标状态（0 表示都不要），等待约 150 个周期让控制信号建立；再在 GPPUDCLK0 中把要修改的引脚对应位写 1，把控制信号“打”进这些引脚，再等 150 个周期保持；最后清除 GPPUD 和 GPPUDCLK0。只有收到时钟的引脚会改变，其余引脚保持原状。

第三步：配置 Mini UART。 顺序很重要：先在 AUX 使能寄存器中打开 Mini UART，因为在此之前它的其他寄存器都不可访问；配置期间先关闭收发器，并永久关闭自动流控（它需要额外的 RTS/CTS 引脚，普通 USB-TTL 线不支持）；关闭收发中断（早期阶段没有中断处理）；把数据位设为 8 位——手册写的是第 0 位，实际硬件要第 0、1 两位都置 1，即写 3；RTS 置为常高；写波特率除数；最后打开收发器。

early_mini_uart 依次完成：

```
early_mini_uart:
    mov x13, x30                // 没有栈，用寄存器保存返回地址
    ldr x9, =0x3f200004          // GPFSEL1: GPIO14/15 = ALT5 (功能码 2)
    ldr w10, [x9]
    ldr w11, =0x0003f000
    bic w10, w10, w11
    ldr w11, =0x00012000
    orr w10, w10, w11
    str w10, [x9]
    ...                          // GPPUD/GPPUDCLKO 时序：关闭内部上下拉
    ldr x9, =0x3f215000          // AUX 基址
    ...                          // 使能 AUX、8N1、清 FIFO
    mov w10, #270
    str w10, [x9, #0x68]         // AUX_MU_BAUD: 115200 @ 250 MHz
    mov w10, #3
    str w10, [x9, #0x60]         // 打开收发
    mov w12, #'E'
    bl early_mini_uart_putc
    ...
```

这里用的都是物理地址，因为 MMU 还没打开。Mini UART 的波特率公式是

$$\text{baud} = \frac{f_{\text{core}}}{8(\text{divisor} + 1)}, \quad \text{divisor} = \frac{250\,000\,000}{8 \times 115\,200} - 1 \approx 270.$$

它依赖 VPU 核心频率，所以 config.txt 必须固定 core_freq=250。

收发都用轮询：AUX_MU_LSR 的第 5 位为 1 表示发送器空闲，可以向 AUX_MU_IO 写一个字符；第 0 位为 1 表示收到数据，可以从 AUX_MU_IO 读出。早期汇编版只需要发送，所以 early_mini_uart_putc 只有三条指令：读 LSR、测第 5 位、写 IO。

在没有调试器的裸机上，串口输出几乎是了解程序内部状态的唯一途径。所以一个裸机项目通常要做的第二件事，就是在串口之上移植一个小型 printf——只需要提供一个发送单个字符的函数。本项目把这件事分成两级：汇编阶段只能输出单个标记字符，C 环境就绪后才由控制台驱动提供完整的 printf。

此后，启动代码在每个关键节点输出一个字符，形成一串“启动指纹”（表 3.3）。真机串口上看到的最后一个字符，就指明了内核停在哪一步。

表 3.3: 启动标记字符

标记	含义	标记	含义
E	进入 EL1，物理地址串口可用	1	kinit1() 完成
B	BSS 清零完成	2	最终内核页表建立
P	启动页表与系统寄存器就绪	3	切换到最终页表
H	MMU 已开，跳到高地址并建好栈	4	kinit2() 完成
O	进入 main()	a~f	kalloc.c 内部细分

正常启动时串口先打出 E，然后是 BPH0abdefc1234，接着才是 printf 的第一行“xv6 kernel is booting”。第 4 章会讲到一次停在 BPH0abde 的故障：它说明第一次 kfree() 已经完成 memset，却没能拿到 kmem.lock。

MMU 打开之后，物理地址 0x3f215000 不再可用。boot_uart_mark() 改用高地址 KERNBASE + 0x3f215000，所以启动页表里专门为 Mini UART 所在的 2 MiB 块加了一个设备映射。

3.2 异常和中断

与设备打交道的大多数场合都是异步的：内核向设备发出一条命令，设备并不立即完成，而是在工作结束后“通知”处理器。这种通知打断了正在执行的指令流，迫使处理器转去执行一段处理程序，所以叫“中断”。对操作系统来说，最重要的中断源是定时器：调度器靠周期性的定时器中断度量每个进程运行了多久，并在时间片用完时把 CPU 交给别人。

本节沿着一次中断从发生到返回的路径，依次讲解 xv6-aarch64 的实现：`kernel/trapasm.S`（向量表与现场保存）、`kernel/trap.c`（分发）、`kernel/bcm2837.c`（中断控制器）、`kernel/timer.c`（定时器）和 `kernel/spinlock.c`（中断屏蔽的嵌套）。

3.2.1 中断与异常

在 ARMv8 中，中断只是“异常”的一种。异常分为四类，表 3.4 列出了它们在 xv6-aarch64 中的用途。

表 3.4: 四类异常在 xv6-aarch64 中的用途

类型	性质	在 xv6-aarch64 中
同步异常	由当前指令引起	用户态 <code>svc #0</code> 即系统调用；用户态访存错误时打印诊断并杀死进程；内核态同步异常直接 <code>panic</code>
IRQ	外部硬件产生，与当前指令无关	通用定时器、Mini UART、DWC2 USB、Arasan SDIO、Unicam CSI、核间中断 (IPI)
FIQ	高优先级的“快速中断”	不使用
SError	外部产生的系统错误	不处理，进入即停机

同步异常的具体原因记录在 `ESR_EL1`（异常综合征寄存器）中：高 6 位 [31:26] 是异常类别 EC，低 25 位 ISS 是该类别下的细节；出错的指令地址在 `ELR_EL1`，出错的数据地址在 `FAR_EL1`。xv6 只识别一种“正常”的同步异常——`EC = 0x15 (21)`，即来自 AArch64 EL0 的 `svc`；其余同步异常都按错误处理。内核态的同步异常不尝试恢复：

```
void kerneltrap()
{
    uint64 esr = r_esr_el1();
    w_esr_el1(0);
    if(intr_get() != 0)
        panic("kerneltrap: interrupts enabled");
    uint64 ec = (esr >> 26) & 0x3f;
    uint64 iss = esr & 0x1ffffff;
    printf("esr %p %p %p\n", esr, ec, iss);
    printf("elr=%p far=%p\n", r_elr_el1(), r_far_el1());
    panic("kerneltrap");
}
```

3.2.2 异常向量

每类异常都要有自己的处理入口；而且同一类异常，从不同的执行状态进入时也要用不同的入口。以运行在 EL1 的内核为视角，进入异常时的执行状态有四种：EL1t（EL1 使用 `SP_EL0`）、EL1h（EL1 使用自己的 `SP_EL1`，内核正是这种模式）、来自 64 位 EL0、来自 32 位 EL0。四类异常乘以四种状态，共 16 个入口，它们排成一张异常向量表：表必须 2 KiB 对齐，每个入口占 0x80 字节（表 3.5）。

每组的第三、四个入口是 FIQ 和 SError。`kernel/trapasm.S` 用 `.balign` 保证对齐，每个入口只放一条跳转指令，真正的处理放在表外（0x80 字节只够放 32 条指令）：

表 3.5: VBAR_EL1 向量表布局

偏移	来源	同步	IRQ	本内核
0x000	当前 EL, 使用 SP_EL0	+0x000	+0x080	未用 (b .)
0x200	当前 EL, 使用 SP_ELx	el1sync	el1irq	内核态异常/中断
0x400	低 EL, AArch64	el0sync	el0irq	系统调用、用户异常/中断
0x600	低 EL, AArch32	+0x600	+0x680	未用

```

.global alltraps
.balign 0x800
alltraps:
// Current EL with sp0
    b .
.balign 0x80
    b .
.balign 0x80
    b .
.balign 0x80
    b .

// Current EL with spx
.balign 0x80
    b el1sync
.balign 0x80
    b el1irq
.balign 0x80
    b .
.balign 0x80
    b .

// Lower EL using aarch64
.balign 0x80
    b el0sync
.balign 0x80
    b el0irq
.balign 0x80
    b .
.balign 0x80
    b .

// Lower EL using aarch32
...                // 四个 b .

```

只有四个入口真正被使用。其余 12 个入口是 b .，一旦进入就原地停住。这在调试初期足够了，但也有缺点：FIQ 或 SError 发生时，现象只是“毫无输出地死机”。更好的做法是让这些入口也保存现场，然后打印入口编号、ESR_EL1 和 ELR_EL1 再停机。

3.2.3 保存寄存器状态

处理完异常回到原来的代码时，所有通用寄存器必须和异常发生前一模一样，否则一个与当前代码毫不相干的中断就会悄悄改变它的行为。进入异常时，硬件只做三件事：把返回地址存入 ELR_EL1，把原来的 PSTATE 存入 SPSR_EL1，换到 SP_EL1。通用寄存器要由软件自己保存。

`trapasm.S` 为两种来源准备了两对宏。来自内核的异常用 `savereg/restorereg`，现场压在当前内核栈上：

```
.macro savereg
    sub sp, sp, #272          // 17 组 × 16 字节
    stp x0, x1, [sp, #16 * 0]
    stp x2, x3, [sp, #16 * 1]
    ...
    stp x28, x29, [sp, #16 * 14]
    mrs x9, elr_el1
    mrs x10, spsr_el1
    add x11, sp, #272         // 异常发生前的内核 sp
    stp x30, x9, [sp, #16 * 15]
    stp x10, x11, [sp, #16 * 16]
.endm
```

来自用户态的异常用 `usavereg/urestorereg`，区别是最后一个字保存的是用户栈指针 SP_EL0，返回时也要写回：

```
.macro usavereg
    sub sp, sp, #272
    stp x0, x1, [sp, #16 * 0]
    ...
    stp x28, x29, [sp, #16 * 14]
    mrs x9, elr_el1
    mrs x10, spsr_el1
    mrs x11, sp_el0          // 用户栈指针
    stp x30, x9, [sp, #16 * 15]
    stp x10, x11, [sp, #16 * 16]
.endm

.macro urestorereg
    ldp x30, x9, [sp, #16 * 15]
    ldp x10, x11, [sp, #16 * 16]
    msr elr_el1, x9
    msr spsr_el1, x10
    msr sp_el0, x11
    ldp x0, x1, [sp, #16 * 0]
    ...
    ldp x28, x29, [sp, #16 * 14]
    add sp, sp, #272
.endm
```

这 272 字节的布局与 `struct trapframe` (`x0~x30`、`elr`、`spsr`、`sp`) 完全一致。`allocproc()` 把每个进程的 `trapframe` 放在它内核栈的最顶端，而用户态陷入时内核栈恰好是空的，所以 `usavereg` 压栈的位置正好就是 `p->trapframe`。`trap.c` 开头的注释画出了两种现场同时存在时一个内核栈的样子：

```

低地址  p->kstack
        | 普通内核栈空间
        | +-----+ <- 内核态中断后 sp = S - 272
        | | el1irq savereg 临时现场 |
        | | x0..x30, elr, spsr, sp |
        | +-----+ <- 中断前 sp = S
        | 函数调用帧、局部变量……
        +-----+
        | struct trapframe | <- p->trapframe (usavereg 的位置)
        | x0..x30, elr, spsr, 用户 sp |
高地址  p->kstack + PGSIZE

```

也就是说：进程在内核里执行系统调用时又被中断，会在同一个内核栈上再压一层 `savereg` 现场；如果中断发生在调度器中，用的则是这个 CPU 自己的调度器栈（`stack0`）。

有了调度器之后，“以一个进程的身份进入中断、以另一个进程的身份离开”是完全合法的：`eret` 依赖的 `elr` 和 `spsr` 都保存在当前进程自己的内核栈上，所以 `ELR_EL1`、`SPSR_EL1` 必须和通用寄存器一起保存和恢复。新进程第一次回到用户态走的是 `usertrapret`，它直接把 `sp` 指向 `trapframe`，借用 `el0irq` 的后半段返回：

```

el0irq:
    usavereg
    mov x0, sp
    bl userirq
trapret:
    urestorereg
    eret

.global usertrapret
usertrapret:                // void usertrapret(struct trapframe *tf)
    mov sp, x0
    b trapret

```

3.2.4 设置向量表

向量表准备好了，还要告诉处理器它在哪里：把地址写入 `VBAR_EL1`（向量基址寄存器）。这是每个核各自的系统寄存器，所以每个 CPU 都要执行一次：

```

void trapinithart(void)
{
    w_vbar_el1((uint64)alltraps);
}

static inline void w_vbar_el1(uint64 x)    // kernel/aarch64.h
{
    asm volatile("msr vbar_el1, %0" : : "r" (x) );
}

```

`alltraps` 是链接时确定的高虚拟地址，所以这一步必须在 MMU 打开、跳到高地址之后进行。主核在 `main()` 中依次调用 `trapinit()`（只初始化 `tickslock`）和 `trapinithart()`；副核被唤醒后只调用 `trapinithart()`。

3.2.5 屏蔽与解除屏蔽中断

有些代码绝不能被中断打断。例如 `usavereg` 执行到一半时若又来一个中断，现场就会被覆盖。所以硬件在进入任何异常时都会自动屏蔽中断；保存好现场之后再解除屏蔽、允许嵌套，也是完全合法的。

PSTATE 中有四个屏蔽位：D（调试异常）、A（SError，也叫异步中止）、I（IRQ）、F（FIQ），用 `msr daifset/daifclr`，`#imm` 设置或清除，立即数是一个 4 位掩码，从高到低对应 D、A、I、F。xv6 的封装在 `kernel/aarch64.h` 中：

```
static inline void intr_on() { asm volatile("msr daifclr, #0xf" ::: "memory"); }
static inline void intr_off() { asm volatile("msr daifset, #0xf" ::: "memory"); }

static inline int intr_get()          // IRQ 是否打开？
{
    uint64 x = daif();
    return ((x >> 6) & 0x2) == 0;    // DAIF 寄存器中 I 在 bit 7
}
```

注意这里用的是 `#0xf`，一次改变全部四位，而不只是 I 位（`#2`）。对 xv6 来说效果相同，因为它只用 IRQ；更精确的写法应只操作 I 位，以免意外打开调试异常或 SError。

3.2.5.1 自旋锁与中断的嵌套

如果一个 CPU 持有某把自旋锁时被中断，而中断处理程序又去获取同一把锁，这个 CPU 就会永远自旋下去。所以 xv6 规定：持有任何自旋锁期间，本 CPU 的中断都是关闭的。`acquire()` 调用 `push_off()`，`release()` 调用 `pop_off()`，二者成对嵌套计数：

```
void push_off(void)
{
    int old = intr_get();
    intr_off();
    if(mycpu()->noff == 0)
        mycpu()->intena = old;    // 只记录最外层进入前的状态
    mycpu()->noff += 1;
}

void pop_off(void)
{
    struct cpu *c = mycpu();
    if(intr_get())
        panic("pop_off - interruptible");
    if(c->noff < 1)
        panic("pop_off");
    c->noff -= 1;
    if(c->noff == 0 && c->intena)    // 最外层释放、且原来是开着的，才重新打开
        intr_on();
}
```

3.2.5.2 中断在哪里被打开

- 启动时，`entry.S` 通过 `SPSR = 0x3c5` 进入 EL1，四位全部屏蔽；
- 每个 CPU 第一次真正接收中断，是在 `scheduler()` 循环中调用 `intr_on()`；
- 用户态陷入后，`usertrap()` 识别出系统调用，在调用 `syscall()` 之前打开中断，让长时间的系统调用也能被设备中断和定时器打断；返回用户态前再 `intr_off()`，因为 `urestorereg` 期间不能被打断；
- `kerneltrap()` 进入时检查 `intr_get()`，若中断是开着的就说明某处逻辑出错，直接 `panic`。

3.2.6 配置中断控制器

设备一般不直接中断 CPU，而是经过中断控制器：控制器负责打开或关闭各个中断源，并告诉 CPU 当前是谁在请求中断。BCM2837 没有 ARM 标准的 GIC，而是两个级联的模块（图 3.1）：

- **legacy 中断控制器** (0x3f00b000) 管理 SoC 共享外设：Mini UART (IRQ 29)、DWC2 USB (IRQ 9)、Arasan SDIO (IRQ 62)、Unicam CSI1 (IRQ 39) 等。中断号 0 ~ 31 由 `ENABLE_IRQS_1/IRQ_PENDING_1` 管理，32 ~ 63 由 `ENABLE_IRQS_2/IRQ_PENDING_2` 管理。所有外设中断汇总成一根“GPU IRQ”；
- **ARM-local 中断路由** (0x40000000) 位于 CPU 簇旁，管理每个核私有的中断源：通用定时器、核间 mailbox、PMU，以及从 legacy 控制器汇总来的 GPU IRQ，最终连到四个核的 IRQ/FIQ 输入。

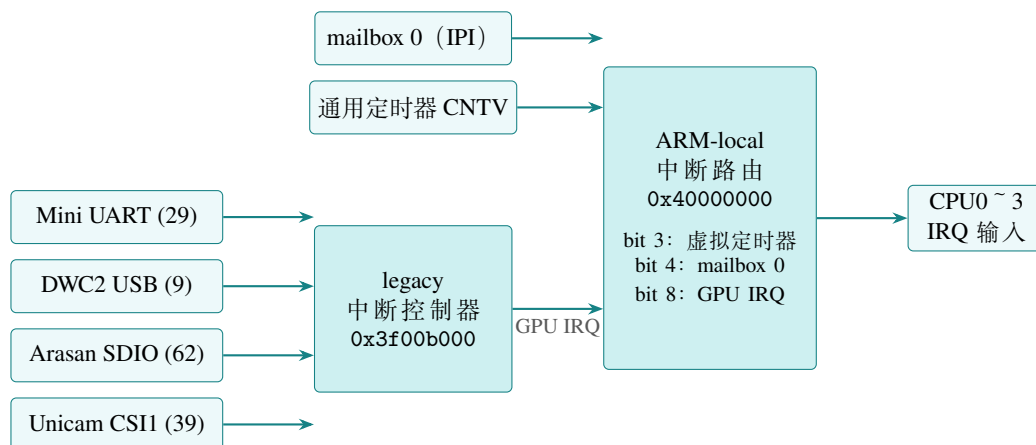


图 3.1: BCM2837 的两级中断结构

为了少改 `trap.c`，项目保留了原 GIC 版本的函数名 `gicv3init()`、`gicv3inithart()`、`gic_iar()`、`gic_eoi()`，实现换成了 `kernel/bcm2837.c`。全局初始化先关掉 legacy 控制器的全部中断，再只打开已经有驱动的几个：

```
void gicv3init(void)
{
    irqwrite(DISABLE_IRQS_1, ~0U);
    irqwrite(DISABLE_IRQS_2, ~0U);
    irqwrite(DISABLE_BASIC_IRQS, ~0U);
    irqwrite(ENABLE_IRQS_1, (1U << UART0_IRQ) | (1U << DWC2_IRQ));
    irqwrite(ENABLE_IRQS_2, 1U << (SDIO_IRQ - 32));
}
```

后来加入的驱动（如摄像头）在 `probe` 成功后调用 `bcm2837_enable_irq(irq)` 打开对应位，按中断号小于还是大于 32 选择 `ENABLE_IRQS_1` 或 `_2`。

每个核还要各自配置 ARM-local 路由：把本核的虚拟定时器接到 IRQ，并打开 mailbox 0 作为“重新调度”核间中断：

```

#define CORE_TIMER_CONTROL(c) (0x40 + 4 * (c))
#define CORE_MBOX_CONTROL(c) (0x50 + 4 * (c))
#define CORE_IRQ_SOURCE(c) (0x60 + 4 * (c))
#define CORE_MBOX0_SET(c) (0x80 + 0x10 * (c)) // 写 1 发出
#define CORE_MBOX0_CLR(c) (0xc0 + 0x10 * (c)) // 写 1 清除
#define CORE_VIRTUAL_TIMER (1 << 3)
#define CORE_MBOX0 (1 << 4)

void gicv3inithart(void)
{
    int cpu = cpuid();
    localwrite(CORE_TIMER_CONTROL(cpu), CORE_VIRTUAL_TIMER);
    localwrite(CORE_MBOX0_CLR(cpu), ~0U); // 先清掉遗留的 mailbox 位
    localwrite(CORE_MBOX0_CONTROL(cpu), 1);
}

void send_resched_ipi(int cpu) // 让 cpu 尽快重新调度
{
    asm volatile("dsb sy" ::: "memory"); // 先让 need_resched 的写入可见
    localwrite(CORE_MBOX0_SET(cpu), 1);
}

```

一个外设中断要真正到达 CPU，三道开关缺一不可：设备自己的中断使能位（例如 Mini UART 的 MU_IER）、legacy 控制器的 ENABLE_IRQS 位，以及 CPU 的 PSTATE.I = 0。

3.2.7 通用 IRQ 处理程序

所有 IRQ 都进入同一个入口，按来源分为两条路径：

```

el1irq: // 内核执行时来的 IRQ
    savereg
    mov x0, sp
    bl kernelirq
    restorereg
    eret

el0irq: // 用户执行时来的 IRQ
    usavereg
    mov x0, sp
    bl userirq

trapret:
    urestorereg
    eret

```

两个 C 处理函数都先调用 devintr() 找出并服务中断源，再决定是否让出 CPU：

```

void userirq(void)
{
    struct proc *p = myproc();
    devintr();
}

```

```

if(resched_pending())      // 时间片用完，或更高优先级进程就绪（定时器或 IPI）
    yield();
if(p->killed)
    exit(-1);
intr_off();
}

void kernelirq()
{
    devintr();
    // 内核态也可以被抢占：只有不持任何自旋锁时中断才是开着的
    if(myproc() != 0 && myproc()->state == RUNNING && resched_pending())
        yield();
}

```

devintr() 先问中断控制器“是谁”，再调用对应设备的处理函数，返回 2 表示定时器、1 表示其他设备、0 表示无法识别：

```

int devintr()
{
    uint32 iar = gic_iar();
    uint32 irq = gic_iar_irq(iar);
    int dev = 0;

    if(irq == UART0_IRQ){      uartintr();      dev = 1; }
    else if(irq == DWC2_IRQ){   dwc2_irq();      dev = 1; }
    else if(irq == SDIO_IRQ){   arasan_sdio_irq(); dev = 1; }
    else if(irq == CSI1_IRQ){   unicam_irq();    dev = 1; }
    else if(irq == TIMER0_IRQ){
        int logical_tick = timerintr();
        if(cpuid() == 0 && logical_tick)
            clockintr();
        sched_tick();
        dev = 2;
    } else if(irq == IPI_RESCHED_IRQ){
        resched_ipi_ack();      // need_resched 已由发送方设置
        dev = 1;
    } else if(irq == 1023){
        // 没有可处理的中断
    } else if(irq){
        printf("unexpected interrupt irq=%d\n", irq);
    }
    if(dev)
        gic_eoi(iar);
    return dev;
}

```

“是谁”的判断在 gic_iar() 中。它先查本核的 ARM-local 源寄存器（定时器和 IPI 是每核私有的），再查 legacy 控制器的挂起寄存器，把两套硬件的状态翻译成一个类似 GIC 的中断号：

```
uint32 gic_iar(void)
{
    int cpu = cpuid();
    uint32 local = localread(CORE_IRQ_SOURCE(cpu));
    if(local & CORE_VIRTUAL_TIMER) return TIMER0_IRQ; // 27
    if(local & CORE_MBOX0) return IPI_RESCHEDED_IRQ; // 伪中断号 1000
    if(irqread(IRQ_PENDING_1) & (1U << UART0_IRQ)) return UART0_IRQ;
    if(irqread(IRQ_PENDING_1) & (1U << DWC2_IRQ)) return DWC2_IRQ;
    if(irqread(IRQ_PENDING_2) & (1U << (SDIO_IRQ - 32))) return SDIO_IRQ;
    if(irqread(IRQ_PENDING_2) & (1U << (CSI1_IRQ - 32))) return CSI1_IRQ;
    return 1023;
}
```

BCM2837 没有 GIC 那样的 EOI 握手，gic_eoi() 是空函数：中断要在源设备处清除——定时器靠重新装载，UART 靠读空接收 FIFO，USB 靠写 HCINT，IPI 靠写 mailbox 清除寄存器。设备的挂起位清除之后，legacy 控制器中对应的位才会消失。多个中断可能同时挂起，devintr() 每次只处理优先级最高的一个；其余的会在 eret 之后立即再次触发中断，所以不会丢失。

3.2.7.1 同步异常：系统调用路径

同步异常走 el0sync/el1sync 入口，保存现场的方式与 IRQ 相同，只是调用的 C 函数不同。用户程序执行 svc #0 时，系统调用号放在 x7，参数在 x0 ~ x5：

```
user: mov x7, #SYS_write ; svc #0
-> VBAR_EL1 + 0x400 -> el0sync -> usavereg (现场 = trapframe)
-> usertrap(): ESR_EL1.EC == 0x15
    -> intr_on(); syscall(): syscalls[x7](), 返回值写回 trapframe->x0
    -> 若唤醒了更高优先级进程: yield()
-> intr_off(); urestorereg -> eret -> 回到 svc 的下一条指令
```

```
void usertrap(void)
{
    struct proc *p = myproc();
    uint64 esr = r_esr_el1();
    uint64 ec = (esr >> 26) & 0x3f;
    w_esr_el1(0);
    if(ec == 21){ // SVC from AArch64
        if(p->killed) exit(-1);
        intr_on();
        syscall();
        if(resched_pending()) yield();
    } else {
        printf("usertrap(): unexpected esr=%p ec=%p pid=%d\n", esr, ec, p->pid);
        printf("    elr=%p far=%p\n", r_elr_el1(), r_far_el1());
        ...
        uvmdump(p->vm->pagetable, p->pid, p->name, "usertrap");
        p->killed = 1;
    }
    if(p->killed) exit(-1);
}
```

```
intr_off();
}
```



硬件保存在 ELR_EL1 中的已经是 svc 的下一条指令地址，所以不需要像 RISC-V 那样手工给 PC 加 4。

笔记 早期版本的 `usertrap()` 先读出 EC 再清零 ESR_EL1，打印错误信息时又重新读一次寄存器，结果永远显示 `ec=0`。现在的代码先把完整的 ESR 存进局部变量 `esr`。调试信息本身的正确性，同样需要检查。

3.2.8 初始化定时器

xv6-aarch64 没有使用 BCM 自己的 System Timer，而是使用 ARM 架构定义的通用定时器（Generic Timer）。它的好处是：每个核都有自己的比较器，中断经 ARM-local 的 bit 3 直接送到本核，不经过 legacy 控制器；寄存器是系统寄存器，用 `mrs/msr` 访问，不需要 MMIO 映射。内核使用虚拟定时器那一组：

- CNTFRQ_ELO：计数器频率（由固件设置）；
- CNTVCT_ELO：当前虚拟计数值，所有核共享，可用来测量时间（`clock_us()`、`delay()` 都基于它）；
- CNTV_TVAL_ELO：距离触发还剩多少个计数，写入即开始倒计时；
- CNTV_CTL_ELO：bit 0 ENABLE、bit 1 IMASK、bit 2 ISTATUS。

`timerinit()` 的顺序是“关闭—装载—打开”，避免改写比较值的过程中产生一个中间中断：

```
#define CNTV_CTL_ENABLE    (1<<0)
#define CNTV_CTL_IMASK    (1<<1)

void timerinit()
{
    disable_timer();
    reload_timer();
    enable_timer();
}

static void disable_timer()
{
    uint64 c = r_cntv_ctl_el0();
    c &= ~CNTV_CTL_ENABLE;
    c |= CNTV_CTL_IMASK;
    w_cntv_ctl_el0(c);
}

static void reload_timer()
{
    uint64 interval = 10000;                // 微秒，即 10 ms
    uint64 interval_clk = interval * (r_cntfrq_el0() / 1000000);
    w_cntv_tval_el0(interval_clk);
}

static void enable_timer()
{
    uint64 c = r_cntv_ctl_el0();
    c |= CNTV_CTL_ENABLE;
    c &= ~CNTV_CTL_IMASK;
    w_cntv_ctl_el0(c);
}
```

```
}

```

`timerinit()` 由每个核各自调用。执行完之后定时器已经开始倒计时，但 `PSTATE.I` 仍然屏蔽着 `IRQ`，要等进入 `scheduler()` 调用 `intr_on()` 时 CPU 才会真正接收它。

3.2.9 处理定时器中断

定时器到期后，中断经 ARM-local 路由到本核，`gic_iar()` 返回 `TIMER0_IRQ`，`devintr()` 调用 `timerintr()`：

```
int timerintr()
{
    int cpu = cpuid();
    int logical_tick;

    disable_timer();
    reload_timer();                // 重新装载即清除了中断条件
    timer_divider[cpu]++;
    logical_tick = timer_divider[cpu] == 10;
    if(logical_tick)
        timer_divider[cpu] = 0;
    if(cpu == 0)
        workqueue_timer_tick();    // 推进 delayed-work 时钟
    enable_timer();
    return logical_tick;           // 每 10 次 (100 ms) 返回 1
}
```

一次 10ms 的硬件中断在 `devintr()` 中引出三件事：

1. **系统时间**：每 10 次中断构成一个 xv6 逻辑 tick。只有 CPU0 调用 `clockintr()` 把全局 `ticks` 加一并 `wakeup(&ticks)`，避免四个核把时间推进四倍。`sleep()`、`uptime()` 因此保持 xv6 原来 100ms 一个 tick 的语义：

```
void clockintr()
{
    acquire(&tickslock);
    ticks++;
    wakeup(&ticks);
    release(&tickslock);
    net_udp_timeout_tick();        // 检查带超时的 UDP 接收是否到期
}
```

2. **下半部时钟**：CPU0 推进 workqueue 的 delayed-work 时钟，TCP 重传、Wi-Fi 连接超时等到期的工作被放进各自的工作队列，由内核线程执行，而不是在中断里直接运行；
3. **调度节拍**：每次中断都调用 `sched_tick()`，扣减当前进程的时间片、累计实时任务的运行时间，必要时设置 `need_resched`（第 5 章）。随后 `userirq()/kernelirq()` 检查 `resched_pending()`，决定是否 `yield()`。

`reload_timer()` 写的是相对倒计时，所以下一次中断发生在“本次处理程序重新装载的时刻 + 10ms”，处理程序的延迟会累积成时钟漂移。若要更稳定的时基，可以改写绝对比较值 `CNTV_CVAL_ELO`，每次在上一个截止时间上加固定间隔。

3.2.10 小结

把本节的代码按 `main()` 中的调用顺序串起来：

```
trapinit();           // 初始化 tickslock
trapinithart();       // VBAR_EL1 = alltraps                (每个核)
gicv3init();          // legacy 控制器：全关，再打开 UART/USB/SDIO
gicv3inithart();      // ARM-local：本核虚拟定时器 + mailbox 0 (每个核)
timerinit();          // 10 ms 虚拟定时器开始倒计时        (每个核)
...
scheduler();          // intr_on()：第一次真正接收中断
```

顺序不能随意交换：先安装向量表，保证中断来了有入口；再配置控制器和路由；再启动定时器；最后才解除 `PSTATE` 的屏蔽。要收到第一个定时器中断，下面这些条件必须同时满足：

- 处于 `EL1h`，且 `EL2` 已允许 `EL1` 访问计数器和定时器 (`CNTHCTL_EL2`)；
- `VBAR_EL1` 指向 2 KiB 对齐的 `alltraps`；
- `CORE_TIMER_CONTROL(cpu)` 打开了虚拟定时器路由；
- `CNTV_TVAL_ELO` 装入了非零值，`CNTV_CTL_ELO` 的 `ENABLE = 1`、`IMASK = 0`；
- `PSTATE.I = 0`，且当前内核栈有效，有空间保存 272 字节的现场。

调试中断问题时，可以在启动阶段打印 `CURRENTEL`、`VBAR_EL1`、`DAIF`、`CNTFRQ_ELO` 和 `CNTV_CTL_ELO`；对未知中断打印 `IRQ_PENDING_1/2` 和 `CORE_IRQ_SOURCE`。但不要在定时器中断里持续 `printf`：串口远比中断慢，会改变调度时序，甚至带来新的锁问题。

第四章 MMU

本章内容

- ❑ 39 位虚拟地址与三级页表
- ❑ 硬件如何遍历页表、块映射
- ❑ 描述符格式与内存属性 (MAIR)
- ❑ 清 BSS、启动页表 (恒等映射加高地址映射)、TCR/MAIR/SCTLR、跳到高地址
- ❑ 多核启动：两道闸门
- ❑ 两阶段的物理内存初始化
- ❑ 最终内核页表与用户页表
- ❑ 第一个进程、fork 与地址空间切换；TLB；缺页与按需分配
- ❑ MMIO、总线地址与 DMA
- ❑ 三个缓存与 TLB 故障

4.1 AArch64 的地址翻译

4.1.1 地址空间划分

本内核使用 4 KiB 页、39 位虚拟地址。TCR_EL1 中 T0SZ=T1SZ= 25，即每个地址空间有 $2^{64-25} = 2^{39}$ 字节 (512 GiB)。AArch64 用虚拟地址的高位选择页表基址寄存器：

- 高位全 0：0x0000000000000000 ~ 0x0000007fffffffff，由 TTBR0_EL1 翻译，用于用户空间，每个进程一张；
- 高位全 1：0xffffffff8000000000 ~ 0xffffffffffffffff，由 TTBR1_EL1 翻译，用于内核空间，所有进程共享一张；
- 中间的地址不合法，访问即产生翻译错误。

两个基址寄存器分开带来一个很大的好处：进程切换时只需改写 TTBR0_EL1，内核映射保持不变，也不用像 xv6-riscv 那样在每个用户页表里复制一份内核映射和 trampoline 页。

4.1.2 三级页表

39 位地址用 4 KiB 粒度时，从第 1 级开始遍历，共三级（没有第 0 级）。每级页表占一页，含 512 个 8 字节表项，用虚拟地址中的 9 位索引（图 4.1）：

高位：全 0 或全 1	L1 索引 VA[38:30]	L2 索引 VA[29:21]	L3 索引 VA[20:12]	页内偏移 VA[11:0]
-------------	-----------------	-----------------	-----------------	---------------

每级 9 位 = 512 项；L1 一项管 1 GiB，L2 一项管 2 MiB，L3 一项管 4 KiB

图 4.1: 39 位虚拟地址的划分

kernel/aarch64.h 用两个宏取出各级索引：

```
#define PXMASK          0x1FF          // 9 位
#define PXSHIFT(level)  (39-(level)*9) // L1: 30, L2: 21, L3: 12
#define PX(level, va)   (((uint64)(va)) >> PXSHIFT(level)) & PXMASK
```

第 1、2 级表项可以是表描述符（指向下一级页表）或块描述符（直接映射 1 GiB/2 MiB 的连续物理内存）；第 3 级表项是页描述符。启动页表用 2 MiB 块，最终页表全部用 4 KiB 页。

4.1.3 硬件如何遍历页表

虚拟内存给每个进程一个错觉：整个地址空间都归它一个人用。进程发出的每个地址都是虚拟地址，由 MMU（内存管理单元）在访存的路上翻译成物理地址，进程本身完全察觉不到。这样做同时解决了两个问题：一是隔离，进程只能访问自己页表里映射了的内存，碰不到别人的数据，也碰不到内核；二是分配简单，每个进程的程
序都可以从地址 0 开始链接，不必关心物理内存里哪些地方已经被别人占了。

MMU 翻译一个地址的过程叫页表遍历（table walk），完全由硬件完成，软件只负责事先把页表在内存里摆好、再把根页表的物理地址写进 TTBR0_EL1 或 TTBR1_EL1。以用户地址为例：

1. 从 TTBR0_EL1 取出 L1 页表的物理地址；
2. 用 VA[38:30] 作下标读出 L1 表项。若它是表描述符，其中 [47:12] 位就是 L2 页表的物理地址；
3. 用 VA[29:21] 在 L2 页表中读出表项，同理得到 L3 页表的物理地址；
4. 用 VA[20:12] 在 L3 页表中读出页描述符，得到物理页框号；
5. 物理页框号拼上 VA[11:0] 的页内偏移，就是最终的物理地址。

每一步读到的地址都是物理地址——MMU 遍历页表时不可能再去翻译页表自己的地址。这就是为什么 walk() 往表项里写的是 V2P(pagetable)，读出来以后又要 P2V() 才能被内核软件访问（4.5.2 节）。途中任何一级遇到无效表项（bit 0 为 0），遍历就停下来，CPU 产生一个翻译错误（translation fault），并把出错的级别记在 ESR_EL1 里、出错的虚拟地址记在 FAR_EL1 里。

块映射。 有时候要映射一大片连续的物理内存，例如启动时的整个内核镜像。这时可以让 L2 表项直接是一个块描述符，指向一个 2 MiB 对齐的物理块，遍历在第 3 步就结束：省去一级页表，页内偏移变成 VA[20:0] 共 21 位（ $2^{21} = 2 \text{ MiB}$ ）。同理，L1 块描述符映射 1 GiB。描述符最低两位区分这几种情况：在 L1/L2 上 01 是块、11 是表；在 L3 上只有 11 合法，表示一个 4 KiB 页——同样是 11，含义随所在的级别而变。xv6-aarch64 只在启动页表里用块映射（4.2 节）。

4.1.4 描述符的属性位

表 4.1 是项目中用到的属性位。

表 4.1: 页表项属性位（kernel/aarch64.h）

宏	位	含义
PTE_VALID / PTE_TABLE	[1:0]	L1/L2: 01 块，11 表；L3: 11 页（PTE_V）
PTE_INDX(<i>i</i>)	[4:2]	AttrIndx: 选择 MAIR_EL1 中的第 <i>i</i> 个内存类型
PTE_U、PTE_RO	[7:6]	AP: EL0 是否可访问、是否只读
PTE_SH_INNER	[9:8]	共享属性: 11 为 Inner Shareable
PTE_AF	[10]	Access Flag: 必须置 1，否则首次访问触发 Access Fault
PTE_PXN、PTE_UXN	53、54	内核态 / 用户态禁止执行

4.1.5 内存属性间接表

页表项里并不直接写“可缓存”“设备内存”之类的属性，而是写一个 3 位的索引，真正的属性放在 MAIR_EL1 的 8 个字节里。项目定义了三种类型：

```
#define AI_DEVICE_nGnRnE_IDX 0x0
#define AI_NORMAL_NC_IDX      0x1
#define AI_NORMAL_IDX         0x2

#define MT_DEVICE_nGnRnE 0x00 // 设备内存：不聚合、不重排、无提前写确认
#define MT_NORMAL_NC     0x44 // 普通内存，不可缓存
#define MT_NORMAL        0xff // 普通内存，Inner/Outer Write-Back
```

```
#define PTE_DEVICE      PTE_INDX(AI_DEVICE_nGnRnE_IDX)
#define PTE_NORMAL_NC  PTE_INDX(AI_NORMAL_NC_IDX)
#define PTE_NORMAL      (PTE_INDX(AI_NORMAL_IDX) | PTE_SH_INNER)
```

用途划分很简单：RAM、内核、用户页面和页表本身用 PTE_NORMAL；UART、GPIO、SD 控制器等外设寄存器用 PTE_DEVICE；摄像头 DMA 缓冲区用 PTE_NORMAL_NC，这样就不必在每帧前后做缓存维护（第 7 章）。

4.2 启动页表

第 3 章的启动代码把处理器降到 EL1、点亮串口之后，内核还运行在物理地址上，MMU 和缓存都是关着的。本节接着讲从这里到跳进高地址的三步：清 BSS、建临时页表、打开 MMU。

4.2.1 清 BSS 与早期栈

C 语言假定未初始化的全局变量为 0，这些变量位于 .bss 段。镜像文件里并不包含 BSS 的内容，所以要先清零。链接脚本导出了 bss_start 和以 8 字节为单位的 bss_size：

```
entry:
    adrp x1, bss_start          // PC 相对：MMU 未开时得到物理地址
    ldr w2, =bss_size
1:   cbz w2, 2f
    str xzr, [x1], #8
    sub w2, w2, #1
    b 1b
2:   ...                      // 输出 'B'
```

把需要清零的数据单独放在 .bss 段，是为了节省镜像空间：ELF 只记录这个段的大小，不存它的内容。代价是装载之后必须由启动代码自己清零，所以链接脚本要导出段的起止位置，并让起点按 8 字节对齐。对齐在这里不只是为了效率：MMU 关闭时所有数据访问都按 Device 内存处理，非对齐访问会直接触发对齐异常，因此一次写 8 字节的 str xzr 要求地址必须是 8 的倍数。

栈也是同样的道理：C 代码需要一个可用的栈指针。最简单的做法是把 sp 设到内核镜像上方一个“足够远”的固定地址（例如内核装在 0 时设为 4 MiB），栈向下增长也碰不到内核。本项目要支持四个核同时运行，所以在 BSS 中定义了 stack0[4096 * NCPU]，每个核按核号取其中一页（见 4.2.3 节末尾的 _start）。

4.2.2 两棵临时页表

entry.S 在 MMU 打开前用汇编构建两棵只有两级的小页表，四张页表页（l1entrypgt、l2entrypgt、l1kpgt、l2kpgt）静态定义在 start.c 中、按页对齐。

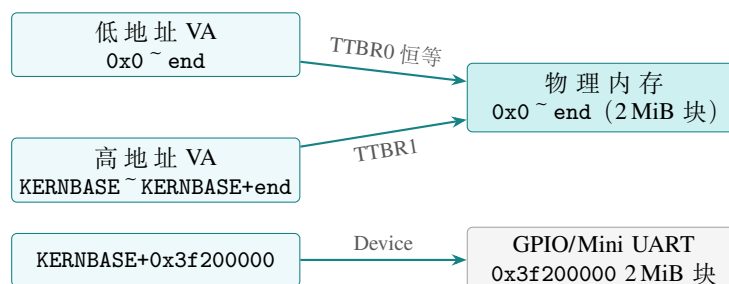


图 4.2: 启动页表：同一段物理内存同时映射到低地址和高地址

如图 4.2，恒等映射保证“打开 MMU 的那条指令之后还能取到下一条指令”，高地址映射保证“跳到链接地址之后一切照常”。两者都用 2 MiB 块描述符，所以每棵树只要一张 L1 和一张 L2：

```

    adrp x0, l2entrypgt
    mov  x1, #0                      // 起始 PA
    ldr  x2, =V2P_W0(end)-1          // 结束 PA
    lsr  x3, x1, #PXSHIFT(2)
    and  x3, x3, #PXMASK              // 起始 L2 索引
    lsr  x4, x2, #PXSHIFT(2)
    and  x4, x4, #PXMASK              // 结束 L2 索引
    mov  x5, #(PTE_AF | PTE_NORMAL | PTE_VALID)
    orr  x6, x1, x5                   // 第一个块描述符
l2epgt_loop:
    str  x6, [x0, x3, lsl #3]         // l2entrypgt[idx] = 块描述符
    add  x3, x3, #1
    add  x6, x6, #0x200000            // 下一个 2 MiB
    cmp  x3, x4
    b.ls l2epgt_loop

```

启动页表只覆盖内核镜像所在的前 2 MiB (EARLYTOP=0x200000)。main() 一开始就检查内核镜像连同 BSS 没有越过这个边界——否则后面的分配器会把内核自己的内存当成空闲页发出去：

```

if(V2P(end) > EARLYTOP){
    for(char *m = "\nkernel image extends past EARLYTOP\n"; *m; m++){
        boot_uart_mark(*m);
    }
    for(;;) asm volatile("wfe");
}

```

4.2.3 打开 MMU：TCR、MAIR 与 SCTL

页表摆好以后，还要告诉 MMU 怎样解释它们。主核在 _entry、副核在 entryothers 里执行的是同一段代码：

```

    adrp x0, l1entrypgt
    adrp x1, l1kpgt
    msr  ttbr0_el1, x0               // 低地址：恒等映射
    msr  ttbr1_el1, x1               // 高地址：内核
    ldr  x0, =(TCR_T0SZ(25)|TCR_IRGN0(1)|TCR_ORGN0(1)|TCR_SH0(3)|TCR_TG0(0)| \
                TCR_T1SZ(25)|TCR_IRGN1(1)|TCR_ORGN1(1)|TCR_SH1(3)|TCR_TG1(2)|TCR_IPS(0))
    msr  tcr_el1, x0
    ldr  x1, =(MT_DEVICE_nGnRnE<<(8*AI_DEVICE_nGnRnE_IDX)) | \
                (MT_NORMAL_NC<<(8*AI_NORMAL_NC_IDX)) | (MT_NORMAL<<(8*AI_NORMAL_IDX)))
    msr  mair_el1, x1
    isb
    ldr  x1, =_start                  // 链接地址（高地址）
    dsb  ish
    tlbi vmalle1                     // 丢掉固件可能留下的翻译
    dsb  ish
    isb
    ic   iallu

```

```

dsb sy
isb
mrs x0, sctlr_el1
orr x0, x0, #1          // M: 打开 MMU
orr x0, x0, #(1 << 2)   // C: 数据缓存
orr x0, x0, #(1 << 12)  // I: 指令缓存
msr sctlr_el1, x0
isb
br x1                  // 跳到高地址

```

TCR_EL1 (Translation Control Register) 的各个字段决定了前面所有的“规则”:

- TOSZ=T1SZ= 25: 两个地址空间都是 39 位, 于是遍历从 L1 开始、一共三级;
- TG0= 0、TG1= 2: 两边都是 4 KiB 粒度。两个字段的编码不一样 (TG0 中 00 是 4 KiB, TG1 中 10 才是 4 KiB), 抄错一个数字, 高地址的页表就会被按 64 KiB 或 16 KiB 粒度解释;
- IRGN/ORGN= 1、SH= 3: MMU 遍历页表时本身也要读内存, 这几位规定这些读取走 Write-Back 缓存、属于 Inner Shareable 域。它们必须和页表页本身的映射属性 (PTE_NORMAL) 一致, 否则 CPU 写进缓存的新表项, MMU 可能看不到 (第 4.8.1 节的故障与此同源);
- IPS= 0: 物理地址 32 位 (4 GiB), 对 BCM2837 足够。

MAIR_EL1 把三种内存类型写进第 0、1、2 字节, 页表项的 AttrIndx 就是这里的下标。SCTLR_EL1 是总开关: M 位打开地址翻译, C、I 位打开数据和指令缓存。打开前的 `tlbi vmalle1` 和 `ic iallu` 清掉固件阶段可能残留的 TLB 和指令缓存; 写 SCTLR_EL1 之后的 `isb` 保证后续取指都已经按新的翻译进行。此时 PC 仍是低地址, 靠 TTBR0 的恒等映射继续执行, 直到 `br x1` 跳进高地址——这就是启动页表要同时建两棵树的原因。

4.2.4 跳到高地址

`ldr x1, =_start` 从文字池中取出链接时确定的高虚拟地址; 这里不能写 `b _start`: 打开 MMU 并不会改变 PC, 而 `b` 和 `adr` 都是相对当前 PC 计算目标的, 会落到“没开 MMU 时 `_start` 所在的低地址”。必须事先用 `ldr` 从文字池取出链接器算好的高地址绝对值; 这条 `ldr` 本身也是 PC 相对的, 所以要在打开 MMU 之前执行。`br x1` 之后, CPU 就运行在 TTBR1_EL1 映射的高地址上了。`_start` 为每个核在 `stack0` 中划出 4 KiB 栈, 然后跳进 C 函数 `main()`:

```

_start:
    ldr x0, =stack0
    mov x1, #1024*4
    mrs x2, mpidr_el1
    and x2, x2, #0x3
    add x2, x2, #1
    mul x1, x1, x2
    add x0, x0, x1          // sp = stack0 + (cpuid + 1) * 4096
    mov sp, x0
    mov w0, #'H'
    bl boot_uart_mark
    b main

```

4.3 多核启动

四个核在上电后都会执行 `_entry`——CPU0 由 `armstub` 直接跳入，CPU1~3 则要等主核在 `spin-table` 里写入地址。如果像 `kernel_old=1` 那样没有 `armstub`，四个核会同时从同一个入口开始执行，内核要立刻用 `MPIDR_EL1` 的低 8 位区分核号，把除 0 号核以外的所有核挂进死循环——这是单核教学内核的标准做法，但被挂起的核就再也没有机会醒来了。本项目需要四核同时调度，所以副核不能死循环，而要在一个可以被唤醒的地方等待。整个过程有两道“闸门”：

第一道闸门：启动页表。 副核到达 `_entry` 后，统一好异常级别（3.1.3 节）就进入 `secondary_wait`，用 `ldar`（带 `acquire` 语义的加载）读取标志 `secondary_release`，为 0 就 `wfe` 休眠。主核建好启动页表后，用 `stlr`（带 `release` 语义的存储）把标志置 1，再执行 `sev` 唤醒所有在 `wfe` 上休眠的核：

```

        adrp x10, secondary_release           // 主核
        add x10, x10, :lo12:secondary_release
        mov w11, #1
        stlr w11, [x10]
        sev

secondary_wait:                               // 副核
        adrp x10, secondary_release
        add x10, x10, :lo12:secondary_release
4:      ldar w11, [x10]
        cbnz w11, entryothers
        wfe
        b 4b

```

`acquire/release` 配对保证：副核看到标志为 1 时，主核在此之前写入的页表内容也一定可见。

第二道闸门：内核初始化。 主核在 `main()` 的最后调用 `start_secondary_cpus()`，把 `_entry` 的物理地址写进三个 `spin-table` 槽，用 `dc cvac` 把这几行缓存写回内存（此时副核的 MMU 和缓存都没打开，只能看到内存里的值），再 `sev`：

```

static void start_secondary_cpus(void) {
    volatile uint64 *spin = (volatile uint64*)P2V(0xe0);
    uint64 entry = V2P((uint64)_entry);
    for(int cpu = 1; cpu < NCPU; cpu++){
        spin[cpu - 1] = entry;
        asm volatile("dc cvac, %0" :: "r"(&spin[cpu - 1]) : "memory");
    }
    asm volatile("dsb sy\n\tsev" ::: "memory");
}

```

副核进入 `main()` 后在 `while(started == 0)` 处等待主核完成全部初始化，然后只做每核相关的部分：切换到最终页表、安装异常向量、打开本核的定时器中断路由，最后进入调度器。真机串口上会看到：

```

PH0hart 3 starting
hart 2 starting
hart 1 starting

```

三个副核几乎同时出发，谁先抢到 `printf` 的锁谁先打印，所以顺序不固定；PHO 是不经过锁的早期标记，会与其他输出交错。

4.4 两阶段的物理内存初始化

建立最终页表需要分配页表页，分配页表页又需要物理页分配器——这是一个“先有鸡还是先有蛋”的问题。xv6 的解决办法是把分配器分两次喂：

```
kinit1(end, P2V(EARLYTOP));           // 阶段一：内核末尾到 2 MiB，启动页表已覆盖
kvminit();                             // 用这些页建最终内核页表
kvminithart();                          // 切换 TTBR1，作废 TLB
kinit2(P2V(EARLYTOP), P2V(PHYSTOP));   // 阶段二：2 MiB 到 112 MiB，最终页表已覆盖
```

分配器本身就是 xv6 的空闲链表：每个空闲页的前 8 字节存放下一页的地址，`kalloc()` 从表头取，`kfree()` 往表头放，全程由 `kmem.lock` 保护。为了尽早暴露“使用已释放内存”的错误，`kfree()` 把页填成 `0x01`，`kalloc()` 把页填成 `0x05`——第 8 章会看到，用户态异常地址 `0x0505050505050505` 正是一页刚分配、尚未初始化的内存。

PHYSTOP 设为 `0x07000000` (112 MiB)。上面保留给 QEMU 时代的 `ramdisk` (`0x07000000`) 和摄像头 DMA 区 (`0x07200000`)，分配器永远不会把它们发出去。

4.5 最终内核页表

4.5.1 `kvmmake()`

```
pagetable_t kvmmake(void) {
    pagetable_t kpgtbl = (pagetable_t) kalloc();
    memset(kpgtbl, 0, PGSIZE);
    // BCM2837 外设窗口 16 MiB，以及 ARM-local 外设一页：设备内存，不可执行
    kvmmap(kpgtbl, PERIPHERAL_BASE, PERIPHERAL_BASE_PA, 0x01000000, PTE_DEVICE | PTE_XN);
    kvmmap(kpgtbl, LOCAL_BASE, LOCAL_BASE_PA, PGSIZE, PTE_DEVICE | PTE_XN);
    kvmmap(kpgtbl, RAMDISK, RAMDISK_PA, RAMDISK_SIZE, PTE_NORMAL | PTE_XN);
    // 摄像头 DMA：不可缓存的普通内存
    kvmmap(kpgtbl, CAMDMA, CAMDMA_PA, CAMDMA_SIZE, PTE_NORMAL_NC | PTE_XN);
    // 物理第 0 页：spin-table 槽 0xe0..0xf0，唤醒副核时要写
    kvmmap(kpgtbl, KERNBASE, 0, PGSIZE, PTE_NORMAL | PTE_XN);
    // 内核代码：只读、可执行
    kvmmap(kpgtbl, KERNLINK, V2P(KERNLINK), (uint64)etext-KERNLINK, PTE_NORMAL | PTE_RO);
    // 内核数据和全部可用 RAM：可读写、不可执行
    kvmmap(kpgtbl, (uint64)etext, V2P(etext), (uint64)P2V(PHYSTOP)-(uint64)etext,
           PTE_NORMAL | PTE_XN);
    return kpgtbl;
}
```

代码段只读可执行，数据段可写不可执行 (W^X)，这样即使内核出现缓冲区溢出，也很难把数据当指令执行。

4.5.2 `walk()` 与 `mappages()`

所有映射最终都落到这两个函数上。`walk()` 从 L1 走到 L3，遇到缺失的中间页表就（按需）分配一页：

```
pte_t *walk(pagetable_t pagetable, uint64 va, int alloc) {
    if(va >= MAXVA) panic("walk");
    for(int level = 1; level < 3; level++) {
        pte_t *pte = &pagetable[PX(level, va)];
        if((*pte & PTE_VALID) && (*pte & PTE_TABLE)) {
            pagetable = (pagetable_t)P2V(PTE2PA(*pte));
        } else {
            if(!alloc || (pagetable = (pde_t*)kalloc()) == 0)
                return 0;
            memset(pagetable, 0, PGSIZE);
            *pte = PA2PTE(V2P(pagetable)) | PTE_TABLE | PTE_VALID;
        }
    }
    return &pagetable[PX(3, va)];
}

int mappages(pagetable_t pagetable, uint64 va, uint64 size, uint64 pa, uint64 perm) {
    uint64 a = PGROUNDDOWN(va), last = PGROUNDDOWN(va + size - 1);
    for(;;){
        pte_t *pte = walk(pagetable, a, 1);
        if(pte == 0) return -1;
        if(*pte & PTE_AF) panic("mappages: remap");
        *pte = PA2PTE(pa) | perm | PTE_AF | PTE_V;
        if(a == last) break;
        a += PGSIZE; pa += PGSIZE;
    }
    return 0;
}
```

注意页表项里存的是**物理地址**（硬件遍历页表时用物理地址），而内核软件读写页表时要先 P2V() 转成虚拟地址。页表页本身也位于被直接映射的 RAM 中，所以内核总能通过 KERNBASE + PA 访问任何一张页表——这不是递归页表技巧，只是直接映射的自然结果。

4.5.3 页表的规模

用户命令 vmmap 会调用 kvmdump() 打印内核页表，每个 2 MiB 窗口汇总一行，再逐个打印各进程的用户页表。表 4.2 是一次快照的统计（当时还保留着 ramdisk 映射）。

表 4.2: 最终内核页表的组成（一次运行时快照）

L1 项	虚拟地址	L2 有效项	内容
L1[0]	KERNBASE+0 ~ 1 GiB	65	内核和 RAM（56 项）、ramdisk（1 项）、BCM2837 MMIO（8 项）
L1[1]	KERNBASE+1 ~ 2 GiB	1	ARM-local 外设 0x40000000
L1[255]	接近 MAXVA	1	进程内核栈及其间的保护页
合计：L1 一张、L2 三张、L3 67 张，共 71 页 = 284 KiB；有效 L3 叶子 32 961 个			

如果改用 2 MiB 块映射 RAM，L3 表可以省掉绝大部分。项目选择全部用 4 KiB 页，是为了让内核栈的保护页、代码段只读等细粒度属性能用同一套 mappages() 实现。

4.6 用户地址空间

4.6.1 内核空间与用户空间

每个进程有自己的 L1 页表，进程切换时写入 TTBR0_EL1：

```
void switchvm(struct proc *p) {
    w_ttbr0_el1(V2P(p->vm->pagetable));
    flush_tlb();
}
```

4.6.2 exec() 之后的布局

exec() 建立的用户地址空间从虚拟地址 0 开始（图 4.3）：先是 ELF 的代码和数据，然后是一页不可访问的保护页和 4 页（16 KiB）用户栈，栈之上是 sbrk() 扩展的堆。用户页面的属性是 PTE_NORMAL | PTE_U，代码页可执行。

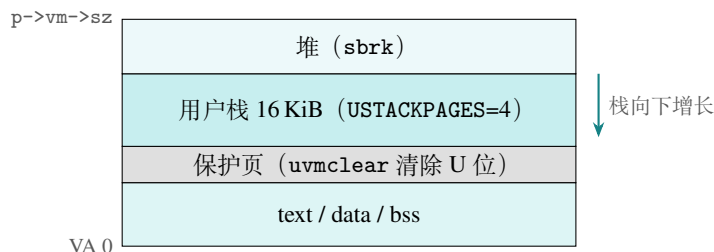


图 4.3: exec() 之后的用户地址空间

进程的内核栈不在用户页表里，而在内核页表高端的 KSTACK(slot) 处，每个栈一页，相邻栈之间隔一页不映射的保护页。内核栈溢出会立刻触发翻译错误，而不是悄悄覆盖相邻进程的栈。第 5 章会介绍内核栈的动态分配。

4.6.3 第一个用户进程的页表

所有用户地址空间都由同一组函数建立。uvmcreate() 分配一页并清零，作为空的 L1 页表——全 0 意味着全部表项无效，这一点很重要：任何从 kalloc() 拿来作页表的页都必须先清零，否则 0x05 填充的垃圾会被 MMU 当成有效描述符。第一个进程没有 ELF 文件可读，内核把一小段编进内核镜像的 initcode 直接拷进一页：

```
void uvminit(pagetable_t pagetable, uchar *src, uint sz) {
    if(sz >= PGSIZE) panic("initvm: more than a page");
    char *mem = kalloc();
    memset(mem, 0, PGSIZE);
    mappages(pagetable, 0, PGSIZE, V2P(mem), PTE_NORMAL|PTE_U);
    memmove(mem, src, sz);           // 通过内核直接映射写入
    uvmsync_icache(pagetable, sz);   // 写入的是指令，要同步 I-cache
}
```

userinit() 再把 trapframe 的 elr 设为 0、sp 设为 PGSIZE：进程第一次“返回”用户态时，从虚拟地址 0 取第一条指令，栈从这一页的顶部向下长。同一个物理页此时有两个虚拟地址：内核通过 KERNBASE+PA 写它，用户通过 VA 0 执行它。拷贝时用的是前者，所以内核根本不需要先切换到这个进程的页表。

4.6.4 fork 时复制地址空间

`fork()` 要给予进程一份与父进程内容相同、但物理上独立的内存。`uvmcopy()` 逐页遍历父进程的页表，为每个已映射的页分配新物理页、拷贝内容、以相同的属性映射进子进程的页表：

```
for(i = 0; i < sz; i += PGSIZE){
    if((pte = walk(old, i, 0)) == 0) panic("uvmcopy: pte should exist");
    if((*pte & PTE_AF) == 0)         panic("uvmcopy: page not present");
    pa = PTE2PA(*pte);
    flags = PTE_FLAGS(*pte);          // 连同 U、RO、XN 一起复制
    if((mem = kalloc()) == 0) goto err;
    memmove(mem, (char*)P2V(pa), PGSIZE); // 经内核直接映射读父进程的页
    if(mappages(new, i, PGSIZE, V2P(mem), flags) != 0){ kfree(mem); goto err; }
}
uvmsync_icache(new, sz);
```

源地址用的是 `P2V(pa)` 而不是用户地址 `i`。用户地址只有在 `TTBR0` 恰好装着父进程页表时才有意义；走内核直接映射则与当前装的是谁的页表无关，在多核上也更不容易出错。失败时 `uvmunmap()` 把已经复制的页全部释放，子进程的页表保持“要么全有、要么全无”。

4.6.5 TLB 与地址空间切换

如果每次访存都要走三级页表，一次 `ldr` 就会变成四次内存访问。MMU 因此把最近用过的翻译结果缓存在 TLB (Translation Lookaside Buffer) 里，命中时不再遍历页表。代价是：一旦软件改了页表，TLB 里可能还留着旧的翻译，必须由软件显式作废。

进程切换正是最大的一次“改页表”——`TTBR0` 换成另一棵树，之前缓存的低地址翻译全都过期了：

```
void switchuvm(struct proc *p) {
    if(p->vm == 0 || p->vm->pagetable == 0) panic("switchuvm: no pagetable");
    w_ttbr0_el1(V2P(p->vm->pagetable));
    flush_tlb(); // dsb ishst; tlbis vmalle1is; dsb ish; isb
}
```

`flush_tlb()` 作废全部 EL1 翻译，连内核的也一起丢掉了。ARMv8 提供了 ASID (地址空间标识)：把一个 8 或 16 位的进程编号写进 `TTBR0` 的高位，TLB 表项会带上这个标签，切换时就不必全部作废。xv6-aarch64 没有使用 ASID，每次切换都整体作废，换来的是实现简单、不必管理 ASID 的回收；对于这个规模的系统，多出来的 TLB 缺失是可以接受的。注意内核页表 (`TTBR1`) 在整个运行期间只装一次 (`kvminithart()`)，这也是把两个地址空间分开的好处。

4.6.6 缺页与按需分配

如果用户进程访问了一个没有映射的地址，页表遍历在某一级遇到无效项，CPU 产生同步异常进入 `usertrap()`。此时 `ESR_EL1` 的 `EC` 字段是 `0x24` (来自 `EL0` 的数据中止) 或 `0x20` (来自 `EL0` 的指令中止)；`ISS` 的低 6 位 `DFSC` 说明原因，`0b0001xx` 是第 `xx` 级的翻译错误，`0b0011xx` 是权限错误；出错的虚拟地址在 `FAR_EL1` 里。

很多内核利用这个异常实现按需分配 (demand paging)：`sbrk` 只增大进程的 `sz`，并不真正分配物理页；进程第一次碰到新页时触发翻译错误，内核检查 `FAR_EL1` 落在 `[0, sz)` 之内，就当场 `kalloc()` 一页、`mappages()` 映射上去，然后 `eret` 重新执行那条出错的指令。进程看到的只是一条稍微慢一点的 `ldr`。

xv6-aarch64 目前走的是另一条路：`growproc()` 调用 `uvmalloc()` 立即分配并清零所有新页，所以合法的访

问永远不会缺页。usertrap() 只处理 EC = 21 (svc)，其余异常一律打印 esr、elr、far，调用 uvmdump() 打印这个进程的整张页表，然后杀掉进程：

```

} else {
    printf("usertrap(): unexpected esr=%p ec=%p pid=%d\n", esr, ec, p->pid);
    printf("          elr=%p far=%p\n", r_elr_el1(), r_far_el1());
    uvmdump(p->vm->pagetable, p->pid, p->name, "usertrap");
    p->killed = 1;
}

```

这份输出正是 4.8 节分析真机故障时用的线索。若要加上按需分配，只需在这个分支里先判断 `ec == 0x24 && (esr & 0x3c) == 0x04 && r_far_el1() < p->vm->sz`，然后分配、映射，再像故障三那样广播作废 TLB 后返回；`growproc()` 则改为只增大 `sz`。

4.7 MMIO、总线地址与 DMA

4.7.1 MMIO

SoC 把外设寄存器和 RAM 放进同一个物理地址空间，CPU 用普通的 `ldr/str` 访问它们，由片内地址译码器决定一次访问是去 DDR 控制器还是去某个外设（表 4.3）。

表 4.3: Raspberry Pi 3 的物理地址布局（本内核使用的部分）

物理地址	用途
0x00000000 ~ 0x06ffffff	内核管理的 RAM (PHYSTOP = 112 MiB)
0x00080000	内核镜像装载地址
0x07200000 ~ 0x072fffff	摄像头 DMA 区 (CAMDMA)
0x3f003000	BCM System Timer
0x3f00b000	legacy 中断控制器；+0x880 处为 property mailbox
0x3f101000	时钟管理器（摄像头 CM_CAM1）
0x3f200000	GPIO
0x3f202000	SDHOST（外置 SD 卡）
0x3f205000	BSC0 (I2C)
0x3f215000	AUX / Mini UART
0x3f300000	Arasan SDHCI（板载 Wi-Fi）
0x3f801000	Unicam CSI1
0x3f980000	DWC2 USB 主机
0x40000000	ARM-local 中断与定时器路由

外设寄存器必须映射为 Device 内存。若映射成可缓存的普通内存，写操作可能停留在缓存里迟迟不到达设备，读操作可能拿到旧值，两次写还可能被合并或重排——而对设备来说，“写 FIFO”这个动作本身就有副作用。运行时可以用 `cat /proc/iomem` 查看内核登记的全部外设地址区间。

4.7.2 三种地址视图

Broadcom 的文档用 VideoCore 总线地址描述外设，例如 Mini UART 在 0x7e215000。同一组寄存器在不同观察者眼里地址不同：

VideoCore 总线地址	0x7e215000	（文档、设备树里的写法）
ARM 物理地址	0x3f215000	（CPU 发出的地址）
内核虚拟地址	0xfffff803f215000	（经 TTBR1 翻译）

DMA 方向的地址转换更容易出错。DWC2 USB、Unicam 这些总线主设备自己访问内存时，看到的是 VideoCore 总线地址，必须把 ARM 物理地址加上别名前缀：

```
#define DWC2_DMA_BUS(a) (0xc0000000U | (uint32)V2P(a))
```

0xc0000000 别名表示“绕过 VideoCore 的 L2 缓存直接访问 SDRAM”。这不是新的内存，也不是 CPU 虚拟地址，而是同一片 RAM 在 GPU 总线上的另一种编址。

4.8 缓存与 TLB：三个真机故障

QEMU 不模拟缓存，也几乎不检查内存属性，下面三个问题都只在真机上出现。它们说明：页表不只是“虚拟地址到物理地址”的映射，还决定了 CPU 怎样访问这块内存。

4.8.1 故障一：停在 BPH0abde

真机串口输出 BPH0abde 后不再有动静。对照表 3.3，d 表示进入了第一次 kfree()，e 表示 memset 已完成，缺少的 f 表示 acquire(&kmem.lock) 没有返回。

当时只有 CPU0 在运行，不可能有别的核持锁。真正的原因在于 acquire() 的实现：

```
retry:  ldaxr w1, [x0]      // 独占加载，建立“独占监视”
        cbnz w1, retry
        stxr w2, w3, [x0]  // 独占存储，失败时 w2 != 0
        cbnz w2, retry
```

ldaxr 读到锁为 0，但 stxr 一直返回失败——这是活锁，不是死锁。当时普通 RAM 被映射成 *Normal Non-cacheable*，也没有 Inner Shareable 属性，数据缓存也没有打开。在 Cortex-A53 上，独占监视器依赖缓存一致性机制工作，对这种属性的内存，独占存储可能永远不成功。

修复包括四处：增加 MT_NORMAL (Write-Back) 类型并让 RAM 使用它；页表项加 PTE_SH_INNER；TCR_EL1 的页表遍历属性设为 Write-Back、Inner Shareable；SCTLR_EL1 打开 C 和 I 位。之后串口变成 BPH0abdefc1234，系统顺利进入 shell。

4.8.2 故障二：ls 跳到了不属于它的代码

打开缓存之后，在 shell 里执行 ls 出现用户异常：

```
usertrap(): unexpected esr=0x92000047 ec=0x24 pid=3
            elr=0x13dc far=0x13f48
```

EC=0x24 是来自 EL0 的数据中止，ISS 表示 L3 翻译错误、写访问。奇怪的是 ls 的代码只到 0xbd7，PC 却在 0x13dc。

原因在于 D-cache 与 I-cache 不一致。exec() 通过内核高地址把新程序的指令写进物理页，新指令此时还在 D-cache 里；用户态从低地址取指，走的是 I-cache，里面可能还留着同一虚拟地址上一个程序 (sh) 的指令。AArch64 不保证数据写入对取指自动可见，必须显式维护：

```
void uvmsync_icache(pagetable_t pagetable, uint64 sz) {
    for(uint64 va = 0; va < PGROUNDUP(sz); va += PGSIZE){
        uint64 kva = uva2ka(pagetable, va);
        if(kva == 0) continue;
        for(uint64 p = kva; p < kva + PGSIZE; p += 64)
            asm volatile("dc cvau, %0" :: "r"(p) : "memory"); // 数据清到统一点
    }
    asm volatile("dsb ish" ::: "memory");
```

```
asm volatile("ic ialluis\n\tdsb ish\n\tisb" ::: "memory"); // 广播作废所有核的 I-cache
}
```

第一版用 `ic ivau` 按内核高地址作废 I-cache，仍然失败：同一物理页有内核高地址和用户低地址两个别名，按其中一个别名维护不能保证覆盖另一个。改用 `ic iallu` 作废整个 I-cache 后问题消失。但打开四核以后它又以另一种形式回来了：`ic iallu` 只作废执行它的那个核的 I-cache，`exec()` 在一个核上装好新程序，进程迁移到另一个核后，那里仍能命中旧程序的指令——表现为 ELR 指向一条根本不访存的指令却报数据中止（第 7 章摄像头一节）。最终版本用广播到 Inner Shareable 域的 `ic ialluis`。代价是每次 `exec()/fork()` 清一次 I-cache，对 xv6 的规模完全可以接受。`exec()`、`uvminit()`（第一个进程）、`uvmcopy()`（`fork`）三处都会调用它。

4.8.3 故障三：sbrk 之后的旧 TLB

进一步的日志显示，上面那次异常还有另一层原因：`sh` 通过 `sbrk()` 新增了 0x13000 这一页，但 `growproc()` 修改页表后没有作废 TLB，CPU 继续使用之前缓存的“该地址无效”的翻译结果。修复是在页表扩展或收缩后执行广播式 TLB 失效：

```
dsb ishst           // 确保页表写入完成
tlbi vmalle1is      // 作废 Inner Shareable 域内所有核的 EL1 TLB
dsb ish
isb
```

`vmalle1is` 中的 `is` 很关键：多核以后，同一进程的线程可能正在别的核上运行，只作废本核 TLB 是不够的。收缩地址空间时顺序必须是“清页表项 → 广播作废 TLB → 释放物理页”，否则别的核可能通过陈旧的 TLB 访问一页已经被重新分配的内存。



笔记 经验：在 AArch64 上，改了“将要执行的内存”要做 D/I-cache 同步，改了“页表”要做 TLB 失效，两者不能互相替代。QEMU 两件事都不检查，所以必须上真机验证。

第五章 进程创建：fork

本章内容

- ❑ 进程的数据结构
- ❑ 动态进程表与按需分配的内核栈
- ❑ 第一个用户进程
- ❑ fork() 逐行解析
- ❑ 上下文切换与调度器
- ❑ exec、exit 与 wait
- ❑ 睡眠、唤醒与同步原语
- ❑ 多核软实时调度

5.1 进程由什么组成

在 xv6 里，一个进程就是一个 `struct proc`，加上它引用的几样东西：一张用户页表、一个内核栈、一个 `trapframe`、一组打开的文件。图 5.1 画出了它们的关系。

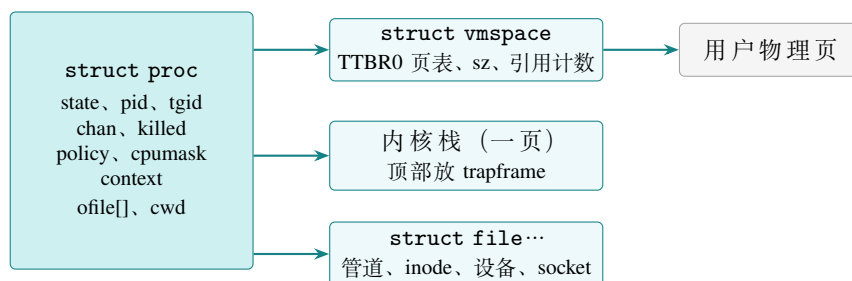


图 5.1: 一个进程的组成

5.1.1 struct proc

```
enum procstate { UNUSED, USED, SLEEPING, RUNNABLE, RUNNING, ZOMBIE };

struct proc {
    struct spinlock lock;
    // 以下字段需持有 p->lock
    enum procstate state;
    void *chan;           // 非 0 表示睡在这个“通道”上
    int killed, xstate;
    int pid, tgid, is_thread; // tid、线程组 id
    int sid, pgid, ctty;    // 会话、进程组、控制终端
    int policy, rt_priority, nice;
    uint cpumask;           // CPU 亲和性
    int slice; uint64 rq_seq; // 时间片、入队序号
    // 以下字段需持有 wait_lock
    struct proc *parent;
    // 以下字段只由进程自己访问
    uint64 kstack, kstack_pa, kstack_slot;
    struct proc *next;    // 动态进程表链表
    struct vmSPACE *vm;   // 用户地址空间（线程间共享）
    struct trapframe *trapframe; // 用户态现场，位于内核栈顶
```

```

struct context context;      // 内核态现场, swtch() 用
struct file *ofile[NOFILE];
struct inode *cwd;
char name[16];
};

```

字段按“谁在什么锁下访问”分组，这是 xv6 的一贯风格。与原版相比，这里多了会话和进程组（TTY 作业控制要用）、调度属性（软实时调度）、线程组 id，并且用户地址空间被抽成了可共享的 `struct vmSPACE`，为第 8 章的用户线程做准备。

5.1.2 两种现场

进程有两份寄存器现场，初学时很容易混淆：

- **trapframe**：用户态被打断时（系统调用、中断、异常）由 `usavereg` 保存，包含全部 31 个通用寄存器、`elr`、`spsr` 和用户 `sp`。它描述“回到用户态时从哪里继续”。
- **context**：内核态主动让出 CPU 时由 `swtch()` 保存，只包含被调用者保存寄存器 `x18 ~ x30` 和 `sp`。它描述“这个进程的内核线程停在了哪里”。

AArch64 的调用约定规定 `x19 ~ x28`、`x29`（帧指针）和 `sp` 由被调用者保存，`x30` 是返回地址。`swtch()` 是一次普通的函数调用，调用者保存寄存器已经由编译器在调用前存好，所以 `context` 只需要这 14 个值。

5.2 动态进程表与内核栈

原版 xv6 用静态数组 `proc[NPROC]`，并在启动时为 64 个槽位全部分配内核栈。本项目改成了按需增长的链表：

```

static struct proc* allocproc(void) {
    acquire(&proc_table_lock);
    for(p = proc_head; p; p = p->next) {          // 先找一个 UNUSED 节点复用
        acquire(&p->lock);
        if(p->state == UNUSED) {
            if(p->kstack_pa == 0 && alloc_kstack(p) < 0) { ... return 0; }
            goto found;
        }
        release(&p->lock);
    }
    p = newprocslot();                             // 没有就新建一个节点
    acquire(&p->lock);
found:
    release(&proc_table_lock);
    p->pid = allocpid(); p->tgid = p->pid;
    sched_defaults(p);
    p->state = USED;
    sp = (char*)p->kstack + PGSIZE;
    sp -= sizeof(*p->trapframe);                    // trapframe 放在内核栈顶
    p->trapframe = (struct trapframe*)sp;
    p->vm = vmSPACE_alloc();                         // 空的用户地址空间
    memset(&p->context, 0, sizeof(p->context));
    p->context.x30 = (uint64)forkret;                 // 第一次被调度时“返回”到 forkret
}

```

```

p->context.sp = (uint64)sp;
return p;                                     // 持有 p->lock 返回
}

```

几个设计要点：

- 每个节点分到一个永不改变的 `kstack_slot`，内核栈虚拟地址为 `KSTACK(slot)`，相邻栈之间隔一页保护页；
- 节点空闲时释放内核栈的物理页，但节点本身永远留在链表里。调度器、`wait()`、`wakeup()` 都无锁遍历这个链表，保留节点就不会出现悬空的 `next` 指针，省去了 RCU 一类的复杂机制；
- 释放内核栈时先清页表项，再广播作废 TLB，最后才 `kfree` 物理页（第 4 章的规则）。`forktest` 同时创建并回收 128 个子进程，验证已经越过了旧的 64 个槽位上限。

5.3 第一个用户进程

所有用户进程都是 `fork()` 出来的，但第一个进程只能由内核手工构造。`userinit()` 分配一个进程，把一段内置的机器码 `initcode` 拷到它的第 0 页：

```

void userinit(void) {
    struct proc *p = allocproc();
    initproc = p;
    uvminit(p->vm->pagetable, initcode, sizeof(initcode)); // 拷贝并同步 I-cache
    p->vm->sz = PGSIZE;
    p->trapframe->elr = 0;           // 用户态 PC
    p->trapframe->spsr = 0;           // M[3:0]=0: EL0t
    p->trapframe->sp = PGSIZE;       // 用户栈顶
    safestrcpy(p->name, "initcode", sizeof(p->name));
    p->cwd = namei("/");
    make_runnable(p, 0);
    release(&p->lock);
}

```

`initcode` 由 `user/initcode.S` 汇编而来，只做一件事：调用 `exec("/init")`。

```

start:  ldr x0, =init                // 路径
        ldr x1, =argv                // 参数表
        mov x7, #SYS_exec
        svc #0
exit:   mov x7, #SYS_exit            // exec 失败才会到这里
        svc #0
        b exit
init:   .ascii "/init\0"

```

这个进程第一次被调度时，`swtch()` 恢复的 `x30` 是 `forkret`，于是“返回”到 `forkret()`。它释放调度器替它拿着的 `p->lock`；如果是系统中第一个进程，还要在进程上下文中初始化文件系统（`fsinit` 读 `superblock`、恢复日志，期间可能睡眠，所以不能放在 `main()` 里）。然后调用 `usertrapret(trapframe)`，从 `trapframe` 恢复寄存器，`eret` 到 `EL0` 的地址 0 开始执行用户代码。

5.4 fork() 逐行解析

```

int fork(void) {
    struct proc *np, *p = myproc();
    if((np = allocproc()) == 0)                // (1)
        return -1;

    acquire(&p->vm->lock);                      // (2)
    if(uvmcopy(p->vm->pagetable, np->vm->pagetable, p->vm->sz) < 0){
        release(&p->vm->lock);
        freeproc(np); release(&np->lock);
        return -1;
    }
    np->vm->sz = p->vm->sz;
    release(&p->vm->lock);

    *(np->trapframe) = *(p->trapframe);         // (3)
    np->trapframe->x0 = 0;                      // (4)

    for(int i = 0; i < NOFILE; i++)            // (5)
        if(p->ofile[i]) np->ofile[i] = filedup(p->ofile[i]);
    np->cwd = idup(p->cwd);
    np->sid = p->sid; np->pgid = p->pgid; np->ctty = p->ctty;
    safestrcpy(np->name, p->name, sizeof(p->name));

    acquire(&p->lock);                          // (6)
    np->policy = p->policy; np->rt_priority = p->rt_priority;
    np->nice = p->nice; np->cpumask = p->cpumask;
    release(&p->lock);
    np->slice = timeslice(np);

    int pid = np->pid;
    release(&np->lock);
    acquire(&wait_lock); np->parent = p; release(&wait_lock); // (7)
    acquire(&np->lock); make_runnable(np, 0); release(&np->lock); // (8)
    return pid;                                // (9)
}

```

1. 分配进程描述符、内核栈和空的地址空间，context 设为“从 forkret 开始”；
2. 逐页复制父进程的用户内存：为每一页新分配物理页，拷贝内容，用相同的权限映射到子进程页表。复制完调用 `uvmsync_icache()`，避免子进程的新物理页命中旧的指令缓存；
3. 复制整个 `trapframe`——子进程回到用户态时，PC、栈指针、全部寄存器都和父进程在 `svc` 之后一模一样；
4. 唯一的区别：子进程的 `x0`（系统调用返回值）改成 0；
5. 复制文件描述符表。注意复制的是 `struct file` 的引用，父子共享同一个文件偏移，这正是 `sh` 实现重定向和管道的基础；
6. 像 Linux 一样，子进程继承调度策略、优先级、`nice` 和 CPU 亲和性；
7. 设置父进程指针，受 `wait_lock` 保护，与 `wait()`、`exit()` 同步；
8. 把子进程放入运行队列，从这一刻起别的 CPU 就可能开始运行它；

9. 父进程返回子进程的 pid。

图 5.2 对比了父子两条返回路径。

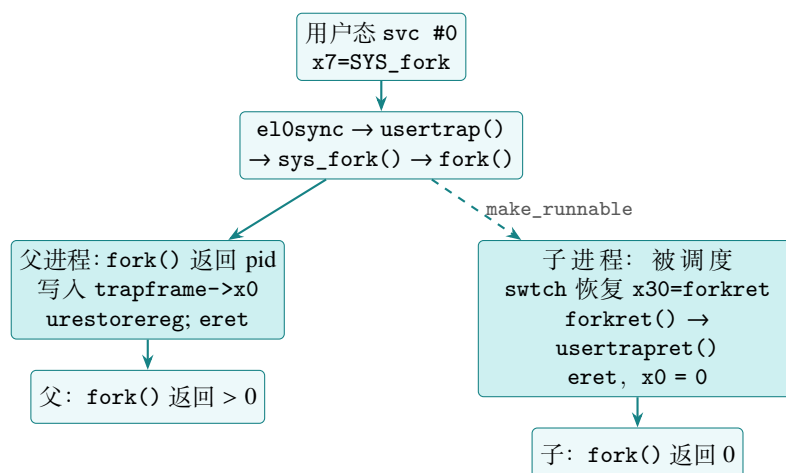


图 5.2: fork() 的两条返回路径

子进程从来没有执行过 fork() 的前半段，它的内核栈上只有一个 trapframe。调度器第一次运行它时，swtch() 恢复的返回地址是 forkret，forkret() 再调用 usertrapret()，从复制来的 trapframe “返回” 到用户态——在用户程序看来，就好像子进程也刚刚从 fork() 返回，只是返回值为 0。

5.4.1 新进程的第一次运行

allocproc() 为新进程伪造了一个“刚刚调用过 swtch()”的上下文：context.sp 指向内核栈上 trapframe 的下方，context.x30（返回地址）指向 forkret。调度器第一次 swtch() 到它时，ret 指令就跳进了一个从未被调用过的函数。这是所有操作系统创建新任务的共同技巧：教学内核里常见的写法是把返回地址设成一段叫 ret_from_fork 的汇编，Linux 也是如此。

这里有一个细节容易忽略：调度器在 swtch() 之前持有新进程的 p->lock，而持锁期间本 CPU 的中断是关闭的，所以新进程从 forkret 开始执行的那一刻不会被抢占。forkret() 的第一件事就是释放这把锁，相当于“完成交接、重新允许抢占”。若这时被定时器打断，也只是正常的抢占。然后它调用 usertrapret()，把父进程复制来的 trapframe 装回寄存器，eret 到用户态：

```

void forkret(void)
{
    static int first = 1;
    struct proc *p = myproc();
    struct trapframe *tf = p->trapframe;

    release(&p->lock);           // 调度器替我们拿着的锁
    if (first) {                 // 第一个进程：在进程上下文中初始化文件系统
        first = 0;
        fsinit(ROOTDEV);
    }
    usertrapret(tf);            // 从 trapframe 返回用户态
}
  
```

5.5 上下文切换与调度器

5.5.1 swtch()

```
swtch:                                // void swtch(struct context *old, struct context *new)
    mov x9, x0
    mov x10, sp
    stp x10, x18, [x9, #16 * 0]      // 保存 sp 和 x18..x30
    stp x19, x20, [x9, #16 * 1]
    ...
    stp x29, x30, [x9, #16 * 6]
    mov x9, x1
    ldp x10, x18, [x9, #16 * 0]      // 恢复新的 sp 和 x18..x30
    ...
    ldp x29, x30, [x9, #16 * 6]
    mov sp, x10
    ret                                // 跳到新上下文的 x30
```

ret 跳到恢复出来的 x30：对于一个曾经让出过 CPU 的进程，那是它当年调用 swtch() 的下一条指令（在 sched() 里）；对于新进程，就是 forkret。

5.5.2 调度循环

每个 CPU 都有一个调度器线程，使用启动时的 stack0 栈：

```
void scheduler(void) {
    struct cpu *c = mycpu();
    int id = cpuid();
    c->proc = 0; c->online = 1;
    for(;;){
        intr_on();                                // 让设备有机会中断
        if((p = pick_next(c, id)) == 0) continue; // 选优先级最高的可运行进程
        acquire(&p->lock);
        if(p->state == RUNNABLE && (p->cpumask & (1U << id))){
            p->state = RUNNING; p->last_cpu = id;
            c->proc = p; c->cur_prio = effective_prio(p, c);
            switchvm(p);                          // TTBR0 = 进程页表
            swtch(&c->context, &p->context);       // 运行进程，直到它 sched()
            switchkvm();                          // TTBR0 = 0
            c->proc = 0; c->cur_prio = PRIO_IDLE;
        }
        release(&p->lock);
    }
}
```

进程让出 CPU 的唯一入口是 sched()：调用者必须持有且只持有 p->lock，并已把状态改成非 RUNNING。锁在两次 swtch() 之间“传递”——进程持锁切到调度器，调度器在 swtch() 返回后释放；反方向同理，新进程在 forkret() 或 sched() 返回后释放。这保证了“设置状态”和“切换上下文”之间不会有别的 CPU 插进来运行同一个进程。

5.5.3 调度对进程是透明的

调度的目标是让多个进程分享 CPU 时间，而每个进程都以为自己独占 CPU。能做到这一点，靠的正是上面两份现场：被中断的那一刻，用户寄存器全部存在 `trapframe` 里；让出 CPU 的那一刻，内核寄存器存在 `context` 里。从进程自己的角度看，它只是调用了一次 `swtch()`，过了一段（也许很长的）时间后函数返回了，期间别的进程运行过这件事它察觉不到。也正因为切换只发生在一次函数调用之内，按调用约定可以被调用者破坏的 `x0 ~ x17` 不需要保存。

5.5.4 谁触发调度

进程让出 CPU 有两种情形：

- **主动让出**：进程在内核中没事可做了，例如 `sleep()` 等待磁盘、管道或网络，`wait()` 等待子进程，`exit()` 结束自己。它们把状态改成 `SLEEPING` 或 `ZOMBIE`，然后调用 `sched()`；
- **被动抢占**：每次 10ms 定时器中断，`sched_tick()` 给当前进程的时间片减一，减到 0 就设置本 CPU 的 `need_resched`；另一个 CPU 唤醒了更高优先级的进程时，会通过核间中断（IPI）设置这个标志。中断返回前，`userirq()/kernelirq()` 检查 `resched_pending()`，若为真就调用 `yield()`，把自己放回运行队列再 `sched()`。

```
void sched_tick(void)                // 每个 CPU 每 10 ms 调用一次
{
    struct cpu *c = mycpu();
    struct proc *p = c->proc;
    if(p){
        acquire(&p->lock);
        p->run_ticks++;
        if(p->policy != SCHED_FIFO && --p->slice <= 0)
            c->need_resched = 1;      // 时间片用完：下次中断返回时让出
        release(&p->lock);
    }
    ...                               // 实时任务节流计数
}
```

5.5.5 关中断与禁止抢占

教学内核里常见一个计数器 `preempt_count`：它不为 0 时，定时器中断不会触发重新调度。xv6 没有单独的计数器，但效果相同：调度相关的代码总是持有 `p->lock`，而持有任何自旋锁时本 CPU 的中断都是关着的（`push_off`），也就不可能被定时器抢占。`sched()` 开头的检查把这个约定写成了断言：必须持有 `p->lock`、只持这一把锁、状态已不是 `RUNNING`、中断是关着的。

“关中断”与“禁止抢占”并不总是一回事。调度器循环在每一轮开头都要 `intr_on()`：如果此时没有任何可运行的进程，CPU 就在循环里空转，而让某个进程重新变成 `RUNNABLE` 的，往往正是一个设备中断（磁盘读完、网络包到达）。如果调度器关着中断空转，就永远等不到这个中断。早期 Linux 的调度函数也有同样的要求：它在运行期间禁止嵌套调度，但必须允许中断发生。

5.5.6 如何选择下一个进程

最早的 Linux (0.01 版) 用一个很巧妙的计数器算法：每个任务有优先级 `priority` 和剩余时间 `counter`，每个定时器节拍当前任务的 `counter` 减一；调度时选 `counter` 最大的可运行任务；若所有可运行任务的 `counter`

都为 0，就给所有任务（包括在睡眠的）执行 $\text{counter} \leftarrow \text{counter}/2 + \text{priority}$ 。睡得越久的任务累积得越多，但不会超过 $2 \times \text{priority}$ （这是迭代的不动点），所以交互式任务醒来后能优先得到 CPU。

xv6-aarch64 的 `pick_next()` 把“剩余时间”和“谁先运行”分开：时间片只决定什么时候让出，选择则按有效优先级，同优先级按入队先后（`rq_seq`）：

```
static struct proc* pick_next(struct cpu *c, int id)
{
    struct proc *best = 0;
    int best_prio = 0;
    uint64 best_seq = 0;

    for(struct proc *p = proc_head; p; p = p->next){
        acquire(&p->lock);
        if(p->state == RUNNABLE && (p->cpumask & (1U << id))){
            int prio = effective_prio(p, c);
            if(best == 0 || prio > best_prio ||
               (prio == best_prio && p->rq_seq < best_seq)){
                best = p;
                best_prio = prio;
                best_seq = p->rq_seq;
            }
        }
        release(&p->lock);
    }
    return best;
}
```

时间片用完的进程在 `yield()` 中重新入队、拿到更大的 `rq_seq`，于是排到同优先级的末尾——这就是轮转（round robin）。完整的优先级规则见第 5.8 节。

5.5.7 跟踪一次时间片轮转

调度代码里有大量异步交互，单看函数很难想清楚整个系统的状态如何随时间变化。下面跟踪进程 A 在用户态运行时被定时器抢占、换成进程 B、之后又换回 A 的全过程，注意每一步发生在哪个栈上：

1. A 在 EL0 运行，使用自己的用户栈（`SP_EL0`）；
2. 定时器中断到来。硬件切到 `SP_EL1`，此时它指向 A 内核栈的顶端；`el0irq` 的 `usavereg` 把 A 的全部用户寄存器存进 A 的 `trapframe`；
3. `userirq()` → `devintr()` → `sched_tick()`：A 的时间片用完，设置 `need_resched`；
4. `userirq()` 发现 `resched_pending()`，调用 `yield()`：拿 `A->lock`，状态改为 `RUNNABLE`，`sched()` → `swtch(&A->context, &cpu->context)`。A 的内核寄存器存进 `A->context`，`sp` 换成本 CPU 的调度器栈；
5. 调度器从上次的 `swtch()` 之后继续：`switchkvm()`，释放 `A->lock`，下一轮 `pick_next()` 选中 B；
6. 调度器拿 `B->lock`，状态改为 `RUNNING`，`switchvm(B)` 把 B 的页表装进 `TTBRO_EL1`，`swtch(&cpu->context, &B->context)`；
7. B 从它自己上次的 `swtch()` 之后继续——如果 B 当初也是被定时器抢占的，那就是它的 `sched()` 返回到 `yield()`，释放 `B->lock`，再一路返回到 `userirq()`、`el0irq`，`urestorereg` 从 B 的 `trapframe` 恢复寄存器，`eret` 回到 B 的用户态；如果 B 是新进程，则 `ret` 进入 `forkret`；
8. 若干时间片之后，B 以同样的方式让出，调度器再次选中 A，`swtch()` 恢复 `A->context`：A 回到第 4 步的 `sched()` 里面，沿 `yield()` → `userirq()` → `el0irq` 返回，`eret` 之后 A 从被打断的那条指令继续执行。

这一过程中，A 在内核里的调用链 `el0irq → userirq → yield → sched → swtch` 一直“冻结”在 A 自己的内核栈上：

A 的内核栈（高地址在下）	调度器栈（stack0）
+-----+	+-----+
swtch/sched/yield 的栈帧	scheduler() 的栈帧
userirq 的栈帧	（每个 CPU 一个）
+-----+	+-----+
trapframe: A 的用户寄存器	
+-----+	<- p->kstack + PGSIZE

这就是“以 A 的身份进入中断、以 B 的身份离开中断”之所以合法的原因：每个进程的中断现场（`trapframe` 中的 `elr`、`spsr`）和内核调用链都保存在它自己的内核栈上，`eret` 使用的永远是当前进程自己的那一份。与把任务结构放在栈页底部的教学内核相比，`xv6` 的每个进程在调度器之外还多了一个独立的调度器上下文，切换总是“进程 → 调度器 → 进程”两步，而不是进程之间直接切换；代价是多一次 `swtch()`，好处是调度器有自己干净的栈，进程退出时也不必在自己的栈上释放自己。

5.6 exec、exit 与 wait

`exec(path, argv)` 读取 ELF 文件头，为每个 `PT_LOAD` 段分配并装入内存，然后在其上方分配一页保护页和 4 页用户栈，把参数字符串和 `argv[]` 指针数组压栈。全部就绪后才替换旧的地址空间——任何一步失败，原进程都不受影响。新页表生效前调用 `uvmsync_icache()`。

`exit()` 关闭所有文件，把子进程过继给 `init`，把自己设为 ZOMBIE 并唤醒父进程，然后 `sched()`，再也不会回来。进程的内存和内核栈此时还不能释放（它正在这个内核栈上运行），要等父进程在 `wait()` 中找到 ZOMBIE 子进程后调用 `freeproc()` 回收。`init` 进程在一个循环里不停地 `wait()`，负责回收所有孤儿。

5.7 睡眠、唤醒与同步原语

5.7.1 sleep/wakeup

`xv6` 的阻塞机制只有两个函数。`sleep(chan, lk)` 原子地释放条件锁 `lk` 并在 `chan` 上睡眠；`wakeup(chan)` 把所有睡在 `chan` 上的进程置为 `RUNNABLE`。“通道”只是一个地址，通常就是被等待对象的地址。

关键是“原子”二字：`sleep()` 先拿到 `p->lock` 才释放 `lk`，而 `wakeup()` 必须拿到 `p->lock` 才能修改进程状态，所以唤醒不可能落在“检查完条件、还没睡着”的间隙里，不会丢失唤醒。被唤醒的进程总要在 `while` 循环中重新检查条件，因为 `wakeup` 会唤醒同一通道上的所有进程。

5.7.2 用户态同步原语

在 `sleep/wakeup` 之上，`kernel/sync.c` 提供了 64 个内核同步对象槽，用户通过带代数（`generation`）的整数句柄访问：

- `mutex`：记录持有者 `pid`，只有持有者能解锁；
- `semaphore`：计数信号量，`sem_wait` 在计数为 0 时阻塞；
- `event`：手动复位事件，`notify` 唤醒全部等待者并保持触发状态；
- `katomic`：跨进程的原子整数，支持 `load/store/fetch_add/compare_exchange`。

对象在 `fork()` 之前创建，父子进程就持有相同的句柄，可以跨越独立的地址空间同步。句柄中的代数防止对象销毁、重新分配之后旧句柄误用新对象。`prodcons` 示例用两个信号量（空槽、满槽）加一个互斥锁实现了容量为 4 的有界缓冲区。

5.8 多核软实时调度

原版 xv6 的调度器按数组下标轮询，找到第一个 RUNNABLE 的进程就运行。为了让高优先级任务被唤醒后能尽快抢占，项目加入了与 Linux 语义一致的三种策略（表 5.1）。

表 5.1: 调度策略

策略	优先级	行为
SCHED_OTHER	nice -20 ~ 19	分时；时间片 = $(20 - \text{nice})/2$ 个节拍（10 ms ~ 200 ms）
SCHED_FIFO	1 ~ 99	一直运行到阻塞、主动让出或被更高优先级抢占，没有时间片
SCHED_RR	1 ~ 99	同 FIFO，但同优先级之间每 100 ms 轮转

选择。 `pick_next()` 在允许当前 CPU 运行的 RUNNABLE 进程中选有效优先级最高的；同优先级选 `rq_seq` 最小的（最早入队）。实时任务的有效优先级是 $100 + \text{rt_priority}$ ，普通任务是 0，所以任何实时任务都排在普通任务前面。被更高优先级抢占、时间片还有剩余的实时任务保留原来的 `rq_seq`，回到队首。

抢占。 进程变成 RUNNABLE 时，`resched_hint()` 检查它允许运行的各个 CPU：如果有空闲 CPU，什么都不用做；否则找到当前优先级最低的 CPU，若新进程优先级更高，就设置该 CPU 的 `need_resched`，并通过 ARM-local mailbox 0 向它发送核间中断（IPI）。目标 CPU 在中断返回、系统调用返回或内核态中断返回三处检查 `need_resched`，调用 `yield()`。

节流。 为防止失控的 FIFO 任务卡死整个系统，每个 CPU 每秒最多给实时任务 950 ms；用完后实时任务的有效优先级降到 -1，让 shell 和内核工作线程在剩下的 50 ms 里运行——与 Linux 默认的 `sched_rt_runtime_us` 相同。

用户可以用与 Linux 同名的 `chrt`、`taskset`、`nice`、`renice` 命令设置这些属性，用 `rtlat` 测量唤醒延迟（第 8 章）。

第六章 文件系统

本章内容

- ❑ xv6 文件系统的分层
- ❑ 磁盘布局与 `mkfs`
- ❑ 块缓存、日志与 `inode`
- ❑ 块设备：从 `ramdisk` 到 SD 卡
- ❑ 根文件系统放在哪里
- ❑ VFS 中间层：`procfs`、`ext2`、`FAT32`
- ❑ `FAT32` 与启动分区：`bootfs`、`BPB`、`FAT` 表、目录项与长文件名
- ❑ `xv6` 根文件系统：`superblock`、`inode`、目录、路径查找与在线更新
- ❑ 烧录 SD 卡

6.1 分层结构

`xv6` 的文件系统是一叠职责单一的层（图 6.1）。本项目保留了中间各层不动，在上面加了一层 `VFS`，在下面把块设备从 `VirtIO` 磁盘换成了真正的 SD 卡。



图 6.1: 文件系统分层（深色为本项目新增或替换的层）

6.2 磁盘布局

`xv6` 原生文件系统的块大小是 1 KiB，磁盘依次为：

[启动块 | `superblock` | 日志 | `inode` 块 | 空闲位图 | 数据块]
0 1 2..

`superblock` 记录各区的起始块号和大小，魔数为 `0x10203040`。几个参数与原版不同：

- `FSSIZE= 32768`：文件系统 32 MiB。原来只有 1000 块，加入网络、`SSH` 等程序后，打包就用掉了 986 块，首次启动创建 `/etc` 配置时 `panic "ballo: out of blocks"`；

- DIRSIZ= 30: 文件名最长 30 字节, 目录项从 16 字节变为 32 字节, 为了让 TFTP 下载的 kernel8-xv6_wifi.img 这样的长文件名能通过普通 ls 显示;
- inode 有 12 个直接块和 1 个一级间接块, 单个文件最大 $12 + 256 = 268$ 块, 即 268 KiB。

 **笔记** 分区再大也不会改变可分配的块数——它由 superblock 里的 size 字段决定。修改 FSSIZE 或 DIRSIZ 之后必须重新生成并整体烧录 fs.img, 只替换内核会让新内核去解释旧格式的磁盘。

6.3 块缓存、日志与 inode

这三层基本沿用 xv6 原样, 这里只说明本项目中与它们相关的两点。

日志. 一次文件系统操作可能修改多个块 (例如创建文件要改 inode、目录、位图)。begin_op()/end_op() 之间的所有修改先写进日志区, 提交时一次性写回原位置; 启动时 fsinit() 检查日志, 把已提交但未写完的事务重做一遍。这样即使在写的过程中断电, 文件系统也只会处于“操作前”或“操作后”两种状态之一。对 SD 卡来说这很重要: 真机经常是直接拔电源关机的。

inode 睡眠锁. inode 操作可能触发 SD 卡读写, 耗时很长, 不能持有自旋锁, 所以每个内存 inode 有一把睡眠锁。项目中出现过一次典型的锁配对错误: 在为文件描述符层加入 FD_DEVICE、FD_VNODE、FD_SOCKET 等分支时, fileread() 的 FD_INODE 分支丢了 iunlock()。

```
} else if(f->type == FD_INODE){
    ilock(f->ip);
    if((r = readi(f->ip, 1, addr, f->off, n)) > 0)
        f->off += r;
    iunlock(f->ip);      // 这一行被误删
}
```

现象非常迷惑: 启动卡在“exec-login: inode found”之后, 调试输出显示 /bin/login 的 inode 被 PID 1 锁着, 而 valid=0。原因是 init 读 /etc/fstab 时没有解锁, 关闭文件后引用计数归零, 这个 inode 缓存槽被 iget() 复用给了 /bin/login——槽换了主人, 锁却还“记得”上一任的持有者。

 **笔记** iput() 不能代替 iunlock(): 前者管“有多少引用”, 后者管“是否正在独占访问内容”。一个有用的调试断言是: iget() 复用 ref==0 的槽时, 若发现锁仍被持有, 立即 panic 并打印旧的 dev/inum/owner。

6.4 块设备: 从 ramdisk 到 SD 卡

6.4.1 三个阶段

块设备驱动经历了三次替换:

1. **ramdisk**: 刚移植到 raspi3b 时, 用 QEMU 的 loader 把 fs.img 装到物理地址 0x07000000, bread/bwrite 直接 memmove。简单, 但改动不会写回宿主机;
2. **Arasan SDHCI** (`kernel/sd.c`): BCM2837 的 EMMC 控制器, GPIO48~53 设为 ALT3。QEMU 用 -drive if=sd 把 fs.img 接到 SD 总线, 数据终于可以持久保存;
3. **SDHOST** (`kernel/sdhost.c`): 后来要驱动板载 Wi-Fi, 而 Wi-Fi 芯片只能接在 Arasan 控制器上 (GPIO34~39)。于是外置 SD 卡改由 Broadcom 自己的 SDHOST 控制器 (0x3f202000) 驱动, GPIO48~53 改为 ALT0。两种 SD 驱动对上都提供同一个接口:

```
int sdsector(uint32 sector, void *buffer, int write); // 读写一个 512 字节扇区
```

xv6 的块是 1 KiB, 因此一个块对应两个扇区: 块 N 是扇区 $2N$ 和 $2N + 1$ 。

6.4.2 SD 卡初始化

无论哪个控制器，SD 卡协议层的初始化顺序都一样（SD 物理层规范 [13]）：

400 kHz 识别时钟	
CMD0	复位，卡进入 idle
CMD8	检查电压范围，确认 SD 2.0
CMD55 + ACMD41	反复发送直到卡完成上电；OCR.CCS=1 表示 SDHC（按扇区寻址）
CMD2	读 CID
CMD3	取得相对地址 RCA
CMD7	选中卡，进入 transfer 状态
(SDSC) CMD16	设置块长 512
切换到 25 MHz 工作时钟	

读一个扇区发 CMD17，写一个扇区发 CMD24，数据通过数据寄存器以轮询 PIO 方式搬运 128 个 32 位字。驱动用一把自旋锁串行化对控制器的访问。

6.4.3 命令超时与恢复

真机在 shell 里执行 ls 时偶尔 panic “fat32: read FS.IMG”。在驱动里加入事务阶段编号后得到：

```
sd: I/O failure lba=295956 write=0 phase=1 status=17f0207 irq=18000 control1=e0807
```

phase=1 表示失败发生在 CMD17 的命令阶段；中断状态 0x18000 是 ERROR 加 COMMAND TIMEOUT。更麻烦的是，超时后控制器的命令/数据状态机停在 inhibit 状态，之后的每条命令都会失败。仅仅清中断位不够，还要用 CONTROL1 的软件复位位分别复位命令线和数据线，等硬件自动清零这两位。修复后的 sdsector() 对同一扇区最多重试三次，每次失败后先 recover_io()。这只复位主机控制器内部的状态机，卡本身仍处于 transfer 状态，不需要重新初始化。

6.5 根文件系统放在哪里

启动时 rootdev_init() 读 SD 卡第 0 扇区，判断根文件系统用哪种布局（表 6.1），此后由 rootdev_rw() 把 xv6 块号翻译成 SD 卡的绝对扇区号。这两个函数都在 kernel/rootdev.c 里。

根块设备不是 FAT32 的一部分。 rootdev.c 不解释任何文件系统格式。它只读 MBR，用 xv6 superblock 的魔数认出根分区，再把 FAT32 启动分区交给 fat32.c 挂起来，供 /boot 和固件加载使用。根分区的块里装的始终是 xv6 自己的 superblock、inode 和目录（6.8 节），与 FAT32 毫无关系。块缓存的读写入口 rootdev_rw() 也只做地址换算，不调用任何 FAT32 函数。

表 6.1: 根文件系统的两种存放方式

方式	识别条件与地址换算	适用场景
原始分区	MBR 中某个非 FAT32 主分区的第 1 块是 xv6 superblock（魔数与大小匹配）；扇区 = 分区起点 + 块号 × 2	真机
裸镜像	没有 MBR；扇区 = 块号 × 2	QEMU

原始分区模式 是真机上的做法：SD 卡分成 4 个区，xv6 文件系统独占一个分区，与 FAT32 启动分区互不干扰。识别时只认 superblock 魔数，而不看 MBR 类型（macOS 只能建 0x83 类型的分区，与 ext2 相同）。启动日志会显示：

```
rootdev: raw xv6 root p2 lba=208896 sectors=819200 size=400 MiB
rootdev: bootfs p1 lba=2048 mounted as FAT32
```

6.6 VFS 中间层

6.6.1 动机

最初的 ext2 驱动只能通过两个专用系统调用 `ext2read`、`ext2readdir` 访问，普通的 `ls`、`cat` 完全不知道它的存在。VFS 层把非原生文件系统包装成 `vnode`，让它们通过统一的文件描述符接口工作：

```
struct vnode_ops {                                // kernel/vfs.c, 参数是挂载点内的相对路径
    int (*stat)(char*, struct stat*);
    int (*read)(char*, uint64 off, void*, int n);
    int (*write)(char*, struct fat32_file*, uint64 off, int user_src, uint64 src, int n);
    int (*readdir)(char*, int index, void*);
    int (*create)(char*);
    int (*truncate)(char*, struct fat32_file*);
    int (*rename)(char*, char*);
};
```

这些操作以路径为参数而不是以 `inode` 为参数，后端每次按路径重新查找。这是一个极简的 VFS：没有 `dentry` 缓存，也没有把原生 `xv6 inode` 纳入 `vnode`，但足以让 `ls`、`cat`、`mv` 等普通命令透明地访问其他文件系统。

`struct file` 增加了 `FD_VNODE` 类型，`fileread()`、`filestat()`、`fileclose()` 遇到它就转发给对应的 `vnode_ops`。

6.6.2 挂载表与路径路由

`sys_open()` 先调用 `vfsopen()`，它在挂载表里做最长前缀匹配：

```
open("/boot/config.txt")
-> findmount(): 绝对路径 /boot/config.txt, 匹配挂载点 /boot, 相对路径 /config.txt
-> fat32_ops.stat("/config.txt")
```

返回值约定是这一层设计的关键：1 表示 VFS 已打开；-1 表示路径属于某个挂载点但打开失败（不能退回原生根文件系统，否则会打开挂载点下面被遮住的同名文件）；0 表示不属于任何挂载点，继续走原生的 `namei()`。

6.6.3 三个后端

- **procfs** (`/proc`)：没有磁盘数据，打开目录时遍历进程表生成每个 PID 的目录，读 `status` 时持锁生成快照。另有 `/proc/meminfo`、`/proc/iomem`、`/proc/devices` 等诊断文件；
- **ext2** (`/mnt/ext2`)：支持 1/2/4 KiB 块，遇到 `ext4 extent` 等不兼容特性就拒绝挂载，可以和 Linux 交换文件。最初只能读、只支持一级间接块；后来补成了可读写、带外置事务日志的后端，见第 9 章；
- **FAT32** (`/boot`)：挂载启动分区，支持根目录的读、创建、截断、顺序写和重命名，支持 ASCII 长文件名。这让系统可以在运行时更新自己的内核：tftp 下载 `kernel8-xv6_wifi.img` 到 `/boot`，重启即生效。

6.6.4 由用户态决定挂载

挂载策略仿照嵌入式 Linux：内核只负责探测驱动，init 读取 `/etc/fstab` 决定挂载什么。首次启动时 init 创建：

```
proc    /proc      procfs ro 0 0
bootfs  /boot       fat32  rw 0 0
ext2    /mnt/ext2   ext2   rw 0 0
```

然后逐行调用 `mount(source, target, fstype, flags)`。启动日志出现 “vfs: mounted fat32 at /boot read-write” 才说明 FAT32 真的挂上了；否则 `/boot` 只是 init 创建的空目录。

6.7 FAT32 与启动分区

前面几节讲的是 xv6 自己的文件系统。SD 卡上还有一个 xv6 无法回避的 FAT32 分区：树莓派的固件只认 FAT32，内核镜像、`config.txt`、Wi-Fi 固件都必须放在这里。本节先看这个分区里放了什么、谁来读，再看 FAT32 在磁盘上的组织，最后逐个讲 `kernel/fat32.c` 里的函数。

6.7.1 bootfs 里有什么，谁来读

图 6.2 按读者把启动分区的文件分成三组。

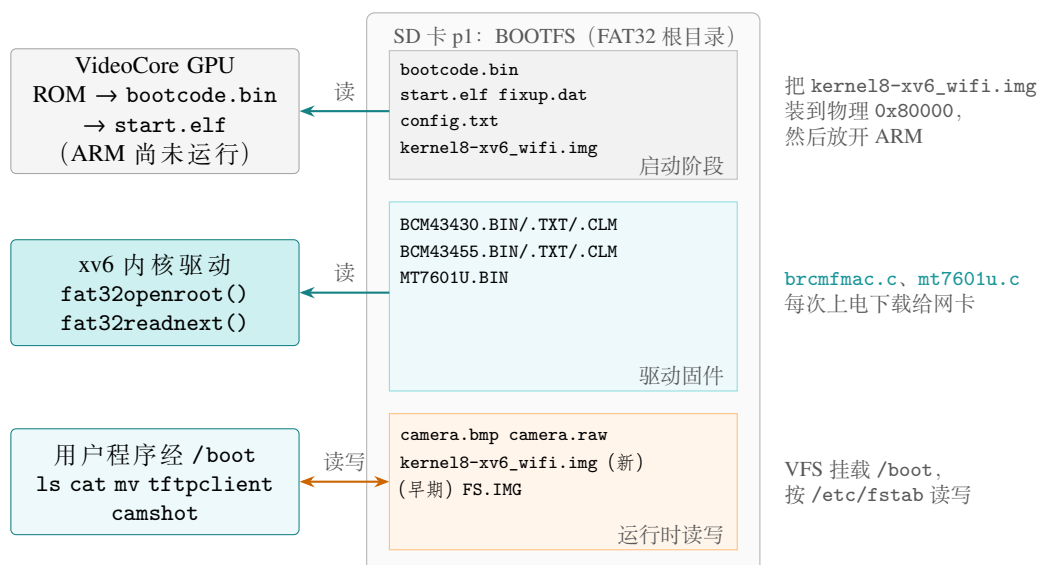


图 6.2: 启动分区里的文件和它们的三类读者。同一个 FAT32 分区，先被 GPU 固件读（ARM 还没开始执行），再被 xv6 内核驱动读，最后作为 `/boot` 被用户程序读写

第一组由 GPU 读，在 ARM 执行第一条指令之前。树莓派上电后先运行的是 VideoCore GPU：片内 ROM 读 `bootcode.bin`，它再装入 `start.elf` 和 `fixup.dat`。`start.elf` 解析 `config.txt`：

```
arm_64bit=1           # 以 AArch64 启动 ARM 核
enable_uart=1         # 打开 Mini UART，固定 core 时钟
uart_2ndstage=1       # 固件自己的启动日志也从串口输出
disable_commandline_tags=1 # 不往内存里写 ATAG/设备树
core_freq=250         # VPU 时钟 250 MHz (Unicam 的下限，串口波特率也依赖它)
core_freq_min=250
```

```
kernel=kernel8-xv6_wifi.img # 内核镜像文件名
camera_auto_detect=0        # 不让固件占用摄像头
```

然后把 `kernel8-xv6_wifi.img` 原样拷到物理地址 0x80000，才放开 ARM 核（第 3 章）。所以内核镜像必须是裸二进制而不是 ELF，固件也从不理会 xv6 的根文件系统。

第二组由 xv6 内核驱动读。 两块 Wi-Fi 网卡每次上电都要由主机下载固件：BCM43430/43455 需要 `.BIN`（R4 的执行代码）、`.TXT`（NVRAM）和 `.CLM`（法规数据库），MT7601U 需要 `MT7601U.BIN`。驱动在 `main()` 里初始化，这时还没有任何进程，更谈不上挂载点，所以它们直接调用 `fat32openroot()` 按 8.3 短名在根目录里找文件，再用 `fat32readnext()` 顺序读出。这也是挂起 bootfs 的 `rootdev_init()` 必须排在 `brcmfmac_driver_init()` 之前的原因。

第三组由用户程序经 /boot 读写。 进入用户态以后，`init` 按 `/etc/fstab` 把同一个分区挂到 `/boot`（6.6 节）。于是 `ls /boot`、`cat /boot/config.txt` 都走普通的 `open/read`；`tftpclient` 把新内核下载成 `/boot/kernel8-xv6_wifi.img`，重启即生效；`camshot` 把超过 xv6 单文件上限的照片写成 `/boot/camera.bmp`，关机后插到电脑上就能看到。

6.7.2 磁盘上的 FAT32

FAT32 的结构可以用一句话概括：文件内容按簇存放，簇与簇之间的先后关系记在一张表（*FAT*）里，目录本身也是一个文件。图 6.3 是它在 SD 卡上的位置和内部划分。

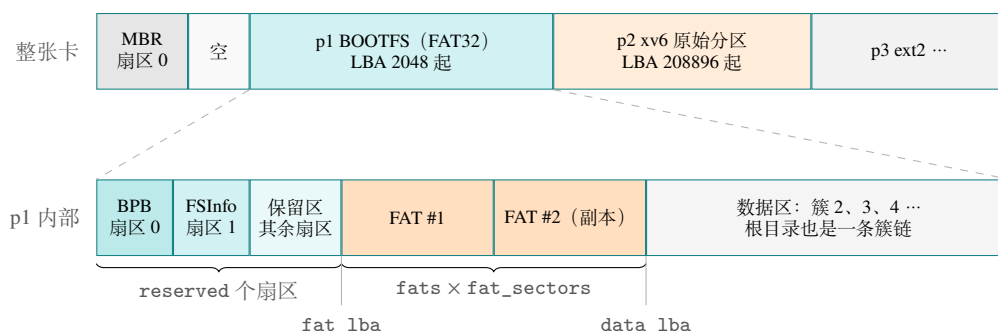


图 6.3: SD 卡与 FAT32 分区的磁盘布局。`fat32mount()` 从 BPB 读出各区的大小，算出 `fat_lba` 和 `data_lba`；簇号从 2 开始编号

MBR 和分区表。 整张卡的第 0 扇区是 MBR：偏移 446 起是 4 个 16 字节的分区表项，最后两个字节是签名 0x55 0xAA。每个表项里第 4 字节是分区类型（0x0B/0x0C 是 FAT32），第 8、12 字节是起始 LBA 和扇区数。读分区表的是 `rootdev.c` 的 `rootdev_init()`，而不是 FAT32 驱动。它先把四个表项全部打印出来，再分两轮挑选：第一轮找 block 1 带 xv6 superblock 魔数的分区做根文件系统（`setup_raw_root()`），第二轮把第一个 FAT32 分区的起点交给 `fat32mount()`。两轮互不依赖，所以根文件系统和启动分区可以各在各自的分区里。FAT32 驱动的工作从 `fat32mount()` 拿到分区起点开始。

BPB：分区的第一个扇区。 FAT32 分区的第 0 扇区叫 BPB（BIOS Parameter Block），记录了整个分区的几何参数。`fat32mount()` 用到的字段如表 6.2。

有了这些字段，任何簇号都能换算成 SD 卡上的绝对扇区号：

```
fat_lba = partition_lba + reserved;
data_lba = fat_lba + fats * fat_sectors;
static uint32 cluster_lba(uint32 cluster) { // 簇号从 2 开始
```

表 6.2: fat32mount() 读取的 BPB 字段

偏移	字段	用途
11	每扇区字节数	必须是 512，否则拒绝
13	每簇扇区数 <code>spc</code>	必须是 2 的幂；一簇通常 4 KiB 或更大
14	保留扇区数 <code>reserved</code>	FAT 表从 <code>partition_lba + reserved</code> 开始
16	FAT 份数 <code>fats</code>	通常为 2，第二份是备份，写入时要同步
32	总扇区数	与 FAT 大小一起算出总簇数
36	每份 FAT 的扇区数	数据区从 <code>fat_lba + fats*fat_sectors</code> 开始
44	根目录首簇	通常为 2；根目录也是一条簇链
48	FSInfo 扇区号	内有“下一个空闲簇”的提示
510	0x55 0xAA	签名

```
return diskmap.data_lba + (cluster - 2) * diskmap.sectors_per_cluster;
}
```

`fat32mount()` 对每个字段都做了合法性检查（2 的幂、不越界、不溢出）——SD 卡上的数据是外部输入，一个损坏的 BPB 不应该让内核算出一个指向别处的 LBA 然后去写它。

6.7.3 FAT 表与簇链

FAT 表是一个巨大的 32 位整数数组，下标就是簇号，第 c 项的值是“簇 c 之后的下一个簇”。只有低 28 位有效，高 4 位保留且写入时要保持原值；0 表示空闲，0x0FFFFFF8 及以上表示链尾（EOC）。一个 512 字节的扇区装 128 项，所以查第 c 项只需读一个扇区：

```
static uint32 fat_next(uint32 cluster) {
    uint32 sector = diskmap.fat_lba + cluster / 128;    // 128 = 512 / 4
    uint32 pos     = (cluster % 128) * 4;
    if(sdsector(sector, scratch, 0) < 0) panic("fat32: read FAT");
    return le32(scratch + pos) & 0x0fffffffU;
}
```

`fat_set_next()` 是对应的写操作，它要把同一个扇区在每一份 FAT 里都读改写一遍，保证两份副本一致。图 6.4 下半部分画的就是一条簇链：从目录项里拿到首簇，然后反复 `fat_next()` 直到 EOC。

6.7.4 目录项与长文件名

目录的内容是一串 32 字节的目录项（图 6.4 上半部分）。短名目录项里有 8.3 格式的名字（11 字节，大写、空格填充）、属性字节、首簇号（拆成高低两个 16 位字段，这是 FAT16 向 FAT32 扩展时留下的痕迹）和文件大小。扫描目录时，首字节有两个特殊值：0x00 表示后面再没有目录项，0xE5 表示这一项已被删除。属性字节的 0x10 表示子目录，0x08 表示卷标。

长文件名（LFN）是后来加的：属性等于 0x0F 的目录项不是文件，而是一片名字，每片装 13 个 UTF-16 字符，倒序排在它所属的短名目录项前面。第一片（物理上最靠前的）序号带 0x40 标志，表示“最后一片”；每片都存着短名的校验和，以便发现“LFN 和短名不配套”（例如老系统删了短名但留下了 LFN）。`fat_find_path()` 扫描目录时把遇到的 LFN 片按序号拼进一个缓冲区，遇到短名项时检查三件事——片序号连续到 1、校验和与短名一致、拼出的名字与要找的名字相同（不区分大小写）——满足才算长名匹配；否则再按短名比较。

创建文件时反过来：`fat32createfile()` 先用 `fat_make_short_alias()` 造一个 `KERNEL~1.IMG` 这样的短名（取前 6 个字母数字、加 ~1~~9 避免重名），算出需要几片 LFN，用 `fat_reserve_root_slots()` 在根目录里找到足够的连续空位（不够就给根目录分配新簇），按倒序写入 LFN 片和短名项，最后补一个全 0 的目录项作为结束标记。



图 6.4: 长文件名、目录项与簇链。文件 `kernel8-xv6_wifi.img` 占两片 LFN 加一个短名目录项; 短名项里的首簇号是 5, 沿 FAT 表 5→6→9 直到 EOC 就是它的全部数据簇

本项目的 FAT32 驱动只处理根目录: 查找、创建、枚举、重命名都在根目录簇链里进行, 不支持子目录, LFN 也只支持 ASCII 字符。对启动分区来说这已经足够——上面列出的所有文件都在根目录里。

6.7.5 函数一览与调用关系

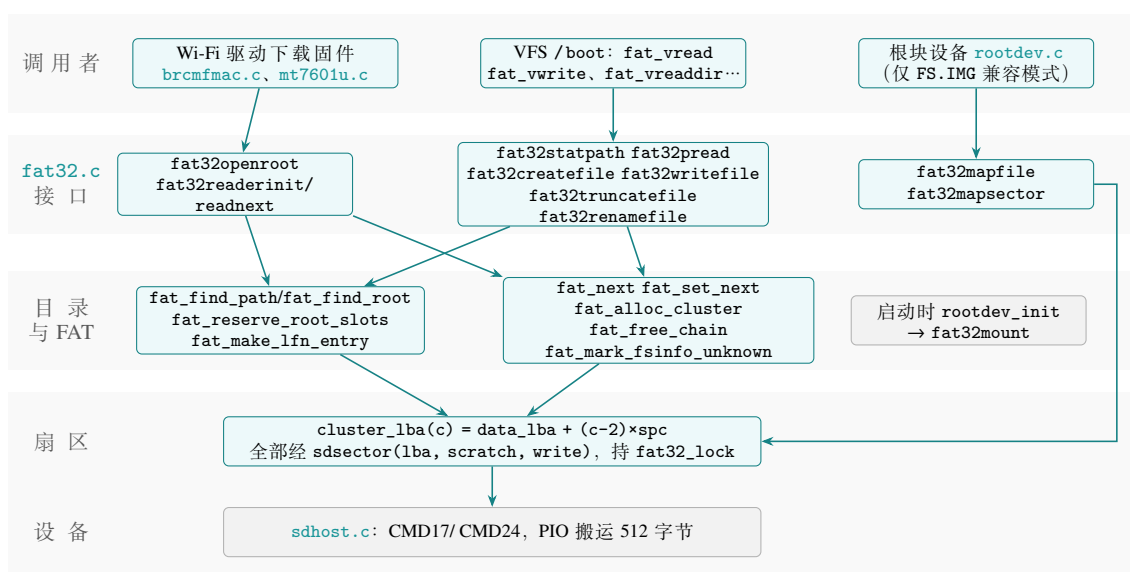


图 6.5: FAT32 驱动的调用关系。三类调用者共用同一组目录与 FAT 操作, 最终都落到一个 512 字节的 `scratch` 扇区缓冲和 `sdsector()` 上

`fat32.c` 的函数可以按图 6.5 分成四层:

- **挂载:** `fat32mount(lba, sectors)` 解析 BPB、读 FSInfo 里的空闲簇提示; `fat32_is_boot_sector()` 判断一个扇区是不是 FAT32 引导扇区, `rootdev.c` 用它识别“没有分区表、整张卡就是一个 FAT32”的 `superfloppy` 格式; `fat32init()` 只初始化 `fat32_lock`。分区表扫描和 xv6 根分区识别都不在这个文件里。
- **对外接口:** 只读的 `fat32openroot()`、`fat32openpath()`、`fat32statpath()`、`fat32readdirroot()`、`fat32pread()`、`fat32readerinit()/fat32readnext()`; 写入的 `fat32createfile()`、`fat32openwrite()`、`fat32writefile()`、`fat32truncatefile()`、`fat32renamefile()`。VFS 的 `fat32_ops` 把它们包装成 /boot 上的 `vnode` 操作。

- **目录与 FAT**：上一小节的查找和 LFN 函数，以及 `fat_next()`、`fat_set_next()`、`fat_alloc_cluster()`、`fat_free_chain()`、`fat_mark_fsinfo_unknown()`。
- **文件扇区映射**：`fat32mapfile()` 记下根目录中某个文件的整条簇链，`fat32mapsector()` 把“文件内第 s 个扇区”换成卡上的扇区号。它们只为兼容早期把根镜像存成 FS.IMG 文件的旧卡而保留，调用者是 `rootdev.c`。FAT32 只回答“这个扇区在哪里”，从不解释扇区里的 xv6 数据。

两个读接口的区别值得一提。`fat32pread(file, offset, ...)` 是无状态的：每次调用都从首簇沿链走 $\lfloor \text{offset}/\text{簇大小} \rfloor$ 步，适合 VFS 偶尔的随机读。如果用它按 512 字节分块读一个 600 KiB 的 Wi-Fi 固件，总共要走 $O(n^2)$ 次 `fat_next()`。`fat32reader` 则记住当前簇和簇内偏移，顺序读时每跨一个簇只查一次 FAT——固件下载就用它。

6.7.6 写入：分配、追加与截断

`fat32writefile()` 只支持追加：要求写入位置正好等于文件当前大小。写入过程是：

1. 重新按路径查找目录项，核对首簇和大小与调用者手里的 `fat32_file` 一致，防止文件已被别人改过；
2. 写入跨进新簇时调用 `fat_alloc_cluster()`：从 FSInfo 提示的位置开始找 FAT 值为 0 的项，把它标成 EOC（每份 FAT 都写），把这一簇清零；再用 `fat_set_next()` 把它挂到链尾；
3. 按扇区写数据：整扇区直接写，不满一个扇区的先读出来再改再写回；
4. 最后一次性更新目录项里的首簇和大小。

`fat32truncatefile()` (`O_TRUNC`) 的顺序恰好相反：先把目录项的首簇和大小清零并写回，再释放旧簇链。这个顺序的意义在于断电时的后果：先改目录项，最坏情况是旧簇成了没人引用的“丢失簇”，浪费一点空间；如果先释放簇链，断电后目录项会指向已被标为空闲、随时可能分给别人的簇，那才是真正的损坏。分配和释放之后，`fat_mark_fsinfo_unknown()` 把 FSInfo 里的空闲簇计数标成“未知” (`0xFFFFFFFF`)，只更新“下一个空闲簇”提示，避免留下一个过时的计数让别的系统误信。

两处性能问题都是真机上暴露出来的，修法相同——缓存当前 FAT 扇区：

- `fat_alloc_cluster()` 最初对每个候选簇调用一次 `fat_next()`，同一个扇区要重复读 128 次，根目录一长就像卡死了一样；现在扫描时只在跨扇区时才读；
- `fat_free_chain()` 最初对每个簇做一次读和两份 FAT 的读改写，截断一张 320×240 的 BMP 要五百多次 SD 事务（第 7 章 7.9.9 小节）；现在在一个扇区内批量清除链项，换扇区时才把整扇区写到每份 FAT。

并发与局限。 所有接口都持同一把自旋锁 `fat32_lock`，共用一个 512 字节的 `scratch` 扇区缓冲，没有块缓存，也没有日志。路径解析、LFN 字符、目录槽这些较大的临时数组也放在一个静态工作区 `fat_work` 里，而不是放在栈上——每个进程的内核栈只有 4 KiB，它们曾经把创建文件的调用链撑爆（第 7 章 7.9.9 小节）；既然有 `fat32_lock` 串行化，一个工作区就够用。这让实现很简单，代价也很清楚：持锁期间 `sdsector()` 以 PIO 方式等待 SD 卡，一次大文件写会长时间占着这把锁；同一时间只能有一个写者；写入途中断电可能损坏分区。启动分区平时几乎只读，偶尔更新内核或写一张照片，这些代价是可以接受的，但修改它之前最好备份。

6.8 xv6 根文件系统：superblock、inode 与目录

前一节分析了启动分区上的 FAT32，这一节用同样的方法看 p2 上的 xv6 根文件系统：它怎么被认出来，磁盘上每一块放的是什么，文件和目录怎样表示，一个路径怎样一步步变成 SD 卡上的扇区。6.5 节讲过根文件系统可以放在哪里；这里只讨论现在的“原始分区”方式。

6.8.1 根文件系统怎样被认出来

`rootdev_init()` 扫描 MBR 时，对每个既不是 FAT32 也不是扩展分区的主分区调用 `setup_raw_root()`（两者都在 `kernel/rootdev.c`）：

```
static int setup_raw_root(uint32 start, uint32 sectors) {
    // xv6 的第 1 块是 superblock; 块大小 1024, 所以它在分区起点之后第 2 个扇区
    if(start == 0 || sectors < FSSIZE * SECTORS_PER_BLOCK ||
        sdsector(start + SECTORS_PER_BLOCK, sector, 0) < 0 ||
        le32(sector) != FSMAGIC || le32(sector + 4) != FSSIZE)
        return -1;
    root.mode = ROOT_RAW;
    root.lba = start;
    root.sectors = sectors;
    return 0;
}
```

判断只看两样东西：superblock 开头的魔数 0x10203040，以及紧跟着的 size 是否等于内核编译时的 FSSIZE。MBR 里的分区类型不作数——macOS 的 diskutil 只能把这个分区建成 Linux 的 0x83 类型，和真正的 ext2 分区一样，靠魔数才能分开。size 的检查同样重要：如果有人改了 FSSIZE 重新编译内核却没重新烧 fs.img，这里会拒绝，而不是让新内核去解释旧格式的磁盘。识别成功后启动日志打印：

```
rootdev: raw xv6 root p2 lba=208896 sectors=819200 size=400 MiB
```

此后块缓存的每一次读写都经过 rootdev_rw()：

```
void rootdev_rw(struct buf *b, int write) {           // kernel/rootdev.c
    uint32 first = b->blockno * SECTORS_PER_BLOCK;    // 1 块 = 2 扇区
    gate_enter();                                     // /dev/sdroot 更新时在此挡住
    for(int i = 0; i < SECTORS_PER_BLOCK; i++)
        if(sdsector(root_sector_lba(first + i),
            b->data + i * SECTOR_SIZE, write) < 0)
            panic(write ? "rootdev: write block" : "rootdev: read block");
    gate_exit();
}
```

原始分区模式下 root_sector_lba() 只做一次加法：扇区号 = root.lba + 块号 × 2 + i。整条路径不经过 fat32.c 的任何函数。分区有 400 MiB，文件系统只用前 32 MiB；可分配多少块由 superblock 决定，与分区大小无关。

6.8.2 superblock 与磁盘布局

块号（1 KiB 一块）；分区内的扇区号 = 块号 × 2

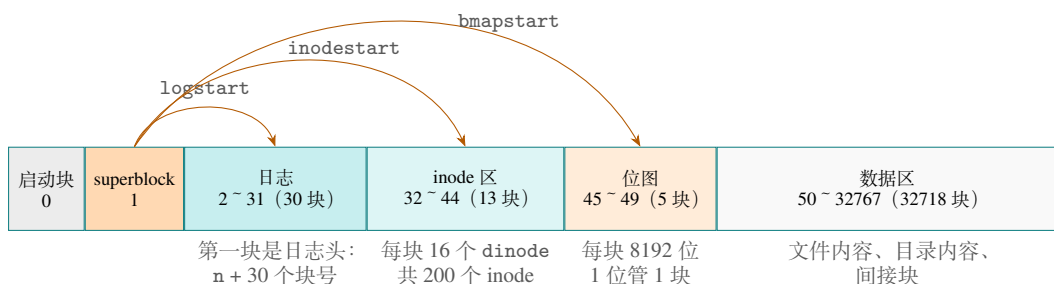


图 6.6: mkfs 生成的 32 MiB 根文件系统布局。所有数字都由 FSSIZE、NINODES、LOGSIZE 算出，并写进第 1 块的 superblock

superblock 是第 1 块（第 0 块是从 Unix 继承下来的“启动块”，xv6 不用），内容就是 8 个 32 位整数：

表 6.3: fs.img 的 superblock (struct superblock, kernel/fs.h)

字段	值	含义与来源
magic	0x10203040	FSMAGIC; fsinit() 和 setup_raw_root() 都检查它
size	32768	总块数, FSSIZE; 32 MiB
nblocks	32718	数据块数 = size - 元数据块数 50
ninodes	200	mkfs 里的 NINODES
nlog	30	LOGSIZE = MAXOPBLOCKS×3
logstart	2	紧跟 superblock
inodestart	32	2 + 30; inode 区 200/16 + 1 = 13 块
bmapstart	45	32 + 13; 位图 32768/8192 + 1 = 5 块

这些数都是 mkfs 在宿主机上算出来写进去的（运行 make fs.img 时会打印 nmeta 50 (boot, super, log blocks 30 inode blocks 13, bitmap blocks 5) blocks 32718 total 32768）。内核启动时 fsinit() 读第 1 块、核对魔数、交给日志层，之后所有“某个东西在哪一块”的问题都由 superblock 加两个宏回答：

```
#define IPB (BSIZE / sizeof(struct dinode)) // 每块 16 个 inode
#define IBLOCK(i, sb) ((i) / IPB + sb.inodestart) // inode i 所在的块
#define BPB (BSIZE * 8) // 每个位图块管 8192 块
#define BBLOCK(b, sb) ((b) / BPB + sb.bmapstart) // 块 b 的位所在的位图块
```

日志区。 30 块中第一块是日志头 struct logheader：一个计数 n 加上 n 个“这些块属于哪个目标块号”。其余是被修改的块的副本。提交时先写副本、再写日志头（这一写就是提交点）、再把副本拷回原位、最后把 n 清零；fsinit() → initlog() 在启动时检查日志头，n 不为 0 就重做一遍。

位图。 每一位对应一块，1 表示已用。mkfs 把前 freeblock 块（元数据加上镜像里已经写入的全部文件内容）一次性标成已用；运行时 balloc() 从头扫描位图找第一个 0 位，置 1、log_write、再把新块清零。早期 FSSIZE 只有 1000 块时，光是打包进镜像的程序就用掉 986 块，首次启动创建 /etc 配置时 balloc 找不到 0 位，panic “out of blocks”——这就是 FSSIZE 改成 32768 的原因。

6.8.3 inode：文件的元数据与块映射

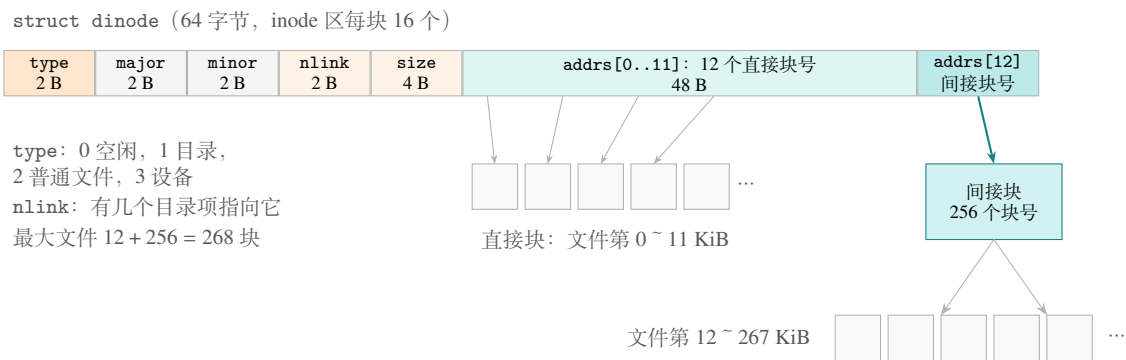


图 6.7: 磁盘上的 inode 和它的块映射。bmap(ip, bn) 把文件内第 bn 块翻译成磁盘块号：小于 12 直接查 addrs[bn]，否则读间接块再查第 bn-12 项；遇到 0 就当场 balloc() 一块

每个文件、目录和设备节点都由一个 64 字节的磁盘 inode 描述（图 6.7）。它不包含文件名——名字在目录里——只有类型、链接数、大小和数据块的位置。内存里的 struct inode 在此之外再加上 dev、inum、引用计数 ref、睡眠锁和 valid 标志（是否已从磁盘读入）。iget() 只在 inode 缓存里占一个槽、不读盘；ilock() 第一次加锁时才 bread(IBLOCK(...)) 把磁盘 inode 拷进来；修改后 iupdate() 写回。

块映射由 bmap() 完成：

```
static uint bmap(struct inode *ip, uint bn) {
    uint addr, *a;
    struct buf *bp;
    if(bn < NDIRECT){                                // 前 12 块：直接块
        if((addr = ip->addrs[bn]) == 0)
            ip->addrs[bn] = addr = balloc(ip->dev);
        return addr;
    }
    bn -= NDIRECT;
    if(bn < NINDIRECT){                              // 之后 256 块：经间接块
        if((addr = ip->addrs[NDIRECT]) == 0)
            ip->addrs[NDIRECT] = addr = balloc(ip->dev);
        bp = bread(ip->dev, addr);
        a = (uint*)bp->data;
        if((addr = a[bn]) == 0){
            a[bn] = addr = balloc(ip->dev);
            log_write(bp);
        }
        brelse(bp);
        return addr;
    }
    panic("bmap: out of range");
}
```

所以单个文件最大 $12 + 256 = 268$ 块，即 268 KiB。这是 xv6 根文件系统最明显的限制：Wi-Fi 固件、640×480 的 BMP (900 KB)、原始帧 (375 KB) 都放不下，只能放在 FAT32 的 /boot (6.7 节)。

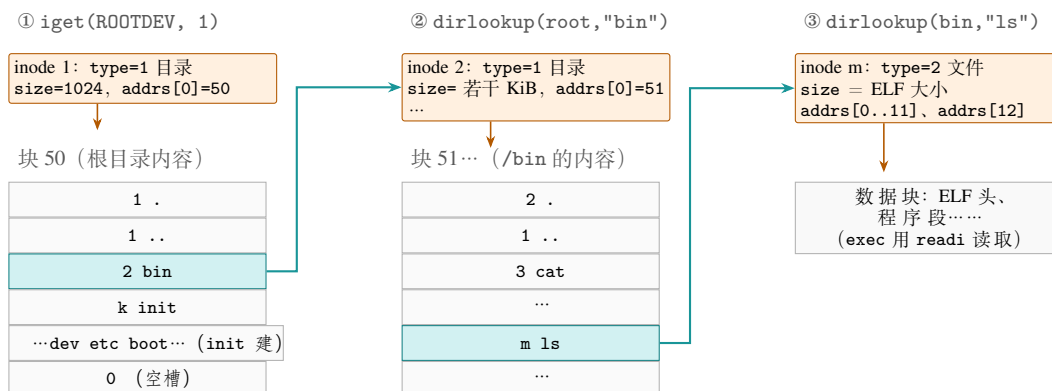
根文件系统里几个特殊的 inode 号：1 号永远是根目录 (ROOTINO)，namex() 遇到以 / 开头的路径就从它开始；2 号是 mkfs 建的 /bin；之后是按 Makefile 顺序装入的每个用户程序。init 是唯一一个在根目录下也有名字的程序（启动代码执行的是 exec("init")），所以 /init 和 /bin/init 是同一个 inode 的两个目录项，mkfs 把它的 nlink 设为 2。

6.8.4 目录与路径查找

目录就是 type=1 的文件，内容是一串 32 字节的 struct dirent：2 字节 inode 号加 30 字节文件名 (DIRSIZ 从原版的 14 扩到 30，见 6.2 节)，一块恰好 32 项，inum=0 表示空槽。mkfs 把目录大小向上取整到整块。目录操作只有两个基本函数：dirlookup(dp, name) 顺序读目录内容、比较名字，命中就 iget() 对应的 inode；dirlink(dp, name, inum) 先确认不重名，再找第一个空槽（没有就追加在末尾）写入新目录项。

路径查找是 namex()，图 6.8 画的是 namei("/bin/ls")：

```
if(*path == '/') ip = iget(ROOTDEV, ROOTINO);        // 绝对路径从 inode 1 开始
else             ip = idup(myproc()->cwd);           // 相对路径从当前目录开始
while((path = skipelem(path, name)) != 0){           // 依次取出 "bin"、"ls"
    ilock(ip);
    if(ip->type != T_DIR){ iunlockput(ip); return 0; }
    if(nameiparent && *path == '\\0'){ iunlock(ip); return ip; } // 停在父目录
    if((next = dirlookup(ip, name, 0)) == 0){ iunlockput(ip); return 0; }
    iunlockput(ip);
    ip = next;
}
```



每个目录项 32 字节: `ushort inum + char name[30]`, 每块 32 项; `inum=0` 表示空槽。
`/init` 与 `/bin/init` 是同一个 inode `k` 的两个名字, 所以它的 `nlink=2`。

图 6.8: `namei("/bin/ls")` 的查找过程。目录本身也是文件, 内容是一串目录项; `nameex()` 每次用 `skipelem()` 取出一个路径分量, 在当前目录的内容里 `dirlookup()`, 得到下一个 inode。块号 50、51 是 `mkfs` 的分配顺序决定的, `k`、`m` 取决于 `Makefile` 里程序的排列顺序

```
}

```

注意加锁的顺序: 每一步只锁当前目录, 查到下一级以后先 `iunlockput()` 当前目录再进入下一级, 任何时刻最多持有一把 inode 锁, 所以不会因为两个进程以相反顺序查找路径而死锁。`nameiparent()` 在最后一个分量之前停下, 返回父目录并把最后一个名字留给调用者——`create`、`unlink`、`link` 都需要它。

根目录的内容分两部分: `mkfs` 写入的 `.`、`..`、`bin`、`init`; 以及 `init` 首次启动时创建的 `dev` (含 `console`、`ttyS0`、`input/event0`、`video0`、`sdroot` 等设备节点)、`etc` (`fstab` 等配置)、`proc`、`boot`、`mnt/ext2`、`root`、`usr/bin`。其中 `proc`、`boot`、`mnt/ext2` 只是空目录, 挂载以后会被 VFS 的挂载表“盖住”(6.6 节)。设备节点是 `type=3` 的 inode, 不占数据块, 只在 `major/minor` 里记下交给哪个驱动。

6.8.5 从路径到 SD 扇区

把这一节和前面几节串起来, 读 `/bin/ls` 的第 0 字节要经过这些层:

```
namei("/bin/ls")      -> inode m (经 inode 1、inode 2 的目录内容)
readi(ip, ..., off=0, n) -> bn = off / 1024 = 0
bmap(ip, 0)           -> 块号 B = ip->addr[0]
bread(ROOTDEV, B)      -> 块缓存未命中, 调用 rootdev_rw(b, 0)
root_sector_lba(2B), (2B+1) -> 扇区 208896 + 2B 和 208896 + 2B + 1
sdsector(lba, buf, 0)  -> SDHOST CMD17, PIO 读 512 字节
```

一个 1 KiB 的 xv6 块对应两个 512 字节扇区。写入的路径与此相同, 只是中间多了日志: `writei()` 修改的块经 `log_write()` 记入日志, `end_op()` 提交时才真正经 `rootdev_rw()` 写到 SD 卡。

6.8.6 mkfs 怎样生成 fs.img

`mkfs` 是一个在宿主机上运行的普通 C 程序, 它和内核共用 `kernel/fs.h`, 因此磁盘格式只有一份定义:

1. 按 `FSSIZE`、`NINODES`、`LOGSIZE` 算出各区大小, 把 32768 块全部写 0, 再在第 1 块写入 `superblock`;
2. `ialloc(T_DIR)` 得到 inode 1, 写入 `.` 和 `..` (都指向自己); 再建 inode 2 作为 `/bin`, 在根目录里加 `bin` 项;
3. 对命令行上的每个程序 (`_ls`、`_cat`... 去掉前缀下划线) 分配一个 inode、在 `/bin` 里加目录项、用 `iappend()` 把文件内容逐块追加——它和内核的 `bmap()` 一样, 前 12 块直接放、之后经间接块; `init` 额外在根目录里加一项;

4. 把两个目录的大小取整到整块；最后 `balloc(freeblock)` 把已经用掉的前 `freeblock` 块在位图里标成 1。

`mkfs` 只按顺序分配块和 `inode`，不需要日志，也不处理删除；所有整数都用 `xint()/xshort()` 写成小端序，与 AArch64 内核一致。

6.8.7 在系统里更新根文件系统

根文件系统的格式一旦改动（例如 `DIRSIZ` 从 14 改成 30），就必须整体重写 p2。最初只能把 SD 卡拔到电脑上 `dd`；后来内核增加了一个受保护的字符设备 `/dev/sdroot`，它只允许写启动时 `setup_raw_root()` 选中的那个分区，绝不暴露整张卡。更新流程是：

```
/bin/tftpcclient 192.168.0.195 fs.img          # 下载到 /boot (FAT32)
/bin/dd if=/boot/fs.img of=/dev/sdroot bs=32768
```

`dd` 在写之前先检查镜像大小恰好是 `FSSIZE×BSIZE`，把整个镜像读进内存并校验 `superblock` 的魔数、大小和各区起点——因为一旦开始覆盖 p2，就再也不能从旧的根文件系统读任何东西，连 `dd` 自己的代码也不行（用户程序是静态链接的，已全部在内存里）。打开 `/dev/sdroot` 时内核冻结根文件系统：不再接受新的事务，等已有事务提交、读者退出；然后每写一个扇区就读回比较一次。结束时打印：

```
sdroot: verified 32/32 MiB
sdroot: update complete and verified; reboot now
dd: 33554432 bytes installed and read-back verified
dd: root filesystem is frozen; power-cycle Raspberry Pi now
```

冻结是单向的，无论成败都必须断电重启——内存里的块缓存和 `inode` 缓存描述的是旧磁盘，不能再拿来解释新磁盘。

6.9 烧录 SD 卡

6.9.1 分区布局

一张为 xv6 准备的 SD 卡分成四个区（表 6.4）。

表 6.4: SD 卡分区（63.8 GB 卡的实例）

分区	名称	格式	起始 LBA	用途
p1	BOOTFS	FAT32	2048	固件、 <code>config.txt</code> 、内核、Wi-Fi 固件
p2	XV6FS	原始	208896	xv6 根文件系统（400 MiB，用到 32 MiB）
p3	EXT2	ext2	1030144	Linux 文件交换
p5	SPFS	原始	123654144	预留给其他教学文件系统

6.9.2 步骤

以 macOS 为例（`disk4` 必须先用 `diskutil list` 确认，写错磁盘会毁掉数据）：

```
# 1. 卸载整张卡并重新分区
diskutil unmountDisk force /dev/disk4
diskutil partitionDisk /dev/disk4 4 MBR \
    FAT32 BOOTFS 100MiB  UFS2_EXTFS XV6FS 400MiB \
    UFS2_EXTFS EXT2 R    UFS2_EXTFS SPFS 500MiB

# 2. 写入 xv6 根文件系统（只能写分区 2，绝不能写整卡 /dev/rdisk4）
```

```
make fs.img
make install-rpi3-rawfs RPI3_XV6_DEV=/dev/rdisk4s2

# 3. 把分区 3 格式化成不带 ext4 特性的 ext2
sudo mke2fs -F -t ext2 -L EXT2 /dev/rdisk4s3

# 4. 在 BOOTFS 中放入树莓派官方固件 (bootcode.bin、start.elf、fixup.dat 等),
#    然后安装内核、config.txt 和 Wi-Fi 固件
make install-rpi3 RPI3_XV6_DEV=/dev/rdisk4s2
```

Makefile 对 RPI3_XV6_DEV 做了检查：变量为空时只更新内核、配置和固件，并给出警告；不是 /dev/diskNsm 形式的分区设备则拒绝执行。这些防护是吃过亏之后加上的——dd 写错一个字母就会覆盖整张卡的分表。

6.9.3 日常更新

改了内核之后不必每次拔卡：运行 make，内核会被拷到宿主机的 TFTP 目录；在树莓派上执行

```
tftp 192.168.0.195 kernel8-xv6_wifi.img
```

文件就会下载到 /boot（FAT32 启动分区），重启后生效。只有用户程序或文件系统格式变化时，才需要重新烧录分区 2。

第七章 驱动架构

本章内容

- ❑ Linux 2.6 风格的总线/设备/驱动模型
- ❑ 中断上半部与 workqueue 下半部
- ❑ 控制台、Mini UART 与 TTY
- ❑ USB 与 HID 键盘：描述符、枚举、split 事务、URB、真机排错
- ❑ I2C (BSC)
- ❑ 网络协议栈
- ❑ SDIO 无线网卡 BCM43430/43455
- ❑ USB 无线网卡 MT7601U
- ❑ CSI-2 摄像头 OV5647

7.1 设备模型

7.1.1 为什么不再只用 devsw[]

原版 xv6 只有一张字符设备函数表：inode.major 直接索引 devsw[major].read/write。它够用来驱动一个控制台，但表达不了“硬件设备”“驱动程序”“总线”以及“二者何时绑定”。随着驱动从一个变成十几个，项目引入了 Linux 2.6 设备模型的最小子集 ([kernel/device.h](#))：

```
struct bus_type {
    char *name;
    int (*match)(struct device *dev, struct device_driver *drv);
    int (*probe)(struct device *dev);
    void (*remove)(struct device *dev);
};

struct device_driver {
    char *name; struct bus_type *bus;
    int (*probe)(struct device *dev);
    void (*remove)(struct device *dev);
};

struct device {
    char *name; int id; struct device *parent; struct bus_type *bus;
    struct resource resource[DEVICE_RES_MAX]; int nresource; // MMIO、IRQ
    struct device_driver *driver; void *driver_data;
};

struct file_operations {
    int (*read)(int user_dst, uint64 dst, int n);
    int (*write)(int user_src, uint64 src, int n);
    int (*open)(struct file *f); // 可选：每次 open 的私有状态
    void (*release)(struct file *f);
    int (*fread)(struct file *f, int user_dst, uint64 dst, int n);
};
```

设备和驱动可以按任意顺序注册：先注册的一方等着，后注册的一方触发总线扫描，match() 成功就调用驱动的 probe()。驱动在 probe() 中初始化硬件，然后用 register_chrdev(major, name, fops) 注册字符设备，或者 register_netdev() 注册网卡。

项目中有三条总线：

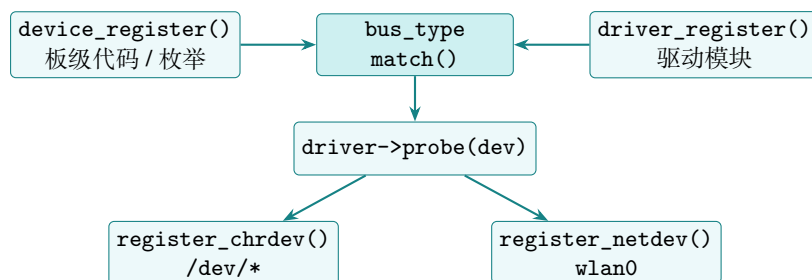


图 7.1: 设备与驱动的绑定

- **platform**: SoC 内固定的设备 (Mini UART、SDHOST、Arasan、DWC2), 无法枚举, 由板级代码静态描述, 按名字匹配;
- **usb**: DWC2 枚举出的 USB 设备, 按 VID/PID 或接口类别匹配;
- **sdio**: Arasan 枚举出的 SDIO function, 按 vendor/device/function 匹配。

字符设备的主设备号见表 7.1。xv6 没有 devtmpfs/udev, 设备节点由 init 用 mknod 创建; cat /proc/devices 可以看到全部总线、设备、资源和已绑定的驱动。

表 7.1: 字符设备主设备号

major	名称	节点
1	console	/dev/console
2	tty (当前进程的控制终端)	/dev/tty
3	ttyS0 (Mini UART)	/dev/ttyS0
4	input (evdev)	/dev/input/event0
5	camera	/dev/video0

7.2 上半部与下半部

中断处理程序运行时, 本 CPU 的中断是关着的, 而且它打断的可能是任何代码, 所以不能睡眠、不能长时间运行。真正耗时的工作——解析 USB 报告、执行几十次 SDIO 命令、处理一个 TCP 段——要推迟到可以调度、可以睡眠的上下文去做。项目实现了与 Linux 同名的 workqueue ([kernel/workqueue.h](#)):

```

struct work_struct {
    void (*func)(struct work_struct *work);
    struct work_struct *next;
    int pending, running, rerun;
    struct workqueue *wq;
};

int queue_work(struct workqueue *wq, struct work_struct *work);
int queue_delayed_work(struct workqueue *wq, struct delayed_work *dw, uint64 delay);
void cancel_work_sync(struct work_struct *work);
  
```

每个 workqueue 有自己的内核线程 (kworker), 在 ps 中可以看到。几条原则贯穿所有驱动:

- 上半部只做三件事: 确认中断来源、屏蔽或清除该中断、queue_work();
- 每个设备使用私有的 work 和 workqueue, 慢设备 (如 SDIO 的 CMD53) 不会阻塞别的下半部;
- 同一个 work 在执行期间再次被排队, 只记一个 rerun 标志, 不会无限堆积;
- 移除设备时先 cancel_work_sync(), 等正在运行的 work 返回后才释放设备内存。

定时器中断每 10 ms 推进一次 delayed-work 时钟, TCP 重传、Wi-Fi 连接超时等都用了 delayed work 实现, 而不是在定时器中断里直接运行协议代码。网络接收还在 workqueue 之上实现了 NAPI 式的预算轮询: 中断只调度

一次，下半部每轮最多处理 16 个报文，没处理完就重新排队，处理完才重新打开中断。

7.3 控制台、UART 与 TTY

7.3.1 Mini UART

Mini UART 驱动 (`kernel/uart.c`) 在 `probe()` 中完成与早期汇编相同的初始化，再打开接收中断：

```
REGS_AUX->enables |= 1U;           // 使能 AUX 中的 Mini UART
REGS_AUX->mu_control = 0;           // 暂停收发
REGS_AUX->mu_ier = 0;
REGS_AUX->mu_lcr = 3;               // 8 位数据
REGS_AUX->mu_iir = 0xc6;            // 清空 FIFO
REGS_AUX->mu_baud_rate = 270;       // 115200 @ core_freq 250 MHz
REGS_AUX->mu_ier = MU_IER_RX_ENABLE; // 接收中断
REGS_AUX->mu_control = 3;           // 打开收发
```

发送使用一个 32 字节的软件环形缓冲区：`uartputc()` 把字符放进缓冲区，缓冲区满时睡眠；`uartstart()` 在发送 FIFO 有空位时搬字符过去，并打开“发送空”中断；`uartintr()` 读空接收 FIFO、把字符交给 `consoleintr()`，再继续发送。缓冲区空了就关掉发送中断，否则会一直触发。内核 `printf` 在 `panic` 等场合使用同步的 `uartputc_sync()`，不依赖中断。

7.3.2 控制台与行规程

`consoleintr()` 实现了最基本的行规程：回显、退格、整行缓冲、Ctrl-D 结束输入、Ctrl-P 打印进程表、Ctrl-C 向前台进程组发送 SIGINT。读进程在 `consoleread()` 里睡眠，直到收到一整行。

7.3.3 TTY 层

`/dev/console`、`/dev/ttyS0`、`/dev/tty` 共用 Mini UART 这一个硬件后端，但含义不同：

- `/dev/console` 是系统控制台，内核日志直接写 UART，不经过文件系统；
- `/dev/ttyS0` 是具体的串口终端，`init` 在上面启动 `login`；
- `/dev/tty` 是“当前进程的控制终端”，按调用者的 `ctty` 动态转发：串口会话转到 `ttyS0`，SSH 会话转到对应的 PTY 从端。

`login` 调用 `tty_attach()` 成为新会话和进程组的组长，并把 `ttyS0` 设为控制终端；之后 `fork` 出的 `shell` 和命令继承会话、进程组和控制终端。`shell` 运行外部命令时把它设为前台进程组，Ctrl-C 产生的 SIGINT 只发给这个组，所以能终止 `ping`、`tftp` 或整条管道，而不会杀掉 `shell` 和 `login`。

一个由此暴露的问题是：旧版 `ps` 通过系统调用让内核 `printf` 直接打印进程表，结果在 SSH 会话里执行 `ps`，输出却出现在串口上。内核 `printf` 写的是系统控制台，不看调用者的 `fd 1`。现在 `sys_ps()` 把快照格式化到用户缓冲区，由 `ps` 自己 `write(1, ...)`。

7.4 USB 与 HID 键盘

USB 键盘是本项目里协议层次最多的驱动：从主机控制器的寄存器，到 Hub、事务转换器、描述符、HID 报告，再到输入子系统和控制台。本节沿着“插上键盘—识别—配置—读按键”的顺序，把每一层讲清楚。

7.4.1 三层软件与硬件路径

USB 驱动分三层：

- **HCD** (`kernel/dwc2.c`)：复位 DWC2 控制器、强制主机模式、开 DMA；给根端口和 Hub 端口上电、复位；用 host channel 发起 control、bulk、interrupt 传输；
- **USB core** (`kernel/usb.c`)：struct `usb_device`、按 ID 或接口类别匹配驱动、Linux 风格的 URB 接口；
- **类驱动**：HID 键盘 (`usbkbd.c`)、CDC-ECM 网卡 (`usbnet.c`, 用于 QEMU)、MT7601U 无线网卡 (`mt7601u.c`)。

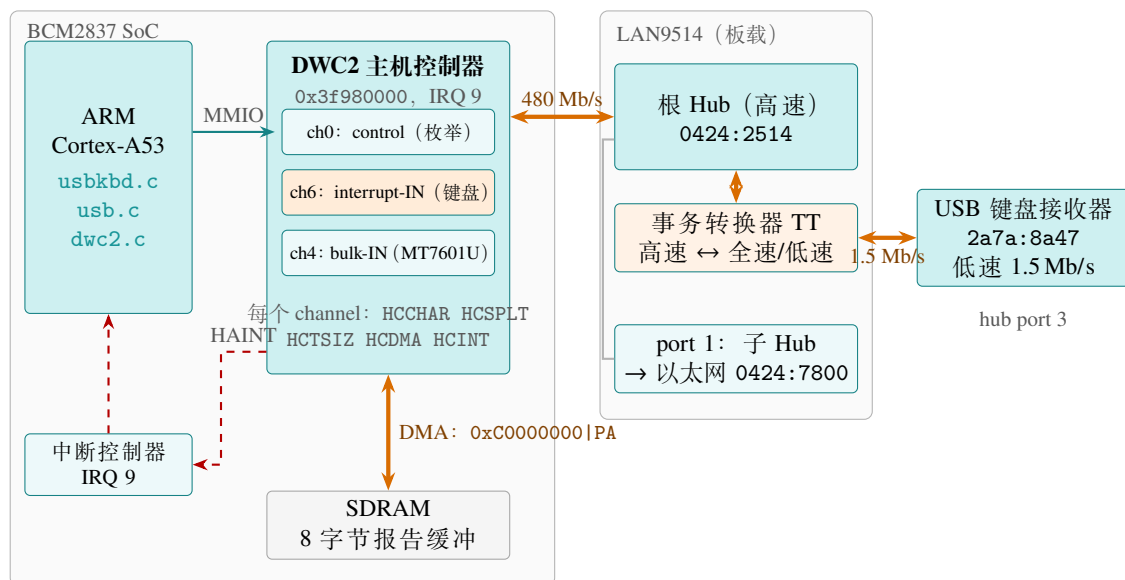


图 7.2: USB 键盘的硬件路径。DWC2 用 host channel 发起事务、用 DMA 读写内存；树莓派 3 的 USB 口都在板载 LAN9514 Hub 后面，低速键盘与 Hub 之间靠事务转换器 (TT) 完成速率转换

图 7.2 是硬件路径。有几个特点和前面的 Wi-Fi、摄像头都不一样：

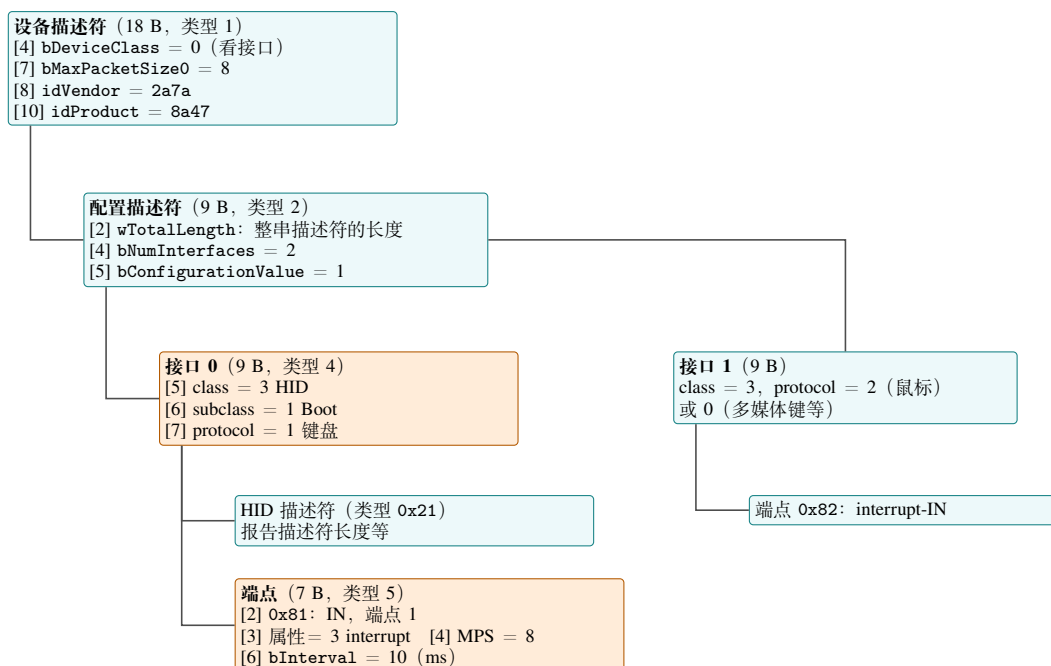
主机发起一切。 USB 是严格的主从结构，设备永远不能主动说话。即使是名为“interrupt”的传输类型，也只是“主机承诺每隔 `bInterval` 毫秒来问一次”：主机发 IN 令牌，设备有数据就回 DATA，没有就回 NAK。所谓“键盘中断”，其实是主机控制器替 CPU 轮询，问到数据后才用 IRQ 9 通知 CPU。

host channel 与 DMA。 DWC2 有 8 个 host channel，每个 channel 是一组寄存器，描述“对哪个设备的哪个端点做一次什么传输”：HCCHAR (设备地址、端点号、方向、传输类型、最大包长、是否低速)、HCSPLT (split 事务参数)、HCTSIZ (字节数、包数、DATA0/1)、HCDMA (缓冲区的总线地址)、HCINT (完成、ACK、NAK、NYET、错误等状态)。本项目固定分工：ch0 做枚举时的 control 传输，ch6 给键盘的 interrupt-IN，ch4 给 MT7601U 的 bulk-IN。和 Arasan 不同，DWC2 自己用 DMA 读写内存，所以缓冲区在提交前要 clean、完成后要 invalidate 缓存，地址也要换成 `0xC0000000 | PA` 的总线地址。

一切都在 Hub 后面。 树莓派 3 的 SoC 只有一个 USB 口，直接接板载 LAN9514，它内部是一个高速 Hub 加一块 USB 以太网卡 (在 3B+ 上还有第二层 Hub)。外面插的键盘一般是低速 (1.5 Mb/s) 设备，而 DWC2 和 Hub 之间跑的是高速 (480 Mb/s)。两种速率之间靠 Hub 里的事务转换器 (TT) 衔接，这就是后面 split transaction 的由来。

7.4.2 描述符：设备、配置、接口与端点

USB 设备用一组描述符介绍自己 (图 7.3)。每个描述符的第 0 字节是长度 `bLength`，第 1 字节是类型。层次是：一个设备有若干配置 (几乎总是 1 个)；一个配置包含若干接口，每个接口是一项独立的功能，有自己的



GET_DESCRIPTOR(Configuration, wTotalLength) 一次返回的字节串，逐项按 bLength 前进：

配置 9	接口 0 9	HID 9	端点 7	接口 1 9	HID 9	端点 7	只在接口 0 范围内挑 interrupt-IN 端点
------	--------	-------	------	--------	-------	------	-----------------------------

图 7.3: 一个 USB 键盘接收器的描述符层次 (示意, 方括号里是字节偏移; VID/PID 取自真机日志)。驱动按类别匹配, 而不是按 VID/PID: 先找 class/subclass/protocol = 3/1/1 的 Boot Keyboard 接口, 再只在这个接口里找 interrupt-IN 端点

class/subclass/protocol; 一个接口有若干端点, 端点是真正收发数据的地方, 有编号、方向、传输类型、最大包长和轮询间隔。端点 0 是所有设备都有的默认控制端点, 枚举全靠它。

很多键盘接收器是复合设备: 接口 0 是键盘, 接口 1 是鼠标或多媒体键。所以驱动不能看设备描述符的 class (它通常是 0, 意思是“看各个接口”), 也不能凭 VID/PID 白名单, 而要遍历配置描述符:

```
// GET_DESCRIPTOR(Configuration) 先取 9 字节拿到 wTotalLength, 再一次取全部
for(int off = 0; off + 9 <= total && ctrl_buf[off] >= 2; off += ctrl_buf[off]){
    if(ctrl_buf[off + 1] == 4 &&                                // 接口描述符
       ctrl_buf[off + 5] == 3 &&                                // class: HID
       ctrl_buf[off + 6] == 1 &&                                // subclass: Boot Interface
       ctrl_buf[off + 7] == 1){                                  // protocol: Keyboard
        sel->interface_number = ctrl_buf[off + 2];
        return 1;
    }
}
```

off += ctrl_buf[off] 按每个描述符自己的长度前进, 所以中间夹着的 HID 描述符 (类型 0x21) 等不认识的类型会被自然跳过; ctrl_buf[off] >= 2 防止一个长度为 0 的坏描述符造成死循环。找到键盘接口后, 后面的端点只在这个接口的范围里挑 (current_interface 限定), 否则复合设备里鼠标的端点会把键盘的覆盖掉。

7.4.3 枚举：control 传输与请求序列

端点 0 上的 control 传输分三个阶段：SETUP 阶段主机发出 8 字节的请求（bmRequestType、bRequest、wValue、wIndex、wLength），可选的 DATA 阶段按最大包长收发数据，STATUS 阶段反方向发一个空包确认。control() 就是在 ch0 上依次执行这三个阶段。表 7.2 是识别一个 Hub 后面的键盘所发出的请求。

表 7.2: 从 Hub 端口到可用键盘的请求序列

对象	请求	bmRequestType	作用
Hub	SET_FEATURE(PORT_POWER)	0x23	给下游端口上电，之后最多等 500ms 连接去抖
Hub	GET_STATUS	0xa3	端口状态：bit 0 连接、bit 1 使能、bit 8 供电、bit 9 低速、bit 10 高速
Hub	SET_FEATURE(PORT_RESET)	0x23	复位端口，设备进入默认状态、地址为 0
设备 0	GET_DESCRIPTOR(Device, 8)	0x80	只取 8 字节，得到端点 0 的最大包长
设备 0	GET_DESCRIPTOR(Device, 18)	0x80	完整设备描述符：class、VID、PID
设备 0	SET_ADDRESS(n)	0x00	分配唯一地址；把父 Hub、端口、速度记入 usb_route[n]
设备 n	GET_DESCRIPTOR(Config, 9)	0x80	得到 wTotalLength
设备 n	GET_DESCRIPTOR(Config, total)	0x80	全部接口和端点，挑出 Boot Keyboard
设备 n	SET_CONFIGURATION	0x00	激活配置，端点开始可用
接口	SET_PROTOCOL(boot)	0x21	让键盘用固定的 8 字节 Boot 报告格式
接口	SET_IDLE(0)	0x21	只在按键状态变化时才回报告

先取 8 字节再取 18 字节是 USB 枚举的惯例：在知道端点 0 的最大包长之前，只能假定它是最小的 8 字节，而设备描述符第 7 字节正好告诉我们真实值。端口状态的例子：真机日志里 Hub port 3 是 0x301（供电、低速、已连接），复位后变成 0x303（多了“使能”）。根 Hub 和嵌套的 Hub 用同一套请求递归扫描，深度限制为两层，防止一个异常的拓扑让启动无限递归。

SET_PROTOCOL(boot) 很关键。HID 设备本来用“报告描述符”描述自己的报告格式，解析它需要一个不小的解释器；而 Boot 协议是 BIOS 时代为了解析报告描述符而规定的固定格式，所有键盘都必须支持。用它，驱动只需理解图 7.4 这 8 个字节。

例：mod=0 keys=10,0,0,0,0,0 表示只按着 HID usage 0x10，即字母 m；
同时按 Shift 时 byte 0 = 0x02；按键超过 6 个时 6 个槽全是 0x01（rollover），驱动保持上一份状态。

修饰键 [0]	保留 [1]	键 1 [2]	键 2 [3]	键 3 [4]	键 4 [5]	键 5 [6]	键 6 [7]
bit 0 LCtrl	bit 1 LShift	bit 2 LAlt	bit 3 LGUI	bit 4 RCtrl	bit 5 RShift	bit 6 RAlt	bit 7 RGUI

图 7.4: HID Boot 键盘报告（8 字节）。byte 0 是 8 个修饰键的位图，byte 2~7 是当前按着的最多 6 个键的 usage 编号

最后，usb_device_register() 把设备交给 USB 总线，总线按 class/subclass/protocol 匹配到 usbhid-keyboard 驱动，调用 usbkbd_probe()：发送上面两个 HID 请求，分配每设备的 usbkbd_state 和 URB，注册 input_dev，然后第一次提交 URB。

7.4.4 Split transaction：高速 Hub 后面的低速设备

DWC2 不能直接对低速设备说话，它要先把事务交给 Hub 里的 TT（图 7.5）：start-split（SSPLIT）告诉 TT “请替我对 port 3 上地址 4 的端点 1 做一次 IN”，TT 回 ACK 表示接受；TT 随后在低速总线上完成真正的事务；主机在随后的微帧里发 complete-split（CSPLIT）取结果，TT 要么回 NYET（还没完成），要么回数据，要么转达设备的 NAK。channel 寄存器里，这些信息体现为：

这里有三条规则，每一条都是在真机上踩过才写进代码的：低速设备的每一个事务都要在 HCCHAR 里置 LSPDDEV；CSPLIT 收到 NAK 说明这次事务已经结束（设备没数据），必须从新的 SSPLIT 重来，而不是继续发 CSPLIT；CSPLIT 收到 NYET 只能在同一个帧的窗口里重试，超出窗口就放弃本轮、等 bInterval 后重新开始。


```
req->last_issue_frame = rd(HFNUM) & 0x3fff;    // 记下发出时的微帧号
```

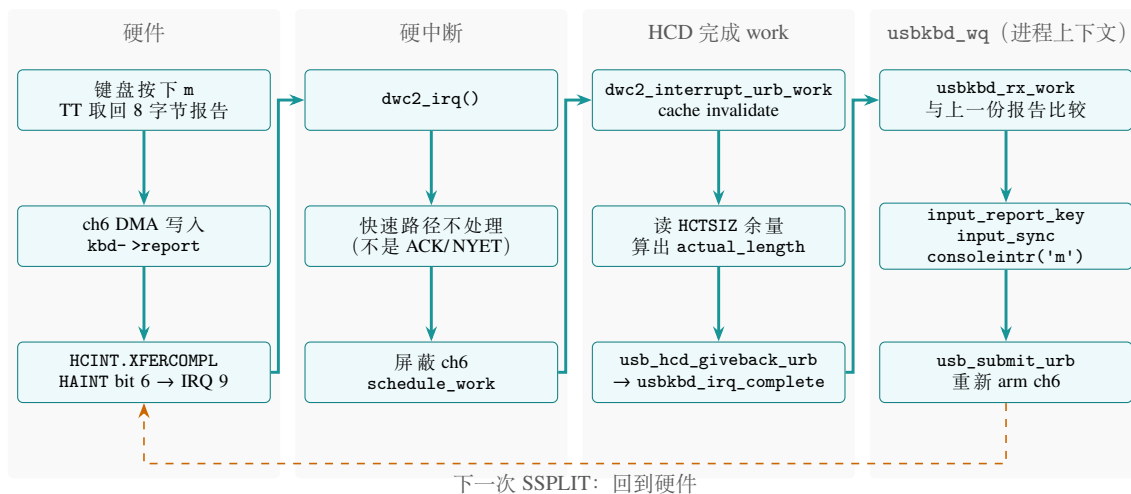


图 7.6: 一份键盘报告从硬件到 shell 的路径。每一列是一种执行上下文：硬中断只做与微帧时序有关的最少工作；HCD 的完成 work 负责缓存维护并把 URB 归还类驱动；HID 解析、输入事件和控制台都在键盘自己的 workqueue 里完成，最后重新提交 URB。键盘回 NAK（没有新按键）时，HCD 完成 work 等 bInterval 后自己重发，URB 不归还类驱动

之后的事全由中断推动（图 7.6）。dwc2_irq() 先交给快速路径 dwc2_interrupt_rx_irq_fast(): SSPLIT 的 ACK 和窗口内的 NYET 在这里当场处理——记下当前微帧，忙等到“发出后第 2 个微帧”（最多 500 μs），立即重发 CSPLIT——因为它们有微帧级的截止时间，交给任何可调度的线程都来不及。只有最终结果（完成、无数据或错误）才屏蔽 ch6 并 schedule_work。HCD 的完成 work 做缓存维护、算实际长度并归还 URB；键盘的完成回调只 queue_work(kbd->rx_work); usbkbd_rx_work() 在进程上下文里比较新旧报告、上报输入事件、调用 consoleintr(), 最后重新 usb_submit_urb()。三种上下文各做各的：硬中断只管时序，HCD work 只管控制器，类驱动只管 HID。

拔出设备时顺序相反：usbkbd_remove() 先标记断开，usb_kill_urb() 停止 ch6 并等正在运行的完成回调返回，cancel_work_sync() 等 rx_work 返回，最后才释放 URB 和设备状态——保证不会有任何回调再碰到已释放的内存。

7.4.6 HID 报告与输入子系统

usbkbd_rx_work() 收到 8 字节报告后与上一份比较：修饰键位图里变化的位分别上报按下/松开；本次出现、上次没有的 usage 是新按下的键，查表转成字符；上次有、本次没有的是松开的键。每份报告最后 input_sync()。按键多到键盘无法分辨时，6 个槽会全填错误码（rollover），这份报告不说明任何键的状态，驱动直接保留上一份，不能当成“全部松开”。

之后的事由输入子系统（kernel/input.c）完成，它把事件分发给两个“处理者”：

- **evdev**：/dev/input/event0。每次 open 都分配一个独立的客户端环形缓冲区（fops.open 设置 f->private_data），读者在自己的缓冲区上睡眠；溢出时插入 SYN_DROPPED；
- **控制台**：把按键转成字符（处理 Shift、Caps Lock、Ctrl），送入 consoleintr()。所以 USB 键盘和串口一样可以登录、输入命令、按 Ctrl-C。

这正是 Linux 的分层：硬件驱动只关心“产生了什么事件”，不关心“谁在读”。

7.4.7 真机排错：从 STALL 到第一份报告

这个驱动在 QEMU 上很快就能工作，到了真机（低速无线键盘接收器 2a7a:8a47 插在 LAN9514 的 port 3）却经历了九个阶段才读到第一个按键。表 7.4 按时间顺序列出每一步的现象、根因和修复；每一步只把串口日志

实际证明的东西当作结论——ACK 不等于下游数据已经返回，“ready”也不等于收到了按键。

表 7.4: USB 键盘真机排错的九个阶段

现象	根因	修复
1 port 3 状态 0x301, 复位后 0x303	低速设备在高速 Hub 后面	记录父 Hub、端口、速度, 走 split 事务
2 control setup failed req=6 addr=0, intr=0x0a	SETUP 的 SSPLIT 没设 LSPDDEV, TT 回 STALL	按 usb_route 给所有低速事务置 LSPDDEV
3 GET_DESCRIPTOR 数据阶段 返回 -2 (超时)	CSPLIT 收到 NAK 后仍反复发 CSPLIT	NAK 时清 complete_split, 下一帧重发 SSPLIT; NYET 才继续 CSPLIT
4 设备描述符 class=0 vid=0 pid=0	18 字节跨 3 个 8 字节包, 被当成一个 split 请求, 只收到前 8 字节	按最大包长分包, 每包单独 SSPLIT/CSPLIT, 完整收到一包后翻转 DATA0/1; 检查长度、类型和 VID
5 “not a boot keyboard”	复合设备的后一个接口覆盖了键盘接口的属性	先定位 Boot Keyboard 接口, 端点只在该接口内选
6 ready 之后又出现 req=6 addr=4 STALL	绑定键盘后仍继续 CDC-ECM 网卡探测, 向键盘 要配置描述符	HID 交给 USB 总线后直接返回
7 按键无响应, 反复 hcint=0x42	把每个 NYET 都当作“TT 还在忙”, 永久重发 CSPLIT	记录 SSPLIT ACK 的帧号, 同帧内最多重试 3 次; 跨帧则结束本轮, 等 bInterval
8 ssack 比 issue 晚几十个微 帧	ACK 交给 kworker 处理, 调度延迟毫秒级, CSPLIT 窗口早过了	SSPLIT ACK 和窗口内 NYET 改在 IRQ 快速路径里处理
9 快速路径启用后仍偶尔跨帧	快速路径之前有一行 printf; 115200 波特率下 一行要几毫秒	中间状态不打印, 只在本轮结束后限量打印 ch6 final

其中几个阶段值得多说几句。

阶段 4: “读到了”但其实没读全。 低速设备端点 0 的最大包长只有 8 字节, 18 字节的设备描述符要分 3 个数据包传。经过 split 事务时, 每个包都要单独走一遍 SSPLIT/CSPLIT, 而且收完一个完整的包后要把 DATA0/DATA1 翻转。旧代码把整个 18 字节当成一次 split 请求, 只取回了第一个包, 后面 10 字节还是缓冲区里的 0——于是日志说 vid=0 pid=0。现在 channel_xfer() 对 split 请求按最大包长切片, get_device_descriptor() 还检查长度、类型和 VID 不为 0, 宁可失败也不接受半截数据。

阶段 7~8: 时间窗口。 周期性的 split 事务有严格的时序: CSPLIT 应在 SSPLIT 之后第 2 个微帧开始, 并且只能在同一个帧内重试。早期代码对 NYET 无条件重试, 永远等不到结果; 改成按帧号判断窗口后, 又发现记录下来的“ACK 帧号”比实际晚了几十个微帧——因为 ACK 是由 kworker 处理的, 而 kworker 要等调度器给它 CPU, 延迟是毫秒级的, 比 125 μs 的微帧粗了一个数量级。解决办法是把这两种有截止时间的中间事件移到中断上下文里 (dwc2_interrupt_rx_irq_fast()), 只把“这一轮已经结束”的结果交给线程。这是“上半部只做最少的事”这条原则的一个反例: 对有硬实时要求的状态推进, 上半部恰恰是唯一来得及的地方。

阶段 9: 观察者效应。 快速路径加上以后, 问题偶尔还在。原因是 dwc2_irq() 开头还留着一行调试 printf: 串口 115200 波特率下每个字符约 87 μs, 一行几十个字符就是好几个毫秒, 足够错过整个窗口。这是典型的“加日志就改变时序”的 Heisenbug。最终版本在有截止时间的分支里一行都不打印, 只在本轮结束之后限量打印 dwc2 ch6 final; 如果确实需要跟踪中间事件, 应该写进内存里的 trace 环形缓冲, 事后由进程上下文打印。

诊断顺序。 整理下来, 按键没反应时按这个顺序看日志: 有没有 ch6 armed (channel 配置并启动了); 按键时有没有 ch6 final (host channel 产生了中断); 有没有 usbkbd: channel 6 IRQ received (URB 已归还、键盘 work 已排队); 有没有 first HID report (DMA 里读到了报告); 报告里的 usage 是否非 0 (键盘真的报了键); 最后才看键码映射和控制台。真机第一次成功时打印的是 usbkbd: first HID report mod=0 keys=10,0,0,0,0,0——usage 0x10, 字母 m。

从两层轮询到中断驱动。 最早的实现其实有两层轮询: 定时器每个 tick 唤醒一次键盘 work, work 里启动一次传输再忙等 HCINT。改成 URB 加中断以后, CPU 只在真的有报告 (或本轮结束) 时才参与, 空闲键盘不再占用任何 CPU 时间。

7.5 I2C

BCM2837 的 I2C 控制器叫 BSC (Broadcom Serial Controller)。项目目前只用到 BSC0，用来配置摄像头传感器。在树莓派 3B+ 上，摄像头排线口的 I2C 走 GPIO44/45 的 ALT1 功能，而不是 40 针排针上的 GPIO0/1。

一次传输的过程是：写从机地址寄存器 A、数据长度 DLEN，清 FIFO，向控制寄存器 C 写入 I2CEN | ST (读还要加 READ)，然后轮询状态寄存器 S：TXW 表示可以继续写 FIFO，RXD 表示 FIFO 有数据可读，DONE 表示传输完成，ERR 表示从机没有应答，CLKT 表示时钟拉伸超时。分频寄存器 DIV = 核心时钟 / 100 kHz。在真机上，BSC0 对摄像头传感器始终返回 NACK，驱动最后改用 GPIO44/45 上的软件 SCCB，排查过程见 7.9.5 小节。

```
int i2c_write(int bus, uint8 addr, const uint8 *buf, int len);
int i2c_read(int bus, uint8 addr, uint8 *buf, int len);
int i2c_write_read(int bus, uint8 addr, const uint8 *wbuf, int wlen,
                  uint8 *rbuf, int rlen);    // 先写寄存器地址，再读
```

摄像头寄存器的访问很短 (写是 2 字节地址加 1 字节数据，读是 2 字节地址后读 1 字节)，所以全部用轮询加超时，不用中断。为了在真机上区分“控制器没配好”和“线上没有设备”，驱动还带了一个用 GPIO 模拟 I2C 的诊断路径：直接把 GPIO44/45 当开漏输出，按位产生起始、地址、应答和停止条件。

7.6 网络协议栈

在讲两块无线网卡之前，先看看它们的上层。[kernel/net.c](#) 参考 MIT xv6 网络实验的分层，逐步扩展成了一个小而完整的协议栈：

- struct net_device: 每块网卡一个，保存 MAC、IP、掩码、网关、DNS 和发送函数 start_xmit；驱动收到帧后调用 net_rx_dev(dev, frame, len)；
- Ethernet、ARP (表项以 (ip, dev) 为键，多网卡互不干扰)、IPv4、ICMP、UDP、DHCP；
- 路由表：最长前缀匹配，同长度按 metric 选择，DHCP 成功后自动加直连路由和默认路由；
- TCP：被动打开 (listen/accept)，每连接 8 KiB 异步发送队列，单段停等式重传；
- socket 文件描述符和 epoll：socket、PTY、管道都是 struct file，可以统一 read/write/epoll。

在这之上，用户态实现了 ping、dhcp、tftp、netdns，以及一个基于 TweetNaCl 的 SSH 服务器 (第 8 章)。

7.7 SDIO 无线网卡：BCM43430/43455

7.7.1 硬件路径

树莓派 3B 板载 BCM43430，3B+ 板载 BCM43455。它们通过 SDIO 接在 Arasan SDHCI 控制器 (0x3f300000，GPIO34~39 ALT3，IRQ 62) 上。这里有两个容易混淆的概念：Arasan 是 SDIO 主机控制器，负责产生 SDIO 命令和时钟、搬运数据；BCM43455 是 FullMAC Wi-Fi 芯片，片内有一颗 Cortex-R4 运行厂商固件，处理大部分 802.11 MAC 工作。

7.7.2 硬件结构：Arasan、SDIO 总线与 FullMAC 芯片

和摄像头一样，先把参与的部件和它们之间的通路理清楚 (图 7.7)。

两颗处理器。 这条链路两端各有一颗 CPU：一端是运行 xv6 的 Cortex-A53，另一端是 BCM43455 片内的 Cortex-R4。R4 执行主机下载进去的厂商固件，负责扫描、认证、关联、管理帧、ACK 与重传、用主机装入的密钥做 CCMP 加解密，以及 802.11 帧与以太网帧之间的转换——这就是 FullMAC 的含义。主机看到的是一个“收发以太

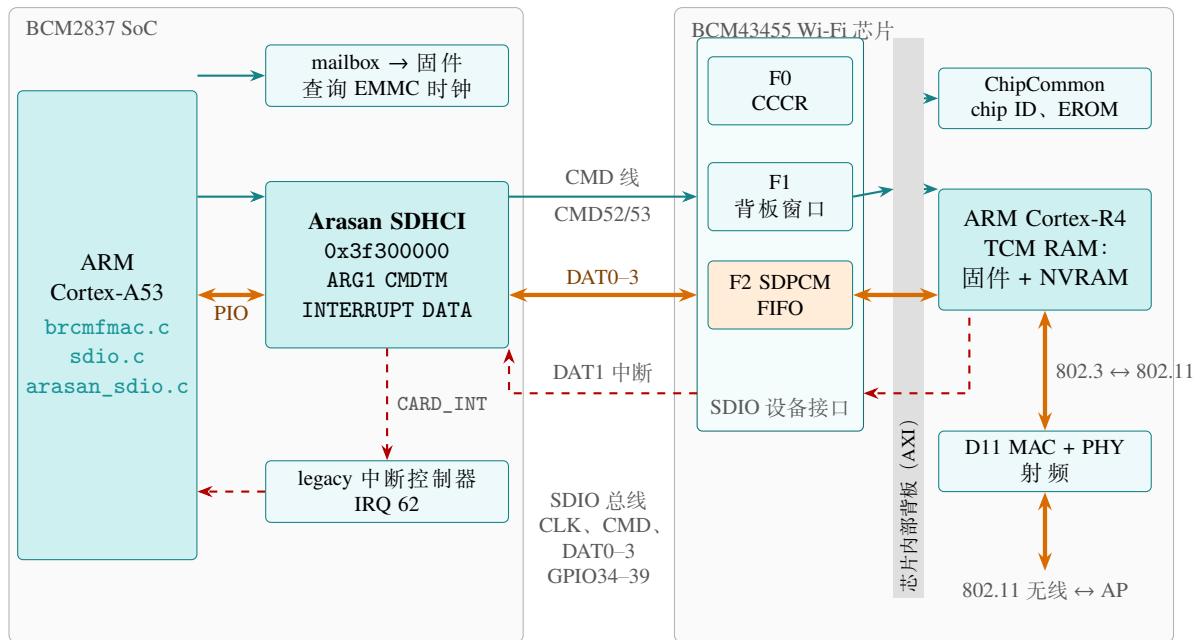


图 7.7: SDIO Wi-Fi 的硬件拓扑。细实线是控制面：CMD 线上的 CMD52/CMD53 经 F0 配置总线、经 F1 访问芯片内部背板（读 chip ID、下载固件、控制 CR4 复位）；粗线是数据面：CPU 以 PIO 方式经 Arasan 的 DATA 寄存器读写 F2 的 SDPCM FIFO，芯片固件再与 802.11 帧互转；虚线是事件：固件经 DAT1 发出卡中断，成为 BCM2837 的 IRQ 62

网帧、另外接受若干控制命令”的设备，802.11 的细节全藏在固件里。7.8 节的 MT7601U 恰好相反，是 SoftMAC：802.11 帧由主机驱动自己组装。

SDIO 总线。 两者之间是 SDIO 总线：一根时钟线 CLK、一根命令线 CMD、四根数据线 DAT0~3，在 3B+ 上走 GPIO34~39 的 ALT3。CMD52 只用命令线，命令里带地址和一个字节，响应里带回一个字节，用来读写寄存器；CMD53 在命令线上发出地址和长度，数据走四根 DAT 线，用来搬运固件和网络帧。DAT1 还有第二个用途：总线空闲时，卡可以把它拉低来请求中断，所以 Wi-Fi 芯片不需要单独的中断引脚。

一个 SDIO 设备，三个 function。 SDIO 卡在协议上分成若干 function，各有独立的 17 位寄存器地址空间：

- **F0** 是公共寄存器区 CCCR，设置总线宽度、使能其他 function、打开中断；
- **F1** 是进入芯片内部的“窗口”。BCM43455 内部的 ChipCommon、SDIO 核、R4 和 TCM RAM 挂在一条背板总线上，地址空间远大于 17 位，所以 F1 只开一个 32 KiB 的窗口：先把目标地址的高位写进窗口寄存器，再用低 15 位访问。读 chip ID、解析 EROM、让 R4 停机、下载固件和 NVRAM、写复位向量，全都经过这个窗口；
- **F2** 是固件启动之后才就绪的 FIFO，运行时所有控制命令、事件和网络帧都从这里进出。

Arasan 是主机控制器，而且工作在 PIO 模式。 Arasan SDHCI 负责产生 SDIO 时钟、把命令编码到 CMD 线上、按 4 位总线收发数据。驱动发一条命令的方式是写 ARG1（参数）和 CMDTM（命令号与传输方式），然后等 INTERRUPT 里的完成位。有数据阶段时，本项目没有使用控制器的 DMA，而是 CPU 等 READ_RDY/WRITE_RDY，再一个 32 位字一个 32 位字地读写 DATA 寄存器。这和摄像头形成鲜明对比：Unicam 的数据完全不经过 CPU，而这里每个字都要 CPU 亲手搬运。这样实现简单，也没有缓存一致性问题，代价是吞吐受 CPU 和单次传输长度限制——实机曾在 512、256、128 字节 CMD53 上出现数据 CRC 错误，驱动逐级减半，最后稳定在 64 字节。这个上限是运行时探测出来的，完成多块传输和时序调优之后应当重新评估。

中断要过三道闸门。 固件有帧要交给主机时，DAT1 上的卡中断要穿过三层开关才能变成 CPU 的 IRQ：芯片内部 SDIO 核的 HOSTINTMASK 决定哪些事件送到 DAT1；F0 的 CCCR 里的 INT_ENABLE 打开 master 位和 F1 位 (sdio_claim_irq()); Arasan 的 IRPT_EN 打开 CARD_INT，BCM2837 的 legacy 控制器再打开 IRQ 62。任何一层没开，接收都只能靠轮询。

固件从哪里来。 芯片每次上电都是空的，固件、NVRAM、CLM 三个文件放在 SD 卡的 FAT32 启动分区里，由另一个控制器 SDHOST (0x3f202000) 读出。两个 SD 控制器各管一条总线：SDHOST 接外置 SD 卡，Arasan 接板载 Wi-Fi，混用会造成 GPIO 和时钟的所有权冲突。

7.7.3 SDIO 枚举

Arasan 驱动 probe 后创建 mmc_host，SDIO core 依次发送 CMD0（复位）、CMD5（协商 I/O 电压，等待就绪）、CMD3（取 RCA）、CMD7（选中），再用 CMD52 读 CCCR、FBR 和 CIS，切换到 4 位总线，最后把三个 function 注册到 sdio 总线。brcmfmac 驱动按 vendor 0x02d0、device 0xa9a6/0xa9bf、function 1 匹配。三条 SDIO 命令的分工是：CMD5 识别和上电；CMD52 单字节读写寄存器；CMD53 批量传输。

7.7.4 固件装载

FullMAC 芯片每次上电都要由主机下载固件。驱动从 FAT32 启动分区读取三个文件（所以必须排在挂起 bootfs 的 rootdev_init() 之后）：

- BCM43455.BIN：芯片 CR4 的执行代码；
- BCM43455.TXT：板级 NVRAM 参数（天线、校准）；
- BCM43455.CLM：各国信道和功率的法规数据库。

Function 1 是“背板窗口”：设置窗口寄存器后，可以用 CMD52/CMD53 访问芯片内部的 ChipCommon、SDIO core、CR4 和片内 RAM。驱动读 chip ID、解析 EROM 找到各个核和 RAM，让 CR4 停机，用 CMD53 把固件和 NVRAM 写进 RAM，写入复位向量后放开 CR4，等待固件就绪的 mailbox 中断，最后才打开 Function 2。Function 2 是运行时的数据 FIFO。

7.7.5 SDPCM 与 BCDC

运行时的每一次 F2 传输都是分层封装的（图 7.8）。

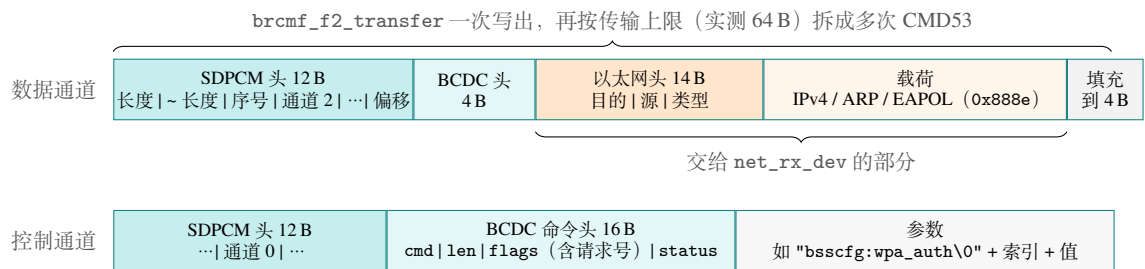


图 7.8: F2 上一帧的封装。SDPCM 是总线一层的帧格式，用通道号区分控制、事件和数据；控制通道里是 BCDC 命令 (ioctl/iovar)，数据通道里是 BCDC 数据头加一个完整的以太网帧

发送控制命令后，下一个到达的帧不一定就是响应——事件和数据帧可能插在前面。接收分发函数必须按通道分流，只有请求号相同的控制帧才是对应的响应。

7.7.6 WPA2 四次握手

这版固件不处理 WPA2 握手，所以主机自己完成四次握手 (`kernel/wpa_crypto.c`)：用 PBKDF2 从口令和 SSID 算出 PMK；收到 M1 后生成 SNonce，派生 PTK，计算 MIC，发 M2；收到 M3 后验证 MIC，用 AES key unwrap 解出组密钥 GTK；把成对密钥和组密钥装进芯片，发 M4。握手完成后才放行普通数据帧，随后执行 DHCP。

7.7.7 数据流与并发

图 7.9 把一帧数据在两个方向上经过的函数排成了层。两个方向的触发方式不同：

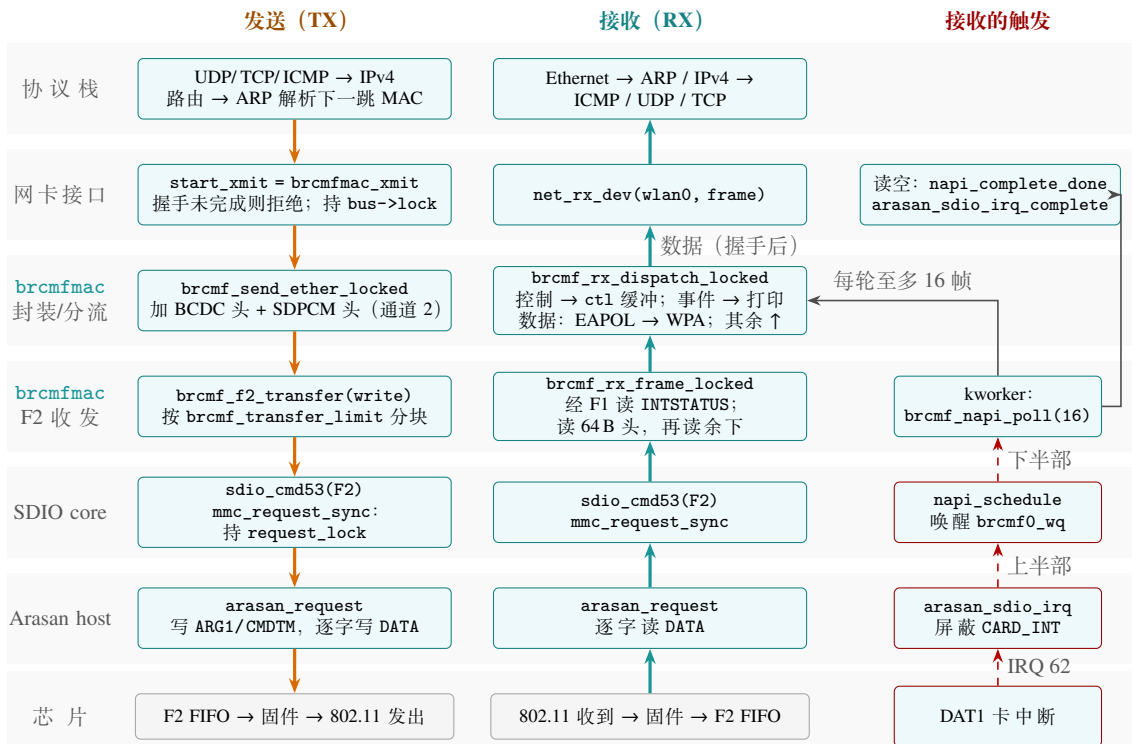


图 7.9: SDIO Wi-Fi 的数据流。发送在调用者的上下文里一路同步向下；接收由 DAT1 卡中断触发，上半部只屏蔽中断并调度 NAPI，真正的 CMD53 读取和分流在 `brcmf0_wq` 内核线程里完成，读空后才重新打开卡中断。两个方向都要经过 `mmc_request_sync()` 的 host 锁，且每个字都由 CPU 经 DATA 寄存器搬运

发送是同步的。 协议栈选好路由、解析出下一跳 MAC 以后，调用 `wlan0` 的发送函数 `brcmfmac_xmit()`。握手完成之前它直接拒绝普通数据。之后在调用者自己的上下文里（可能是系统调用，也可能是 TCP 重传 work）持 `bus->lock` 加上 BCDC 头和 SDPCM 头，经 `brcmf_f2_transfer()` 拆成若干次 CMD53 写进 F2，函数返回时数据已经交给了芯片。

接收是中断触发、线程执行的。 DAT1 卡中断到来时，上半部 `arasan_sdio_irq()` 只做三件事：在 Arasan 上屏蔽 `CARD_INT`、清掉状态位、调用 `napi_schedule()` 唤醒设备私有的 `brcmf0_wq` 内核线程。CMD53 要等总线、可能睡眠，绝不能在中断里做。下半部 `brcmf_napi_poll()` 在 `kworker` 里循环：经 F1 读 SDIO 核的 `INTSTATUS`，看到“有帧”就从 F2 读 64 字节的头、校验 SDPCM 长度和它的反码、再读剩余部分，然后按通道分流。每轮最多处理 16 帧；没读空就让 NAPI 重新排队，读空了才 `napi_complete_done()` 并重新打开 `CARD_INT`。卡中断是电平触发的，在芯片里的帧被取走之前一直有效，所以必须先屏蔽、读空再打开，否则 CPU 会被同一个中断反复打断。

同步的控制命令。 控制面是请求/应答式的: `brcmf_dcmd()` 发出一个带递增请求号的 BCDC 命令, 然后反复调用分流函数, 直到读到请求号相同的控制帧。事件和数据帧可能插在应答前面, 所以分流函数不能假定“下一帧就是回复”, 必须按通道放到各自的去处: 控制帧存进 `ctl` 缓冲, 事件帧打印出来, 数据帧中的 EAPOL 交给 WPA2 握手逻辑, 其余在握手完成后交给 `net_rx_dev()`。

两把锁, 两个粒度。 `bus->lock` 以设备为粒度, 保证同一块网卡的 SDPCM 序号、收发缓冲和连接状态不会被发送路径、NAPI 和连接 `watchdog` 同时改动。`host->request_lock` 以控制器为粒度, 保证 Arasan 那一套 ARG1/CMDTM/INTERRUPT/DATA 寄存器在一条 CMD52/CMD53 完成之前不被另一条覆盖。7.9.9 小节的 CMD52 failed status=df0001 就是缺了后一把锁造成的。两把都是睡眠锁或在进程上下文中持有, 因为一次 SDIO 事务可能要等上毫秒级的超时。

从轮询到中断的演进。 这条接收路径经历过三个版本: 最早 CPU0 在每次定时器中断里直接轮询 F2, 完整的接收和协议处理都跑在中断上下文里; 后来有了 DAT1 中断, 但仍等下一次定时器才去读; 现在中断直接调度 NAPI, 定时器只负责推进 delayed-work 的时钟 (连接 `watchdog`、TCP 重传)。早期每 100 ms 轮询一次时, SSH 输入一个字符要等 100 ~ 300 ms 才回显; 改成中断驱动后, 这个延迟基本消失。

7.8 USB 无线网卡: MT7601U

MT7601U 是一块常见的 USB 无线网卡 (VID 148f, PID 7601)。与 BCM43455 最大的不同在于它是 *SoftMAC*: 芯片只负责射频、基带和收发帧, 扫描、认证、关联、加密握手等 802.11 逻辑都要由主机驱动完成 (表 7.5)。

表 7.5: 两块无线网卡的对比

	BCM43430/43455	MT7601U
总线类型	SDIO (Arasan)	USB (DWC2 + LAN951x 集线器)
固件控制	FullMAC: 固件完成扫描、关联	SoftMAC: 主机构造 802.11 帧
数据网卡名	BIN + NVRAM + CLM, 经背板写入 RAM	MT7601U.BIN, 经 Bulk OUT 写入 MCU 的 ILM/DLM
	BCDC 命令	MCU 命令 (EP8 发、EP5 收) 和寄存器读写
	SDPCM 数据通道里的以太网帧	TXWI 描述符 + 802.11 帧, EP8 发、EP4 收
	wlan0	wlan1

驱动初始化的主要步骤是:

1. 解析固件头, 把 IVB、ILM、DLM 三段通过 Bulk OUT 写入 MCU, 等待 MT_MCU_COM_REG0 变成 1;
2. 读 eFuse 得到永久 MAC 地址、频偏、RSSI 和温度校准参数;
3. 初始化 MAC、BBP (数字基带)、RF (模拟射频) 三组寄存器表, 执行各项校准;
4. 扫描信道, 解析 beacon/probe response, 选择目标 BSSID;
5. 发送 Open System 认证帧和带 RSN IE 的关联请求;
6. 完成 WPA2 四次握手, 安装密钥, 注册 wlan1。

发送一个 IP 包要经过: 以太网帧 → LLC/SNAP 封装 → 802.11 数据帧头 → TXWI → USB DMA 头和填充 → EP8 Bulk OUT。接收端在 EP4 上长期挂着一个 Bulk IN 请求, DWC2 用 DMA 把数据写到缓冲区 (总线地址要加 0xC0000000 别名, 见第 4 章), 完成中断后由下半部解析。网络层的路由表决定一个包从 wlan0 还是 wlan1 发出, 两块网卡可以同时工作。

7.9 CSI-2 摄像头：OV5647

7.9.1 路线选择

树莓派官方的摄像头栈运行在 GPU 固件里 (`start_x.elf + MMAL`)，ARM 端只是发命令。本项目选择与 Linux 主线相同的“ARM 直接驱动”路线：内核自己通过 I2C 配置传感器，打开电源域和时钟，用 Unicam 接收 MIPI CSI-2 数据并 DMA 到内存。硬件是 Camera Module v1.3 (OmniVision OV5647)，接在 3B+ 的 CSI1 口。

7.9.2 硬件结构：I2C、CSI-2、Unicam 与 DMA

摄像头驱动之所以涉及这么多部件，是因为一帧图像从产生到进入内存要经过几条性质完全不同的通路 (图 7.10)。弄清楚每条通路传什么、由谁发起，后面的代码就很好读了。

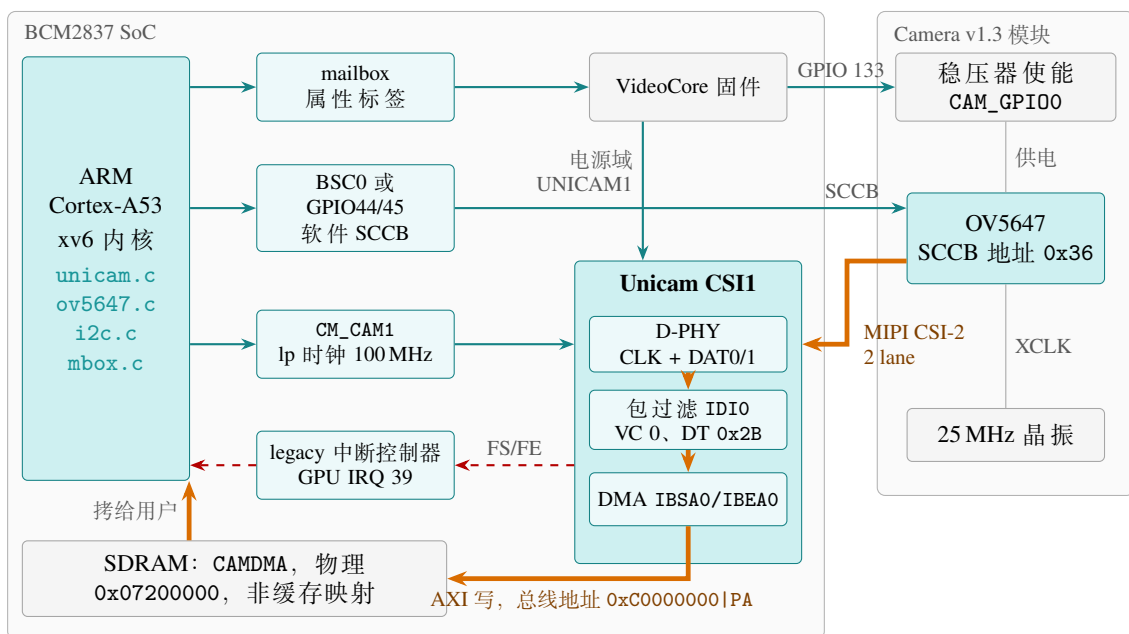


图 7.10: 摄像头的硬件拓扑。细实线是控制面：mailbox 上电和电源域、SCCB 写传感器寄存器、lp 时钟；粗线是数据面：CSI-2 → Unicam → DMA → 内存；虚线是事件：Unicam 的帧开始/结束中断，目前屏蔽，由 CPU 轮询 STA/ISTA

传感器有两个接口。 OV5647 对外有两个互不相干的接口。一个是低速的 *SCCB* 控制口 (OmniVision 对 I2C 的兼容实现，两根线 SDA/SCL，约 100 kHz)，CPU 通过它读芯片 ID、写一百来个配置寄存器：分辨率、binning、帧长、曝光、增益、MIPI 输出方式、测试图案，最后写“退出待机”让传感器开始出图。另一个是高速的 *MIPI CSI-2* 输出口：一对时钟线加两对数据线 (lane)，每对都是差分的 D-PHY 链路，空闲时处于低功耗的 LP-11 状态，传数据时切换到高速模式。图像只走 CSI-2，配置只走 SCCB——I2C 再慢也不影响帧率，它只决定“开机配置”要花多久，这也是驱动后来能退到软件 SCCB 的原因。

CSI-2 链路上传的是包。 传感器把每一帧编码成一串 CSI-2 包：帧开始 (FS) 和帧结束 (FE) 是不带数据的短包；中间每一行像素是一个长包，包头里有虚拟通道号 (VC) 和数据类型 (DT, RAW10 是 0x2B)，包尾有 CRC。接收方据此知道一帧何时开始、何时结束、每个包该放到哪里，CRC 和 ECC 错误也是在这一层被发现的 (STA 里的 CRCE、PBE 等位)。

Unicam 是接收器，也是 DMA 主设备。 SoC 一侧的 Unicam CSI1 把上面的事情全做了，它内部可以分成三段：

- **D-PHY 前端**：接收差分信号，完成低功耗态和高速态之间的切换。判断“稳定进入高速态”要等若干个 lp 时钟周期（CLT/DLT 里的 settle 和终端使能计数），所以需要时钟管理器 CM_CAM1 提供的 100 MHz lp 时钟——时钟频率不对，settle 时间就跟着错，表现为链路误码；
- **包处理**：解析包头、校验 CRC，FS/FE 短包变成 ISTA 里的帧开始/结束事件；长包经 IDIO 按 VC 和 DT 过滤，只有 VC 0、RAW10 的行数据才往下走。本项目不让 Unicam 解包（IPIPE=0），行数据保持 CSI-2 原样的 10 位打包格式；
- **DMA 写入**：Unicam 自己就是 AXI 总线上的主设备，把每行按 IBSL 的行跨度写到 IBSA0 ~ IBEA0 之间的内存，写到哪里由 IBWP 指示。这里的“DMA”不是 BCM2837 通用的 DMA 控制器，而是 Unicam 内置的写引擎；它看到的是 VideoCore 总线地址，所以内核写进去的是 0xC0000000 | PA（第 4 章“三种地址视图”）。

内存一侧的约束。Unicam 只认一段起止地址，所以缓冲区必须物理连续，不能用 kalloc() 逐页拼；它放在 PHYSTOP 以上的 CAMDMA 区，分配器永远碰不到。这段内存映射成非缓存的普通内存：DMA 写进 DRAM 的数据，CPU 直接读就是最新的，不需要在每帧前后作废缓存行。

电源和时钟由谁管。摄像头模块上的稳压器由 CAM_GPIO0 使能，这根线接在 VideoCore 管理的 GPIO 扩展器上，Unicam 所在的电源域也归固件管，ARM 只能通过 mailbox 发属性标签请固件代办。CM_CAM1 时钟则是普通的时钟管理器寄存器，内核直接写。传感器自己的像素时钟来自模块上的 25 MHz 晶振，不需要 SoC 提供。

CPU 只在两头出现。把这些放在一起看，CPU 的角色就清楚了：开始时它是配置者——经 mailbox 上电、写时钟、经 SCCB 配传感器、经 MMIO 配 Unicam；数据开始流动以后，传感器 → Unicam → 内存这条路完全不经过 CPU；CPU 只需要对 FS/FE 事件作出反应，在恰当的帧开始前换掉 DMA 地址（图 7.12），最后把整帧拷给用户。事件本可以用中断（GPU IRQ 39）通知，目前为了安全改成 CPU 每 1 ms 读一次 STA/ISTA。

顺序约束。几条通路之间有严格的先后关系，驱动的步骤顺序就是由它们决定的：先上电，SCCB 才可能应答；传感器先停在 LP-11，Unicam 才能在干净的空闲态上完成模拟复位和校准；接收器先使能、传感器后出流，否则第一个 FS 可能丢失；停止时反过来，先停传感器再停接收器，避免接收器在一帧中间被关掉。表 7.6 列出了这些部件在 Pi 3B+ 上的具体资源。

7.9.3 sensor_ops：仿照 V4L2 子设备

桥接驱动 unicam.c 不认识任何具体传感器，传感器驱动也不碰 Unicam。两者之间只有一层接口，对应 Linux 的 v4l2_subdev_ops：

```
struct sensor_ops {
    int (*power_on)(struct camsensor *s);           // 供电、读芯片 ID、进入 LP-11
    void (*power_off)(struct camsensor *s);
    int (*stream_on)(struct camsensor *s, const struct cam_mode *mode);
    int (*stream_off)(struct camsensor *s);
    int (*set_ctrl)(struct camsensor *s, int id, int value); // 如测试图案
};

struct camsensor {
    char *name; const struct sensor_ops *ops;
    const struct cam_mode *modes; int nmodes; // 宽、高、行字节数、CSI-2 数据类型
    struct camsensor_bus bus; // lane 数、是否非连续时钟、虚拟通道
    int warmup_frames; // 开流后丢弃的帧数
};
```

启动时 `ov5647_init()` 调用 `camsensor_register()`, `camera_init()` 按注册顺序逐个调用 `power_on`, 第一个读到正确芯片 ID 的传感器被绑定到 `/dev/video0`。要支持 Camera v2 (IMX219), 只需新写一个实现了 `sensor_ops` 的文件。

7.9.4 板级资源

表 7.6 列出了从 Raspberry Pi 设备树和 Linux 驱动中查到的资源。

表 7.6: 摄像头相关的板级资源 (Pi 3B+)

项目	值
传感器 I2C	BSC0 走 GPIO44/45 (ALT1), 从机地址 0x36, 芯片 ID 0x5647
传感器供电	固件扩展 GPIO 5, mailbox 编号 $128 + 5 = 133$
传感器时钟	摄像头板上自带 25 MHz 晶振
接收器	Unicam CSI1, 寄存器 0x7e801000, lane 时钟门控 0x7e802004
中断	GPU IRQ 39
电源域	UNICAM1: 设备树编号 13, 发给固件时要加 1, 即 14
lp 时钟	时钟管理器 CM_CAM1, 100 MHz
VPU 时钟	Unicam 要求至少 250 MHz, <code>config.txt</code> 中 <code>core_freq=250</code>
CSI-2 链路	2 条数据 lane, 非连续时钟, RAW10 (数据类型 0x2B)

供电和电源域都要通过 VideoCore 固件的 property mailbox: 内核在一块 16 字节对齐的缓冲区里写好“标签”(如 0x00038041 设置扩展 GPIO、0x00038030 设置电源域), 把地址写入 mailbox 寄存器, 等固件把结果写回同一块缓冲区。项目把原来分散在 SD、USB 驱动里的私有 mailbox 代码收拢成了带锁的 `kernel/mbox.c`。电源域编号是一个容易出错的地方: 设备树里的 UNICAM1 是 13, 发给固件时要加 1。

7.9.5 上电与探测

启动时 `main()` 先调用 `ov5647_init()` 登记传感器驱动 (此时不碰硬件), 再由 `camera_init()` 注册字符设备 CAMERA (主设备号 5, `/dev/video0`), 然后对每个登记过的传感器先检查其默认模式 Unicam 能否接收, 再调用 `power_on`。OV5647 的做法是: 经 mailbox 把固件扩展 GPIO 5 先拉低 20 ms、再置 1, 等 100 ms, 把 GPIO44/45 交给传感器的控制总线, 读寄存器 0x300a/0x300b, 应得 0x56 0x47。下面几段讲这条控制总线是怎样从硬件 I2C 一步步退到 25 kHz 软件 SCCB 的。成功时打印:

```
camera: OV5647 on CSI1 (2 lanes) -> /dev/video0, 640x480 RAW10
```

mailbox. 供电走的是固件 property mailbox。电源控制现在与 Linux `gpio-raspberrypi-exp.c` 一致: 先用 `GET_GPIO_CONFIG` 读出并保留 `CAM_GPIO0` 的极性, 再设置方向和电平, 最后用 `GET_GPIO_STATE` 回读, 标准 Pi 3 应显示 `direction=output polarity=normal state=1`。消息本身按 Linux `rpi_firmware_property()` 的约定组织: 标签的请求/应答长度字填 0; 发送前清空 FIFO 里其他驱动超时后遗留的旧回复 (日志里的 `stale replies K`); 只接受地址与本次请求相同的回复; 判断成败时不看单个标签的 `response` 位, 而看整条消息成功后返回结构的第一个字。这些规则是怎么来的, 见 7.9.9 小节的问题 1。

控制总线: 从 BSC0 到软件 SCCB. 读 16 位寄存器地址需要“写地址、不发 STOP、repeated START、读数据”的组合事务, `i2c_write_read()` 照 Linux `i2c-bcm2835.c` 实现了它 (BSC 的写 FIFO 不能预填, 要先启动空写事务、等 TXW 再填)。但在真机上, BSC0 对 0x36 始终 NACK, 而 GPIO bit-bang 探测能得到 ACK, 降速也无济于事。最终的驱动 (`gpio-sccb-v8`) 因此从第一次访问起就用 GPIO44/45 上约 25 kHz 的软件 SCCB: STOP/START 分离读法, 拒绝任何不是 0x5647 的读数, 失败就断电重来, 不再扫描其他地址。BSC0 仍按 100 kHz 初始化并保留两段式中止, 只是不再发事务。这条路是怎么一步步走过来的, 见 7.9.9 小节的问题 2~6。

延迟探测。 /dev/video0 在探测之前就已注册，所以 open 成功不代表绑定了传感器。启动时只探测一次，避免没有摄像头时拖慢开机；如果启动探测偶然失败，第一次 read() 或 pattern=N 写入会在拍照睡眠锁 cam.busy 下重新探测，最多三个完整的断电/上电周期，每次失败等 100 ms。

7.9.6 拍一帧

用户每 read() 一次 /dev/video0 就拍一帧，返回 40 字节的帧头 (magic、宽、高、行字节数、格式、长度、序号、时间戳) 加打包的 RAW10 数据。帧头里的 format 同时给出 Bayer 顺序 (OV5647 是 BGGR, IMX219 是 RGGB)，用户程序据此去马赛克。cam_capture() 的顺序与 Linux 相同——开始时先启动接收器再让传感器出流，结束时先停传感器再停接收器；整条调用链见图 7.11：

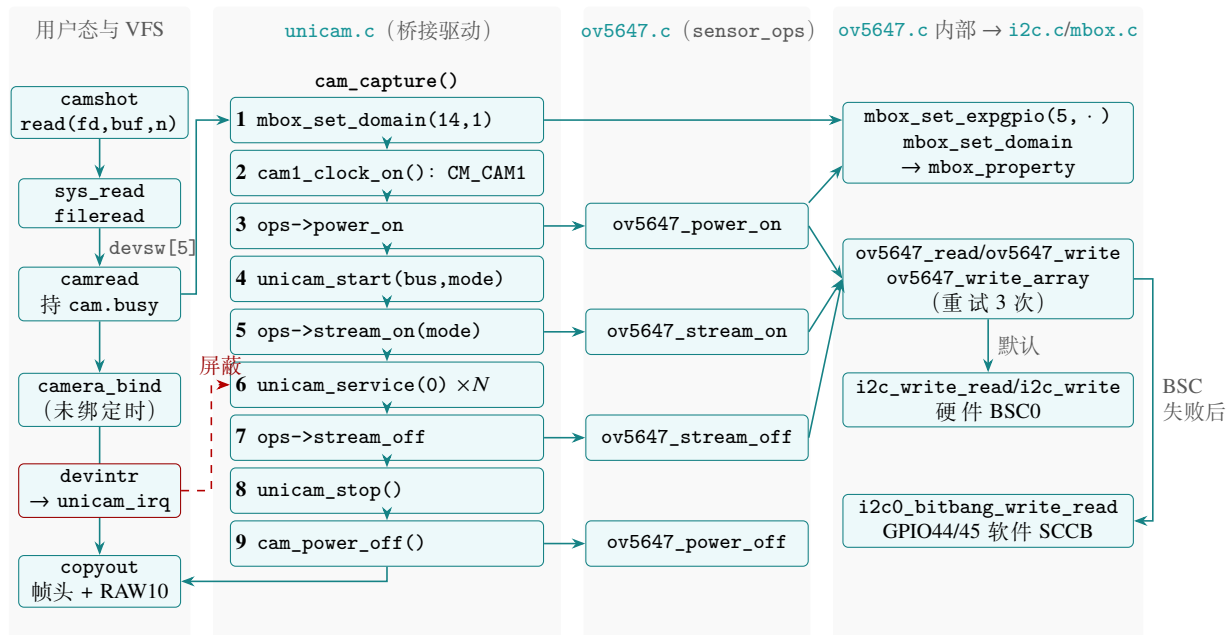


图 7.11: 一次 read("/dev/video0") 的调用关系。unicam.c 只通过 sensor_ops 调用传感器驱动；传感器驱动再经 mailbox 上电、经 BSC0 或软件 SCCB 写寄存器。第 6 步每 1 ms 轮询一次 Unicam 状态，直到 DONE 或 4 s 超时，是目前采集帧的唯一路径，中断入口已屏蔽；第 9 步的 ov5647_power_off 同样经 mailbox 断电

1. mbox_set_domain(14, on) 打开 UNICAM1 电源域；
2. CM_CAM1 时钟：PLLD_PER (500 MHz) ÷ 5 = 100 MHz；若 BUSY 位不置起，就退回 19.2 MHz 晶振并打印提示；
3. power_on: 上电、校验 ID、进入 LP-11；
4. 按传感器的总线参数配置 Unicam (照 Linux unicam_start_rx()): lane 数、虚拟通道、数据类型、行字节数 (16 的倍数)，DMA 先指向一块大小为 0 的“假缓冲区”；
5. stream_on: 依次写公共寄存器表、640×480 模式表、HTS/VTS/曝光/增益、缓存的测试图案，退出软件待机，开始发送；
6. 帧状态机：WARMUP 丢弃前 30 帧 (约 0.5 s) 让传感器自带的 AEC/AGC 收敛；ARMED 在下一个帧开始时把 DMA 地址换成真缓冲区；CAPTURE 一开始就写回假缓冲区；帧结束后进入 DONE；
7. stream_off → 停 Unicam → power_off → 关时钟和电源域，把数据从 CAMDMA (非缓存映射，物理 0x07200000) 拷给用户。

第 6 步的关键 (图 7.12) 是：写进 IBSA0/IBEA0 的新地址要到下一个帧开始才生效。所以真缓冲那一帧刚开始就把假缓冲写回去，这一帧会继续完整地写进真缓冲，之后的帧则落进假缓冲，缓冲区里恰好留下一整帧。假缓冲的大小编程为 0，是照搬 Linux 绕开循环缓冲模式越界写缺陷的做法。

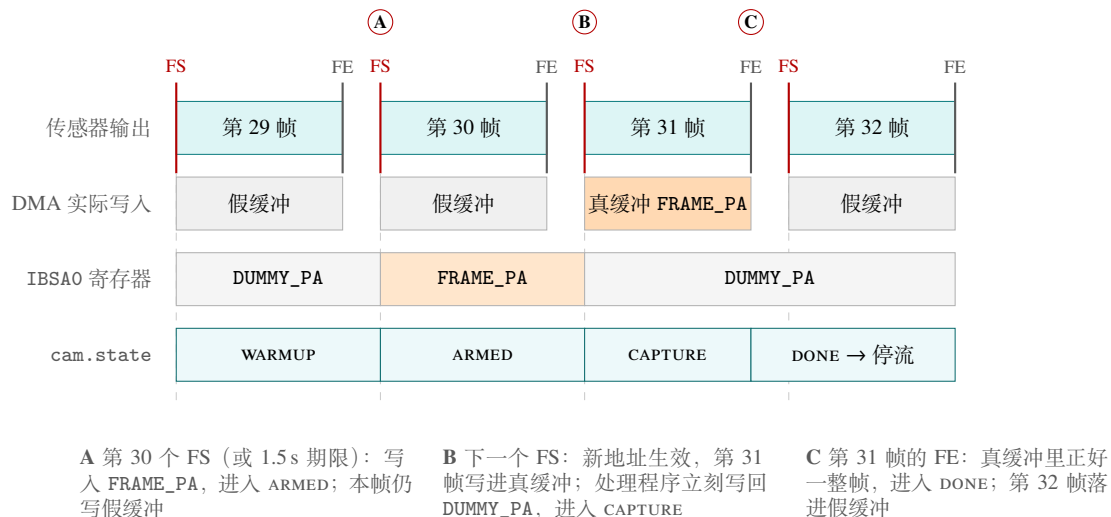


图 7.12: 帧状态机的时序。IBSA0/IBEA0 的新值要到下一个帧开始 (FS) 才被硬件锁存, 所以“写真地址”和“写回假地址”都比它们生效的那一帧早一个 FS

为什么用轮询而不是中断. 设备树说 CSI1 是 GPU IRQ 39, 但这个编号没有在真机上验证过; 第一次打开它时 CPU0 陷入了清不掉的电平中断 (7.9.9 小节问题 7)。所以目前屏蔽 IRQ 39, 用 CNTVCT_ELO 计一个 4 秒期限、每 1 ms 轮询一次 Unicam 状态。现在真机上的日志是:

```
unicam: frame captured by polling; CSI1 IRQ 39 intentionally masked
```

它表示 sensor → CSI-2 → Unicam → DMA 的整条路径已经打通, 只是 IRQ 路由还留待单独验证。

7.9.7 用户程序 camshot

camshot 用纯整数运算完成图像处理 (用户程序以 `-mcpu=cortex-a53+nofp` 编译): 解包 RAW10 (每 4 个像素占 5 字节), 减黑电平 16, BGGR 双线性去马赛克, 灰度世界白平衡 (增益限制在 0.5 ~ 4), 把绿色通道的第 99 百分位拉到满幅, 查表做 gamma 2.2 校正, 写成 24 位 BMP。-t N 让传感器输出内部测试图案 (1 彩条、2 色块、3 随机), 这时改用固定的 1.0 白平衡和固定电平, 不做统计——测试图案本身是校准过的, 正好单独验证传感器、CSI-2、DMA、转换和写盘; -s 输出 320×240 半尺寸。

640×480 的 BMP 有 900 KB, 超过原生 xv6 文件系统单文件 268 KB 的上限 (MAXFILE= 12+256 块), 所以默认写到以读写方式挂载的 FAT32 启动分区 `/boot/camera.bmp`, 关机后把 SD 卡插到电脑上就能看到。为了让慢速写盘不被误判成“卡死”, camshot 在每个阶段打印一行 (收到帧、解包、统计、打开输出、写头、转换), BMP 按 16 行一批写入 (半尺寸图从 240 次 FAT32 写降到 15 次), 启动时打印的版本串 `converter fixed-pattern-v2 fat-progress batch16` 用来确认 SD 卡上的程序确实已经更新。

7.9.8 一次成功拍照的日志解读

下面是 v8 驱动在真机上一次成功拍照的完整串口输出 (`camshot -t 1 -s -o /boot/cam02.bmp`: 传感器彩条、半尺寸、写到启动分区)。左边的 (1) ~ (6) 是为了讲解加的阶段编号, 与下文 ① ~ ⑥ 一一对应:

```
(1) camera: driver gpio-sccb-v8 unicam-poll-v2
ov5647: CAM_GPI00 direction=output polarity=normal state=1
i2c0: GPIO44/45 ALT1, 100000 Hz (core 2500000000), BCM2837 two-stage abort
ov5647: probing with 25 kHz GPIO SCCB; BSC0 bypassed
ov5647: software SCCB power-on chip-id=5647
ov5647: sensor outputs enabled
```

```

ov5647: power-on complete, lanes in LP-11
camera: OV5647 on CSI1 (2 lanes) -> /dev/video0, 640x480 RAW10
$ /bin/camshot -t 1 -s -o /boot/cam02.bmp
(2) camshot: converter fixed-pattern-v2 fat-progress batch16
camshot: capturing...
ov5647: CAM_GPIO0 direction=output polarity=normal state=1
... (与 (1) 相同的上电五行)
ov5647: power-on complete, lanes in LP-11
(3) unicam: sensor powered, receiver setup begins
unicam: receiver enabled, starting sensor stream
ov5647: programming common registers (51)
ov5647: programming mode registers (29)
ov5647: programming controls (11)
ov5647: streaming started
unicam: sensor streaming, waiting for frame
(4) unicam: frame captured by polling; CSI1 IRQ 39 intentionally masked
(5) camshot: frame received 640x480, 384040 bytes
camshot: unpacking RAW10...
camshot: sensor test pattern; fixed WB/level
(6) camshot: preparing output /bovfs: boot create begin path=/boot/cam02.bmp sub=/cam02.bmp mode=601
ot/cam02.bmp (230454 bytes)...
vfs: boot create stat complete exists=0
vfs: boot create directory-entry begin
fat32: create lookup begin path=/cam02.bmp
fat32: create lookup complete exists=0
fat32: create reserve begin slots=2
fat32: create reserve complete
vfs: boot create directory-entry complete
vfs: boot create truncate begin / complete
vfs: boot create verify-stat begin / complete
vfs: boot create openwrite begin / complete
camshot: output opened; writing BMP header...
camshot: converting and writing /boot/cam02.bmp (230454 bytes)...
camshot: 320x240 frame 31, WB R x1.0 B x1.0, level x1.0 -> /boot/cam02.bmp

```

① **启动时的探测与绑定。** 这是 `main()` → `camera_init()` → `camera_bind()` → `ov5647_power_on()` 的输出。第一行是驱动版本串，用来确认 SD 卡上跑的确是新内核。第二行来自 mailbox 的回读：CAM_GPIO0 是输出、极性正常、电平为 1，模块的稳压器已经打开。第三行由 `i2c0_init_camera_pins()` 打印：BSC0 仍按 100 kHz 初始化（分频 $250\text{ MHz}/100\text{ kHz} = 2500$ ），出错时采用“两段式中止”——先保持 I2CEN 清 FIFO 等 TA 归零，再关控制器（7.9.5 小节）。但紧接着的一行说明这次根本不用它：v8 直接用 25 kHz 软件 SCCB。读到 5647 之后，驱动写一小段寄存器打开传感器的输出引脚，再写停流序列让 MIPI 链路停在 LP-11 空闲态——Unicam 稍后要在 这个干净的空闲态上做模拟复位和校准。最后一行表示绑定成功；绑定之后 `camera_bind()` 立即 `power_off()`，模块在两次拍照之间是断电的。

② **每次拍照都重新上电。** 用户 `read()` 进入 `camread()`，持 `cam.busy` 调 `cam_capture()`，后者再次 `ops->power_on()`，所以上电五行原样再出现一次。这是“每次拍照完整上电、预热、断电”这个设计的直接体现，也是一次拍照要一秒左右的主要原因。

③ **先接收器、后出流。** unicam: receiver enabled 在 starting sensor stream 之前，顺序与 Linux 一致。随后 ov5647_stream_on() 写三张表：公共寄存器 51 项、640×480 模式 29 项、曝光增益等控制 11 项，再加上虚拟通道、测试图案、退出待机、MIPI 控制等几次读改写。它们全部经 25 kHz 软件 SCCB：一次 16 位地址的写是 4 个字节、约 36 个时钟，约 1.5 ms，九十多次写加起来约 0.15 秒——慢，但只在开流时发生一次。

④ **一帧到手。** cam_capture() 每 1 ms 调一次 unicam_service(0)，状态机经过 WARMUP（丢 30 帧）、ARMED、CAPTURE，在帧结束时进入 DONE（图 7.12）。“intentionally masked”说明 IRQ 39 是有意屏蔽的，不是丢了中断。

⑤ **数字对得上。** read() 返回 384040 字节：40 字节帧头加 $640 \times 480 \times 10/8 = 384000$ 字节 RAW10。测试图案模式下 camshot 用固定的 1.0 白平衡和电平，所以下一行直接是 fixed WB/level。

⑥ **创建输出文件。** 这一段同时有用户态和内核两个来源的输出，所以第一行被“切开”了：camshot 的 printf 在用户态逐字符 write(1, &c, 1)，打到 "/bo" 时进程陷入 open()，VFS 的内核 printf 插了一整行进来，返回用户态后才接着打出 "ot/cam02.bmp (230454 bytes)..."。传给内核的路径并没有坏，path=/boot/cam02.bmp 写得清清楚楚。之后各行对应 vfsopen() 的创建流程：

- mode=601: 0x001|0x200|0x400，即 O_WRONLY|O_CREATE|O_TRUNC；
- stat complete exists=0: fat32statpath() 正常返回，文件还不存在；
- fat32: create lookup: fat32createfile() 内部再查一次，确认不重名；
- reserve begin slots=2: cam02.bmp 本身就是合法的 8.3 名字，不需要 LFN，所以只要 2 个目录项——一个短名项，加一个全 0 的结束标记；根目录尾部空位够用，不必扩展目录簇；
- truncate: 对新建的空文件截断，首簇为 0，什么都不用释放；
- verify-stat、openwrite: 重新确认目录项，并把首簇、大小装进 vnode 的 fat32_file，供之后的追加写使用。

最后一行的 230454 是 54 字节 BMP 头加 $320 \times 240 \times 3 = 230400$ 字节像素；frame 31 是帧头里的序号，即帧结束计数——30 帧预热之后的第 31 帧。输出是 320×240，是因为 -s 把每个 2×2 Bayer 块合成一个像素，不是 Unicam 丢了一半分辨率。

这份日志证明了什么。 对照 7.9.2 小节的四条通路：mailbox 供电（CAM_GPIO0 回读）、SCCB 控制（读到 5647、三张表写完）、CSI-2 数据面与 DMA（frame captured、长度正确）、事件（轮询完成，IRQ 有意屏蔽）全部打通；用户态转换和 FAT32 写盘也都完成。它同时也是 7.9.9 小节问题 5、6（软件 SCCB 读 ID 不稳）和问题 11（新建文件停在 directory-entry begin）修好之后的验证记录。

7.9.9 真机调试记录

摄像头从“QEMU 上能探测到没有摄像头”到“真机写出第一张 BMP”，经历了下面十一个问题。前七个属于摄像头本身的控制面和数据面；后四个是摄像头作为第一个同时用到大块 DMA、长时间用户态计算、FAT32 大文件写和 Wi-Fi 并发的程序，逼出来的系统问题。每个问题都按同样的顺序记录：先是串口上看到的日志，再是分析，最后是解决方案。日志中凡是标注“格式”的，是按驱动源码里的 printf 格式写出的形式，其余是真机原样输出。

问题 1: mailbox 打开摄像头电源失败

日志输出。

```
ov5647: firmware GPIO 133 (camera power) failed, firmware rev N          (格式)
ov5647: SET_GPIO_CONFIG: <结果> (status .., tag response .. = done D len L,
first word W, stale replies K)
```

```
ov5647: SET_GPIO_STATE: <结果> (...)
ov5647: no reply at i2c0 0x36 (GPIO44/45), ...
```

问题分析. 传感器的稳压器由固件扩展 GPIO 133 (CAM_GPIO0) 使能, ARM 只能请 VideoCore 固件代办; 固件说“不支持”, 传感器就没电, I2C 上自然读不到 0x36。第一行的结果字段可以这样判读: no reply 是固件根本没回; message rejected 是整条消息被拒; done 1 len 0 是标签被应答但没有数据 (QEMU 就是这样); len 8 且 first word 133 是固件认识这个标签、但认为 GPIO 133 无效。对照 Linux 的 rpi_firmware_property() 和 gpio-raspberrypi-exp.c, 旧的 mailbox 代码有三处不同: 标签的请求长度字填了数据长度, Linux 填 0; 发送前不清 FIFO, 而 SD、USB 驱动各有一份私有 mailbox 代码, 它们超时后留下的回复会被当成本次的回复读走; 收到回复不核对地址。另外, 旧代码在复制数据之前就因为单个标签的 response 位设置而判失败, Linux 并不这样判。最后一列 stale replies 就是为验证第二点加的: 它统计发送前清掉了多少条别人的回复。

解决方案. 把各处私有代码收拢成带锁的 kernel/mbox.c, 按 Linux 的约定重写: 请求长度字填 0; 发送前清空 FIFO; 只接受地址与本次请求相同的回复; 以“整条消息成功”加“返回结构第一个字为 0”判断 GPIO 操作成败。电源控制也按 Linux 的做法: 先 GET_GPIO_CONFIG 读出并保留极性, 再设置, 最后 GET_GPIO_STATE 回读。修复后的日志是 ov5647: CAM_GPIO0 direction=output polarity=normal state=1。如果某个固件确实不处理这个标签, 兜底方案是自制 dt-blob.bin, 让固件启动时就把 p133 置为输出高。

问题 2: BSC0 对 0x36 始终 NACK

日志输出.

```
ov5647: no reply at i2c0 0x36 (GPIO44/45), BSC raw=90000131 flags=131 \
      now=... div=2500 fsel=55 levels=3
(更早的内核把 raw 打成 -6ffffecf)
```

问题分析. 供电已经正常, BSC0 却在 repeated-START 和 STOP/START 两种读法下都报错。逐个字段看: flags 的 0x100 是 ERR (从机没应答), 0x200 才是时钟拉伸超时; div=2500 是 250 MHz/100 kHz; fsel=55 表示 GPIO44/45 都是 ALT1; levels=3 表示 SDA、SCL 都是高电平, 总线空闲、没有被谁拉住。控制器确实发出了事务, 从机确实没应答——但仅凭这一行, 还分不清是“传感器不在”还是“控制器这条路不通”。另外两个副产品: 真 BCM2837 在 TA (transfer active) 仍为 1 时撤掉 I2CEN, 状态机可能冻结, QEMU 不模拟这一点; 内核 printf 的 %x 把最高位为 1 的值当有符号数打印, 0x90000131 成了 -6ffffecf。

解决方案. 出错时采用两段式中止: 先保持 I2CEN、清 FIFO, 等 NACK+STOP 让 TA 归零, 再关控制器、清粘滞状态位; %x 改为无符号。为了回答“传感器在不在”, 增加一个与 BSC 完全无关的 bit-bang 探测 (问题 3)。

问题 3: bit-bang 探测时而 ACK、时而 NACK

日志输出.

```
camera: GPIO bitbang probe 0x36=ACK 0x10=NACK 0x1a=NACK      (某次启动, 格式)
camera: GPIO bitbang probe 0x36=NACK 0x10=NACK 0x1a=NACK      (同一模块, 另一次)
```

问题分析. bit-bang 探测把 GPIO44/45 临时切成开漏 GPIO, 只发一个地址字节、读第九个时钟上的 ACK, 不读写任何寄存器。它本该给出确定的答案, 同一个模块的结果却不稳定, 问题出在诊断代码自己: 第九个时钟调用的是通用的 bb_release(), 它释放 SDA 之后会等 SDA 回到高电平——这对数据位是对的, 对 ACK 却正好相反, ACK 的定义就是从设备把 SDA 拉低。于是每次都白等 100 μs 超时, 再去采样时从设备可能已经松开了 SDA。

解决方案. ACK 周期只把 SDA 切成输入，在 SCL 高电平的中间直接采样。修正以后结果稳定为 0x36=ACK。这个结论很有分量：bit-bang ACK 而 BSC NACK，说明传感器、地址、供电、上拉和排线都没问题，故障只在 BSC0 的复用或时序这一段。同时规定无论 bit-bang 结果如何，都继续执行问题 4 的恢复扫描。

问题 4：BSC 四档降速全部失败

日志输出.

```
ov5647: BSC retry 100000 Hz failed status=... now=... div=...    (格式)
ov5647: BSC retry 50000 Hz failed status=...
ov5647: BSC retry 25000 Hz failed status=...
ov5647: BSC retry 10000 Hz failed status=...
```

问题分析. 既然传感器在，就怀疑速率或寄存器写入顺序：依次用 100、50、25、10 kHz 重读芯片 ID，每档都按 Linux 的比例同时更新 BSC_DIV 和边沿采样延迟 BSC_DEL（不继承固件留下的值），并在写完地址、长度、FIFO 后加 dsb sy，保证配置在 START 之前到达控制器。四档全部是 ERR，速率和 MMIO 写序都被排除了；剩下的只能是 BSC0 到 GPIO44/45 这条路由本身，或者固件对 BSC0 的占用。继续在这里花时间，收益已经很低。

解决方案. 绕开 BSC0：用 GPIO 完成完整的软件 SCCB 事务（写寄存器地址、STOP、START、读一个字节），读到 0x5647 后打印 using GPIO software SCCB; BSC0 bypassed，此后传感器的所有读写都走这条路。寄存器表里 0x0103=1 会让传感器软件复位、期间不应答，写完这一项等 10 ms；普通寄存器写最多重试三次。软件 SCCB 慢，但只用来写一百来个寄存器，图像走 CSI-2，不受影响。

问题 5：软件 SCCB 偶发读出 0x3631，随后 BUS-STUCK

日志输出.

```
ov5647: software SCCB chip-id=3631 attempt=1                    (格式)
...
camera: full GPIO I2C scan bus stuck at 0x..
```

问题分析. 地址被 ACK 了，读回的却是 0x3631——它不是任何传感器的 ID，事务层面成功、数据位却采错了，说明时序处于边缘。旧代码随后的两个动作让情况更糟：每次软件事务前无条件送 9 个 SCL 恢复脉冲；读到错误 ID 后立即扫描整个 7 位地址空间。对一个状态已经不稳的模块，这些都是额外的扰动，扫描最终把总线卡住 (BUS-STUCK)。

解决方案. (gpio-sccb-v7) 软件时钟降到约 25 kHz（半周期约 20 μs）；任何不是 0x5647 的完整读数都被拒绝重读（日志 rejected unstable chip-id=..）；恢复脉冲只在发现 SDA 被拉低时才送，最多 9 个，然后统一发 STOP；只要 0x36 已经完成过寄存器事务，就不再做全地址扫描，而是打印 address 0x36 responds but chip-id is unstable; power-cycle retry required 后直接断电重来；上电前 CAM_GPIO0 先拉低 20 ms 让模块放电，置高后等 100 ms 再访问。

问题 6：0x36 ACK，但三次读 ID 都失败

日志输出.

```
ov5647: software SCCB ID read retry 1/3 (probe=ACK)            (格式)
ov5647: software SCCB ID read retry 2/3 (probe=ACK)
ov5647: software SCCB ID read retry 3/3 (probe=ACK)
ov5647: address 0x36 ACK but chip-id read failed; power-cycle retry required (fsel=.. levels=..)
```

问题分析. v7 在真机上仍会出现这种情况。看调用顺序就明白了：每次上电，驱动仍然先用 BSC0 读 ID，失败几次之后才切到 GPIO。问题 4 已经证明这些 BSC 事务注定失败，而它们留下的半截事务会让传感器的 SCCB 接口进入不确定状态，紧接着的软件读取就跟着失败。

解决方案. (gpio-sccb-v8) 从第一次读 0x300a/0x300b 起就直接用 25 kHz 软件 SCCB，BSC0 只初始化、不再发事务，日志变为 probing with 25 kHz GPIO SCCB; BSC0 bypassed 和 software SCCB power-on chip-id=5647。如果 0x36 已 ACK 但读寄存器仍失败，本轮立即断电，交给 camera_bind() 做下一次完整的电源周期（用户首次 read() 时最多三次，每次间隔 100ms），不再探测其他地址。7.9.8 小节的日志就是 v8 的样子。

问题 7：开始出流后整机无响应

日志输出.

```
unicam: receiver enabled, starting sensor stream
ov5647: streaming started
unicam: sensor streaming, waiting for frame
（此后再无任何输出，4 秒超时也没有触发）
```

问题分析. 第一次真机出流时打开了设备树给出的 CSI1 中断 GPU IRQ 39。“4 秒超时也没触发”是关键线索：超时是按 xv6 的 ticks 计的，而 ticks 由 CPU0 的通用定时器中断推进；如果它不走了，说明 CPU0 一直在处理别的中断。IRQ 39 是电平触发的，若这个编号实际不对应 Unicam，或者处理程序清不掉它的源，CPU0 一从中断返回就立刻再次进入，定时器中断永远得不到服务——超时机制本身依赖于它要监督的那条路径，所以失效了。

解决方案. bring-up 阶段屏蔽 IRQ 39，改用与中断无关的 CNTVCT_ELO 计 4 秒期限，每 1 ms 轮询一次 Unicam 的 STA/ISTA；无论成败都能返回，失败时打印 FS/FE 计数、STA、IBWP。成功时的日志是 unicom: frame captured by polling; CSI1 IRQ 39 intentionally masked——“intentionally”说明中断是有意屏蔽的，不是丢了。以后再验证中断时，也只在拍照期间打开，连续 100 次没有发现 Unicam 状态就自动关掉，并始终保留这个有界轮询作兜底。

问题 8：转换期间偶发 SDIO CMD52 failed

日志输出.

```
camshot: calculating white balance and level...
arasan-sdio: request CMD52 failed status=df0001 irq=100 c1=e0807
```

问题分析. 两行挨在一起，很容易得出“用户程序抢占了 Wi-Fi 工作线程”的结论，但这站不住：camshot 和 brcmf0_wq 都是 SCHED_OTHER、nice 0，用户态并不自动低于内核线程，SDIO 中断上半部也随时能打断用户程序。真正的线索在数值里：status 的 bit 0 是 CMD_INHIBIT，irq=100 只有卡中断、没有本次的 CMD_DONE——上一个事务还没结束，另一个就开始写寄存器了。原来 sdio_cmd52()/sdio_cmd53() 直接调用 Arasan 的 request()，而 Wi-Fi 的状态 work、NAPI 收发和 BCDC 控制请求可能在不同核上同时运行，共用同一组 ARG1/CMDTM/INTERRUPT/DATA 寄存器。

解决方案. 给 struct mmc_host 加睡眠锁 request_lock，所有 CMD52/CMD53 都经 mmc_request_sync() 串行化。锁的粒度是物理控制器而不是某个 function 或某块网卡，对应 Linux MMC core 的 host claim；用睡眠锁是因为一次请求可能要等命令、数据和超时（7.7.7 小节）。启动枚举时还没有进程，这时只有启动 CPU 发请求，直接执行、不取锁。此前为 camshot 加的 sched_yield() 撤回——它掩盖不了并发访问。

问题 9: 执行不访存的指令却报数据中止

日志输出.

```

usertrap(): unexpected esr=0x92000004 ec=0x24
            elr=0x0000000000000b40 far=0xf3910003fda9a57b

```

问题分析. 更新 /bin/camshot 后, 程序偶尔在第一行输出之前就退出。EC=0x24 是来自 EL0 的数据中止, 可用当前 ELF 反汇编, ELR=0xb40 是 strncpy() 里一条纯寄存器的 sub, 不可能访存; FAR 又是一个随机指针。CPU 实际执行的并不是磁盘上这个 ELF 的指令。exec() 通过内核高地址把新程序写进物理页, uvmsync_icache() 做了 dc cvau, 最后却用 ic iallu——它只作废执行它的那个核的指令缓存。进程在一个核上装好, 迁移到另一个核运行, 那里的 I-cache 仍命中同一低地址上旧程序的指令: 执行的是旧代码, 用的却是新程序的数据和栈。

解决方案. 改为广播到 Inner Shareable 域的 ic ialluis, 前面的逐行 dc cvau 和 dsb ish 保留 (第 4 章故障二)。exec()、uvminit()、uvmcopy() 都经过这个函数。排查这类问题时, 要用实际装进 SD 卡的那个 ELF 去反查 ELR, 不能用另一次构建的符号文件。

问题 10: 输出到 fixed WB/level 后长时间停顿

日志输出.

```

camshot: frame received 640x480, 384040 bytes
camshot: unpacking RAW10...
camshot: sensor test pattern; fixed WB/level
(长时间没有新输出)

```

问题分析. 看起来像转换死循环, 但测试图案模式下转换并不做统计, 下一步其实是以 O_TRUNC 打开已存在的 /boot/camera.bmp。截断要释放旧文件的簇链, 旧的 fat_free_chain() 对每个簇都读一次 FAT、再对两份 FAT 各做一次读改写, 覆盖一个 320x240 的 BMP 就要五百多次 SD 扇区事务, 而且每次都是 PIO 轮询。

解决方案. fat_free_chain() 缓存当前 FAT 扇区, 在其中批量清除属于这条链的项, 换扇区时才把整扇区写到每份 FAT (6.7 节)。同时 camshot 在打开输出文件、写 BMP 头和开始转换之前各打印一行阶段日志, BMP 每 16 行写一次, 并用版本串 converter fixed-pattern-v2 fat-progress batch16 确认 SD 卡上的程序已经更新——以后再停住, 可以直接看出停在哪一步。

问题 11: 新建文件停在 directory-entry begin

日志输出.

```

camshot: sensor test pattern; fixed WB/level
camshot: preparing output /boot/cavfs: boot create begin path=/boot/cam02.bmp sub=/cam02.bmp mode=601
m02.bmp (230454 bytes)...
vfs: boot create stat complete exists=0
vfs: boot create directory-entry begin
(此后再无输出)

```

问题分析. 先排除两个干扰: /boot/cavfs: ... m02.bmp 是用户态逐字符输出与内核 printf 交错的结果, 路径本身没坏 (7.9.8 小节); exists=0 说明 stat 已经正常返回, 阻塞点在 fat32createfile() 里。这和问题 10 不是同一个问题: exists=1 且停在 truncate begin 才是释放旧簇链慢。第一个猜测是“根目录要扩展, 簇分

配器扫描太慢”，于是先给 `fat_alloc_cluster()` 加了按扇区缓存和 `FSInfo` 提示，并加了 `fat32: extending root directory` 日志——真机上仍停在同一行，连新日志都没出现，说明根本没走到分配器。

真正的原因是内核栈。每个进程的内核栈只有 4 KiB，旧的 `fat32createfile()` 在栈上放了一个路径匹配结构、22 个目录槽和 256 字节的文件名，又嵌套调用 `fat_find_path()`，后者再放 256 字节文件名、520 字节 LFN 字符数组和 20 个目录槽。光这些数组就接近 3.5 KiB，加上 VFS、系统调用的栈帧和保存的寄存器，越过了 4 KiB。`stat` 只用到其中一层，所以没事。至于为什么是“停住”而不是一条报错：内核栈之间隔着不映射的保护页（第 4 章），越界先落进保护页、触发 EL1 数据中止；而 EL1 的异常入口要在同一个 `SP_EL1` 上压 272 字节保存现场（3.2 节），这一压又落在保护页里，异常入口自己再次中止，CPU 在异常向量里反复进出，串口上什么也不会出现。

解决方案。 把这些大数组移出栈，放进 FAT32 私有的静态工作区 `fat_work`。所有 FAT32 操作本来就持 `fat32_lock` 串行执行，一个工作区就够，不必加大每个进程的内核栈。反汇编确认栈帧降到 `fat32createfile` 464 字节、`fat_find_path` 176 字节。`create` 路径上加了更细的阶段日志（`create lookup`、`create reserve`、`extending root directory`），可以直接区分是查找、保留目录项还是分配簇出了问题；之前对分配器的优化也保留下来。修复后的完整日志见 7.9.8 小节。

小结。 回头看这十一个问题，有效的做法是一样的：让每一层都有自己的日志，日志里带上能判读的原始数值（`flags`、`fsel`、`levels`、`status`、`esr`、`exists`），然后只把日志实际证明了的東西当作结论。几次走弯路，都是因为把“两行日志挨在一起”当成了因果（问题 8），或者先修了猜测中的原因而没有确认程序是否真的走到那里（问题 11）。

7.9.10 当前状态

真机（Pi 3B+，Camera v1.3）上，v8 驱动从第一次访问起就用 25 kHz 软件 SCCB，稳定读到芯片 ID 0x5647 并绑定到 `/dev/video0`；Unicam 以轮询方式拿到完整帧，`camshot -t 1 -s -o /boot/cam02.bmp` 能把彩条写成 BMP（7.9.8 小节）。还没有完成的工作有：

- 验证 CSI1 中断号并打开中断路径（保留轮询兜底）；
- 查清 BSC0 硬件读不到芯片 ID 的原因（控制器路由还是固件占用），以便退回硬件 I2C；
- 更多模式：Linux 的 1296×972 等模式每帧超过 1.5 MB，需要扩大 CAMDMA，桥接驱动不用改；目前也还没有选择模式的接口；
- 每次拍照都完整地上电和预热（约 1 秒），没有连续取流和 `poll()`，也没有手动曝光；
- `arasan_sdio.c`、`dwc2.c`、`sd.c` 里仍各有一份私有 mailbox 代码，应统一到 `mbox.c`；
- `config.txt` 不能打开固件的摄像头支持（`start_x=1`、`camera_auto_detect=1`），否则固件会占用同一个传感器和 I2C。

注 摄像头驱动是全书最能体现“分层”价值的例子：mailbox、I2C、时钟、电源域、CSI-2 接收器、DMA、帧状态机、字符设备、用户态图像处理，每一层都可以单独验证。真机调试也正是这样一层层推进的：先看 mailbox 回读，再用 bit-bang 判断 I2C 故障在控制器内外，再用传感器内部彩条（`camshot -t 1`）验证 CSI-2 和 DMA，最后才看真实画面。

第八章 用户进程、线程与 libc

本章内容

- ❑ 用户程序的形态与系统调用桩
- ❑ 从 init 到 shell
- ❑ 用户程序一览
- ❑ 用户线程：clone、texit、tjoin
- ❑ libc 与 POSIX 源码兼容层
- ❑ 在 xv6 里编译 C 程序
- ❑ 一个大型用户程序：sshd

8.1 用户程序的形态

8.1.1 为什么需要系统调用

进程隔离的第一步，是把所有用户程序放到特权最低的 EL0：在这一级，程序不能修改页表基址、不能屏蔽中断、不能访问没有映射给它的内存，也就无法改写内核为隔离所做的任何设置。否则，其他一切隔离手段都没有意义。

可是用户程序总要打印字符、读文件、发网络包，而这些都需要直接操作设备。我们不能让每个程序直接去写 UART 寄存器，而是由内核提供一组受控的入口——系统调用。它不能是普通的函数调用，因为每次调用都要把执行级别从 EL0 提升到 EL1，而提升级别只有一条路：产生异常。所以每个系统调用本质上都是一次同步异常：用户程序准备好参数，执行 svc 指令；硬件把它带到 EL1 的异常向量；内核检查参数、完成请求，然后执行普通的异常返回，从 svc 的下一条指令继续。进程隔离的第三个方面——每个进程有自己独立的内存视图——由页表完成（第 4 章）。

8.1.2 链接与装载

xv6 的用户程序是静态链接的 ELF64 文件，链接在虚拟地址 0，入口是 main：

```
_%: %.o $(ULIB)
$(LD) $(LDFLAGS) -N -e main -Ttext 0 -o $@ $^
```

-N 让代码段和数据段放在同一个可读写可执行的段里，exec() 只需处理一个 PT_LOAD。没有动态链接，也没有 C 运行时启动代码：exec() 把 argc、argv 放进 x0、x1，把 PC 设为入口地址，main 返回后由用户库调用 exit()——实际上 xv6 的程序习惯在 main 末尾显式调用 exit(0)。

8.1.3 系统调用桩

user/usys.pl 为每个系统调用生成三条指令：

```
.global write
write:
    mov x7, #SYS_write    // 系统调用号放 x7
    svc #0                // 陷入 EL1
    ret                   // 返回值已在 x0
```

参数按 AArch64 调用约定已经在 x0~x5 里，所以桩函数什么都不用搬。内核侧 syscall() 用 trapframe->x7 索引函数表，返回值写回 trapframe->x0。目前共有 70 多个系统调用：除了 xv6 原有的 21 个，还有同步对象、网络（UDP、ICMP、TCP socket、epoll）、挂载、TTY 会话、线程、调度属性、高精度时钟等。

 **笔记** 系统调用号放在 x7，这是本项目沿用的约定；Linux AArch64 用的是 x8。所以 Linux 的二进制或汇编系统调用包装不能直接拿来用，第 8.5 节的 POSIX 层是源码兼容，不是二进制兼容。

8.1.4 内核如何分发系统调用

svc 进入 el0sync，usavereg 把全部用户寄存器存进 trapframe，usertrap() 检查 ESR_EL1 的异常类别是否为 0x15（来自 AArch64 的 SVC），然后打开中断、调用 syscall()（第 3.2 节）。分发函数在 `kernel/syscall.c` 中：

```
void syscall(void)
{
    int num;
    struct proc *p = myproc();

    num = p->trapframe->x7;
    if(num > 0 && num < NELEM(syscalls) && syscalls[num]) {
        p->trapframe->x0 = syscalls[num]();
    } else {
        printf("%d %s: unknown sys call %d\n", p->pid, p->name, num);
        p->trapframe->x0 = -1;
    }
}
```

syscalls[] 是一张以系统调用号为下标的函数指针表。处理函数先检查调用号是否越界，越界就返回 -1，而不是像教学内核那样直接停机——一个用户程序传错调用号，不应该让整个系统死掉。

参数不需要搬运：用户态执行 svc 时它们就在 x0 ~ x5 中，现在原样躺在 trapframe 里，各个 sys_* 函数用 argint()/argaddr()/argstr() 按编号取出：

```
static uint64 argraw(int n)
{
    struct proc *p = myproc();
    switch (n) {
        case 0: return p->trapframe->x0;
        case 1: return p->trapframe->x1;
        ...
        case 5: return p->trapframe->x5;
    }
    panic("argraw");
    return -1;
}
```

指针参数只取出数值，不在此处检查合法性；真正访问用户内存时由 copyin()/copyout() 查用户页表，非法地址会使系统调用返回 -1，而不会让内核出错。

返回值写回 p->trapframe->x0，而不是直接放进 x0 寄存器。原因是返回用户态时 urestorereg 会从 trapframe 恢复所有寄存器，直接写寄存器的值会被覆盖；只有改写保存的那一份，用户程序在 svc 之后才能在 x0 中看到它。

8.1.5 两个栈指针与返回级别

EL0 和 EL1 各有自己的栈指针。从用户态陷入时，硬件立刻把 `sp` 换成 `SP_EL1`（内核栈），用户栈指针原样留在 `SP_EL0` 中。即使内核从不使用 `SP_EL0`，也必须把它连同其他寄存器一起保存和恢复：陷入期间可能发生进程切换，下一个返回用户态的进程需要的是它自己的用户栈指针。所以 `usaverreg` 把 `SP_EL0` 存进 `trapframe` 的 `sp` 字段，`urestorereg` 把它写回。

内核态的 `saverreg` 则不需要恢复 `sp`：内核栈是同一个，即使中间发生了进程切换，`swtch()` 也已经把 `sp` 换成了正确的值。

`eret` 回到哪一级，由 `SPSR_EL1` 的模式字段决定，不需要另外指定，所以总是回到异常发生时所在的那一级。利用这一点，内核可以“伪造”一次从未发生过的异常来进入用户态：`userinit()` 在第一个进程的 `trapframe` 中写入 `elr = 0`、`spsr = 0`（模式 EL0t）、`sp = PGSIZE`，`usertrapret()` 一执行 `eret`，CPU 就第一次来到 EL0（第 5 章）。启动时从 EL2 降到 EL1 用的也是同一个技巧（第 3 章）。

8.2 从 init 到 shell

第一个用户进程 `initcode` 执行 `exec("/init")`（第 5 章）。`user/init.c` 做的事情相当于一个嵌入式 Linux 的启动脚本：

1. 创建 `/dev`，用 `mknod` 建立 `console`、`tty`、`ttyS0`、`input/event0`、`video0` 设备节点；以 `/dev/ttyS0` 打开 `fd0`，复制为 `fd1`、`2`；
2. 创建 `/proc`、`/boot`、`/mnt/ext2`、`/etc`、`/root`、`/bin` 等目录，为根目录下的程序在 `/bin` 中建立硬链接；
3. 首次启动时生成 `/etc/hostname`、`/etc/fstab`、`/etc/passwd`（默认账号 `root`，密码 `xv6`）、`/etc/rc`；
4. 按 `/etc/fstab` 逐行调用 `mount()`；
5. 如果存在 `/etc/wifi.conf`，先运行 `/bin/wifi` 连接无线网络；
6. 在一个循环里 `fork` 并 `exec /bin/login`；`login` 退出后回收它，再启动一个新的登录提示；同时回收所有过继来的孤儿进程。

`login` 校验密码后调用 `tty_attach()` 建立会话，切到 `passwd` 中指定的家目录，`exec` 指定的 `shell`。`sh` 在 `xv6` 原版基础上增加了：读取 `/etc/rc`、`PATH` 搜索（找不到时再回退到根目录，作为存储异常时的恢复路径）、`export`、`exit/logout` 内建命令，以及把外部命令设为前台进程组的作业控制。

8.3 用户程序一览

表 8.1: 用户程序 (UPROGS)

类别	程序
基本工具	<code>sh</code> 、 <code>init</code> 、 <code>login</code> 、 <code>ls</code> 、 <code>cat</code> 、 <code>echo</code> 、 <code>grep</code> 、 <code>wc</code> 、 <code>mkdir</code> 、 <code>rm</code> 、 <code>ln</code> 、 <code>mv</code> 、 <code>touch</code> 、 <code>kill</code> 、 <code>ps</code> 、 <code>edit</code> 、 <code>file</code>
文件系统	<code>ext2ls</code> 、 <code>ext2cat</code> 、 <code>stressfs</code>
网络	<code>wifi</code> 、 <code>dhcp</code> 、 <code>ping</code> 、 <code>netdns</code> 、 <code>nettest</code> 、 <code>tftp</code> 、 <code>tcpd</code> 、 <code>epollserver</code> 、 <code>sshd</code>
调度与同步	<code>chrt</code> 、 <code>taskset</code> 、 <code>nice</code> 、 <code>renice</code> 、 <code>rtlat</code> 、 <code>spin</code> 、 <code>syncdemo</code> 、 <code>prodcons</code>
线程与 POSIX	<code>threadtest</code> 、 <code>posixtest</code>
设备	<code>camshot</code> 、 <code>ptytest</code> 、 <code>vmmap</code>
开发	<code>tcc</code>
测试	<code>usertests</code> 、 <code>forktest</code> 、 <code>grind</code> 、 <code>zombie</code>

几个调度工具的用法与 Linux 相同，它们都是“先修改自己的调度属性，再 `exec` 目标程序”，利用了 `fork/exec` 对调度属性的继承：

```
chrt -f 80 rtlat      # 以 SCHED_FIFO 优先级 80 运行 rtlat
taskset 0x8 spin &   # 只允许在 CPU3 上运行
```

```
nice -n 10 grind           # 降低普通进程的时间片
chrt -f -p 60 6           # 修改 PID 6 (某个 kworker) 的策略
```

8.4 用户线程

8.4.1 设计

原来的模型是“一个 struct proc 对应一个独立的地址空间”。加入线程时，项目没有引入新的“线程控制块”，而是让每个线程仍然是一个完整的 struct proc——有自己的内核栈、trapframe、调度属性、TID 和 CPU 亲和性——只是同一线程组的成员共享同一个引用计数的 struct vmSPACE (图 8.1)。

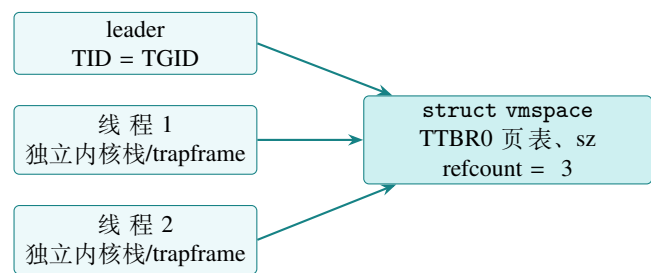


图 8.1: 线程组共享 vmSPACE

调度器切换到任何一个线程时，switchuvm() 都写入 p->vm->pagetable，所以同组线程看到相同的代码、全局变量、堆和彼此的栈；而它们的内核栈和 trapframe 各自独立，可以同时多个 Cortex-A53 核上运行。

8.4.2 系统调用

表 8.2: 线程相关系统调用

调用	作用
clone(entry, arg, stack_top)	创建共享地址空间的新调度实体: x0=arg, SP=stack_top, ELR=entry
texit(status)	只退出调用线程; leader 调用时等同于整个进程退出
tjoin(tid, &status)	等待并回收同组中的指定线程 (0 表示任意)
gettid() / getpid()	返回线程 ID / 线程组 ID

用户库在其上提供更安全的包装，它负责分配 16 字节对齐的 16 KiB 用户栈，并保证线程函数返回后自动调用 texit()：

```
static void thread_boot(void *opaque) {
    struct thread_start start = *(struct thread_start *)opaque;
    free(opaque); // 启动记录用完即释放
    start.fn(start.arg);
    texit(0); // 线程函数返回后自动退出
}

int thread_create(thread_t *thread, void (*fn)(void *), void *arg) {
    thread->stack = malloc(THREAD_STACK_SIZE + 16);
    struct thread_start *start = malloc(sizeof(*start));
    start->fn = fn; start->arg = arg;
    uint64 top = ((uint64)thread->stack + THREAD_STACK_SIZE + 16) & ~15ULL;
    thread->tid = clone(thread_boot, start, (void *)top);
}
```

```
...
}
```

用户栈要等 `thread_join()` 中内核回收了线程之后才释放，否则线程可能还在用这块栈。

8.4.3 clone 为什么“返回两次”

`fork()` 会返回两次：父进程中返回子进程的 `pid`，子进程中返回 0，因为子进程的 `trapframe` 是父进程的完整拷贝，只把 `x0` 改成了 0。Linux 的 `clone` 系统调用语义相同，所以 `glibc` 的 `clone()` 包装函数要在汇编里处理两次返回：调用前把函数指针和参数存进不会被系统调用破坏的寄存器；系统调用返回非 0 说明还在原线程，直接返回调用者；返回 0 说明已经身处新线程，就取出函数指针调用它，函数结束后再调用 `exit`——新线程永远不会回到包装函数的调用者那里。

xv6-aarch64 的 `clone(entry, arg, stack_top)` 换了一种更简单的语义：内核直接把新线程的入口、参数和栈写进它的 `trapframe`，新线程第一次回到用户态时就从 `entry` 开始执行，用户态不需要任何汇编包装：

```
int threadclone(uint64 entry, uint64 arg, uint64 stack_top)
{
    ...
    if(entry == 0 || stack_top < 16 || (stack_top & 15) != 0)
        return -1;                // 栈必须 16 字节对齐
    ...
    vm->refcount++;                // 共享同一个地址空间
    vm->space_put(np->vm);
    np->vm = vm;
    np->is_thread = 1;
    np->tgid = p->tgid;

    *(np->trapframe) = *(p->trapframe);
    np->trapframe->elr = entry;      // eret 之后从这里开始
    np->trapframe->sp = stack_top;   // 新的用户栈
    np->trapframe->x0 = arg;         // 第一个参数
    np->trapframe->x30 = 0;          // 不允许“返回”
    ...
    make_runnable(np, 0);
    return tid;
}
```

`x30` 被清零，线程函数如果直接 `ret` 就会跳到地址 0 出错，所以用户库的 `thread_boot()` 包了一层，保证函数返回后调用 `texit()`。线程退出后变成 ZOMBIE，等 `tjoin()` 回收——与进程的 `exit/wait` 是同一种约定：先保留“尸体”，让等待者能取到退出状态，再由等待者释放资源。

8.4.4 多核下的细节

- `sbrk()` 修改共享的页表时持有 `vm->space.lock`，改完执行 `tlbi vmalleis` 广播失效，覆盖其他核上正在运行的同组线程；
- 用户态的 K&R `malloc` 加了一把基于 `ldaxr/stxr` 的锁；
- `exec()` 只在线程组只剩调用者时才允许执行；`exit()` 先终止并回收同组线程；`kill(pid)` 以 TGID 为目标，标记并唤醒整组；

- `clone()` 像 `fork()` 一样复制文件描述符引用——底层 `struct file` 和偏移共享，但描述符表独立。这与 Linux 的 `CLONE_FILES` 不同，是有意保留的简化。

`threadtest` 创建两个线程，用内核同步对象保护共享计数器，允许它们被调度到不同 CPU，预期输出 `threadtest: PASS counter=2000`。

8.5 libc 与 POSIX 源码兼容层

8.5.1 libc.a

用户库被打包成一个静态库：

- `ulib.c`：字符串、内存函数、`gets`、`stat`、`atoi`；
- `usys.S`：系统调用桩；
- `printf.c`：`printf/fprintf`，支持 `%d %x %p %s %c`；
- `umalloc.c`：K&R 风格的 `malloc/free`，用 `sbrk` 向内核要内存；
- `thread.c`：上一节的线程包装；
- `posix.c`：POSIX 包装层；
- `tweetnacl.c`、`ssh_crypto.c`：密码学原语（`sshd` 使用）。

8.5.2 POSIX 层

目标是让只依赖常见 POSIX 接口的 C 源码能直接用 xv6 工具链重新编译。它由三部分组成：

1. `include/` 提供标准名字的头文件：`stdio.h`、`stdlib.h`、`string.h`、`unistd.h`、`fcntl.h`、`errno.h`、`time.h`、`pthread.h`、`semaphore.h`、`sys/types.h`、`sys/stat.h`、`sys/socket.h`、`netinet/in.h`、`arpa/inet.h`、`sys/epoll.h`；
2. `user/posix.c` 把标准 API 映射到 xv6 已有的能力；
3. 内核补上无法只靠库函数实现的语义，如 `lseek()`，以及 `socket()` → `bind()` → `listen()` → `accept()` 的生命周期。

表 8.3: POSIX 接口到 xv6 的映射

POSIX 接口	实现
<code>open/read/write/lseek/fstat</code>	直接对应系统调用； <code>O_CREAT</code> 是 <code>O_CREATE</code> 的别名
<code>pthread_create/join/exit</code>	<code>clone/tjoin/texit</code> ； <code>pthread_join</code> 同时回收用户栈
<code>pthread_mutex_*</code> 、 <code>sem_*</code>	内核同步对象；支持 <code>PTHREAD_MUTEX_INITIALIZER</code> 延迟初始化
<code>clock_gettime(CLOCK_MONOTONIC)</code>	<code>clock_us()</code> ，读 <code>CNTVCT_ELO</code>
<code>sleep/usleep/nanosleep</code>	秒语义；原来按 tick 计的 <code>sleep</code> 改名为 <code>sleep_ticks</code>
<code>socket/bind/listen/accept</code>	统一的 <code>socket</code> 文件描述符，可用 <code>read/write/close/epoll</code>

`posixtest` 只包含标准头文件，验证文件读写与 `lseek`、POSIX 信号量、两个 `pthread` 并发更新共享变量、静态初始化的互斥锁以及 `socket` 生命周期。

这仍是第一阶段：系统调用失败只返回 `-1`，没有传回具体错误码，`errno` 也还不是线程局部的；缺少 `waitpid`、`dup2`、条件变量、目录流和 `TCP connect()`。

8.6 在 xv6 里编译 C 程序

`tcc` 是一个面向教学的极小 C 编译器，运行在 xv6 用户态，直接输出 `exec()` 能加载的 ELF64 文件，不需要汇编器和链接器：

```
$ edit hello.c
int main() {
    int answer = 6 * 7;
    printf("hello from xv6 C\n");
    return answer - 42;
}
$ tcc hello.c -o hello
$ hello
hello from xv6 C
```

它支持整数局部变量、四则运算、`return`、只含字面量的 `printf/puts` 和注释。生成的代码用 `x29` 指向 256 字节的局部变量区，表达式结果放在 `x0`，字符串地址用 `adr` 指令取得，所以无需重定位；输出和退出直接用 `svc #0` 调用 `SYS_write` 和 `SYS_exit`。文件布局是一个 ELF 头、一个程序头、机器码和字符串池。

一个能在自己身上编译并运行程序的系统，才算是有了一点“自举”的味道。

8.7 一个大型用户程序：sshd

`sshd` 有一千多行，是整个系统里最复杂的用户程序，也是对内核各子系统的一次综合考验。它用到了：

- TCP socket 与 `accept`——网络协议栈；
- 密码学全部在用户态完成：基于 TweetNaCl[3] 的 X25519 密钥交换和 Ed25519 主机密钥签名，传输层用 AES-128-CTR 加密、HMAC-SHA256 校验；熵来自内核的硬件随机数发生器 (`kernel/rng.c`)；
- `/etc/passwd` 口令认证；
- PTY：为每个会话分配一对伪终端，`shell` 的 `fd 0/1/2` 指向 PTY 从端，`sshd` 读写主端并加密转发；
- 会话与控制终端：SSH 会话中的 `/dev/tty` 指向该会话的 PTY。

调试 `sshd` 时发现的问题推动了内核的多处改进：字符回显慢暴露了“网卡每 100ms 轮询一次”和“TCP `write()` 同步等 ACK”两个瓶颈，促成了 SDIO 中断、`workqueue` 下半部、NAPI 和 TCP 异步发送队列；`ps` 输出跑到串口上，促成了 `sys_ps()` 改为返回缓冲区。

注 从 `_entry` 的第一条 `mrs` 指令，到一个可以通过 Wi-Fi 被 SSH 登录、能在多核上运行实时任务、能用 `tcc` 编译程序的系统，中间经过的每一层——串口、页表、进程、文件系统、驱动、协议栈、`libc`——都写在这本书里了。它们都还很简单，但每一层都在真机上跑过、出过错、被修好过。这也许就是“从裸机到操作系统”最有价值的部分。

第九章 可选工作

本章内容

- ❑ 为什么要在 xv6 里读写 ext2，分区与日志放在哪里
- ❑ ext2 的磁盘结构：superblock、块组、inode、目录
- ❑ 从文件偏移到块号：直接块与三级间接块
- ❑ 接入 VFS：按 inode 打开、孤儿 inode
- ❑ 持久化：sdflush() 写屏障
- ❑ 外置事务日志 xjournal：格式、提交协议、崩溃恢复
- ❑ 用 WAL 程序和随机断电做测试

前面八章是系统的主线：没有它们，内核就启动不了、跑不了程序、用不了网络。本章收录的是主线之外的工作。它们不影响系统运行，但能把某个子系统做得更完整，也适合作为读者自己动手的练习。第一项是 ext2：让 xv6 能可靠地读写一块 Linux 也认识的分区，并在断电后保持一致。代码在 `kernel/ext2.c`、`kernel/xjournal.c` 和 `user/waltest.c`，说明文档是 `readme/readme_ext2.md`。

9.1 ext2 读写与外置事务日志

9.1.1 动机与分区布局

xv6 的根文件系统（第 6 章）只有 xv6 自己认识，开发机上的 macOS 或 Linux 无法直接打开它。要在两边交换文件，需要一种双方都懂的格式。FAT32 启动分区可以，但它只支持根目录、没有权限位，平时也不该频繁写。ext2 是 Linux 最经典的文件系统，结构简单、文档齐全，非常适合在教学内核里实现。

第 6.6 节介绍 VFS 时，ext2 后端还只能读：只支持直接块和一级间接块，默认按只读挂载。本节把它补成一个可写的后端，并解决可写之后立刻出现的问题：写到一半断电怎么办。

ext2 放在 SD 卡的第三个分区，xv6 根文件系统仍在 p2，二者互不替代（图 9.1）。ext2 本身没有日志，这里给它配了一个外置的事务日志，放在 p2 尾部：p2 有 400 MiB，xv6 镜像只用前 32 MiB（`FSSIZE = 32768` 块，即 65536 扇区），后面的空间本来就闲着。

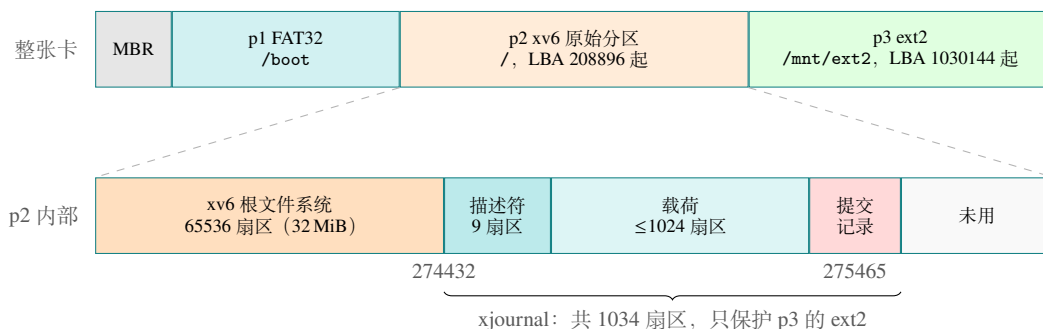


图 9.1: ext2 与外置日志在 SD 卡上的位置。下排数字是真机上的绝对 LBA：日志起点 = p2 起点 208896 + 65536

日志的位置在启动时算出。ext2init() 先向根块设备（第 6.5 节）要 p2 的位置，再跳过 xv6 镜像：

```
uint32 rlb, rsec, jstart = FSSIZE * (BSIZE / SECTOR); // 65536
if(rootdev_raw_info(&rlb, &rsec) == 0 && rsec > jstart &&
    xj_init(rlb + jstart, rsec - jstart) == 0) // 先恢复日志
    e2.journal = 1;
else
    printf("ext2: no raw xv6 root partition; external journal disabled\n");
```

`xj_init()` 在解析任何 ext2 元数据之前就检查并重放日志，这样后面读到的 `superblock` 和位图都已经是一致的。QEMU 裸镜像没有 p2，这时日志关闭，ext2 退回逐扇区同步写。更新根文件系统的 `/dev/sdroot`（第 6.8 节）只写前 `FSSIZE` 块，不会碰到日志区。

9.1.2 ext2 的磁盘结构

ext2 把分区切成大小相同的块（1、2 或 4 KiB），再把连续的块分成若干块组。每个块组都有自己的位图和 `inode` 表，这样文件的数据块可以和它的 `inode` 放在同一个组里，减少磁头移动——这是为机械硬盘设计的，但结构本身对 SD 卡同样适用。图 9.2 是 4 KiB 块时第 0 组的布局。

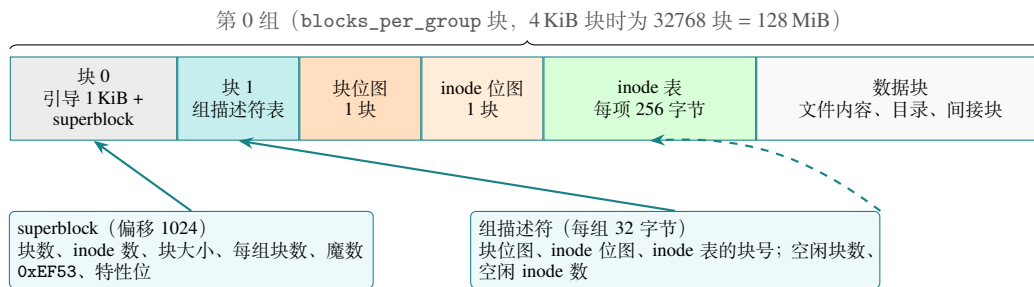


图 9.2: ext2 第 0 块组的布局。其他块组结构相同；开启 `sparse_super` 时只有第 0、1 和 3、5、7 的幂次组保存 `superblock` 与描述符表的备份

`setup_ext2()` 从 `superblock` 读取表 9.1 中的字段，并在用之前逐个检查。最重要的是三组特性位：`compat` 中的 `has_journal` 说明这是 ext3/ext4，`incompat` 中除 `filetype` 以外的任何一位（如 `extent`、64 位块号）都意味着本驱动看不懂磁盘布局。遇到这些就拒绝挂载，而不是把一个 ext4 分区当成 ext2 去写坏它。

表 9.1: `setup_ext2()` 读取的 `superblock` 字段

偏移	字段	用途
0 / 4	<code>inodes_count / blocks_count</code>	越界检查；块数 × 块大小不能超过分区
12 / 16	空闲块数 / 空闲 inode 数	分配、释放时加减
20	<code>first_data_block</code>	1 KiB 块时为 1，否则为 0
24	<code>log_block_size</code>	块大小 = 1024 << 该值
32 / 40	每组块数 / 每组 inode 数	块号、inode 号到组号的换算
56	<code>magic</code>	必须是 0xEF53
58	<code>state</code>	第一次写时清除 <code>VALID_FS</code>
88	<code>inode_size</code>	动态版本里一般为 256
92 / 96 / 100	<code>compat / incompat / ro_compat</code>	拒绝 <code>journal</code> 、 <code>extent</code> 、64bit 等

inode 与目录。 inode 号从 1 开始，2 号是根目录。找 n 号 inode 时，先算组号 $g = (n - 1) / \text{每组 inode 数}$ 和组内序号 $i = (n - 1) \bmod \text{每组 inode 数}$ ，读第 g 个组描述符拿到 inode 表的块号，inode 就在表内偏移 $i \times \text{inode_size}$ 处：

```
gd_off = (uint64)e2.gd_block * e2.block_size + group * e2.desc_size;
diskbytes(gd_off, inodebuf, e2.desc_size);
table = le32(inodebuf + 8); // inode 表块号
off = (uint64)table * e2.block_size + (uint64)index * e2.inode_size;
diskbytes(off, inodebuf, e2.inode_size);
ip->mode = le16(inodebuf); ip->links = le16(inodebuf + 26);
ip->size = le32(inodebuf + 4);
if((ip->mode & EXT2_S_IFMT) == EXT2_S_IFREG) // 普通文件的高 32 位
    ip->size |= (uint64)le32(inodebuf + 108) << 32;
for(int i = 0; i < 15; i++) ip->block[i] = le32(inodebuf + 40 + i*4);
```

off 必须用 64 位计算：一个几 GiB 的分区里，后面块组的 inode 表很可能位于 4 GiB 之后，32 位乘法会悄悄溢出，读到另一个位置的“inode”。文件大小同理，高 32 位放在 inode 偏移 108 处。

目录是普通的数据块，里面是一串变长目录项：inode 号（4 字节）、本项长度 rec_len（2 字节）、名字长度和文件类型（各 1 字节），然后是名字。删除一项时不移动后面的数据，而是把它的 rec_len 并入前一项；新增时找一个 rec_len 比实际需要大的项把它切开。e2dirlookup()、e2diradd()、e2dirremove()、e2dirreplace() 分别实现查找、插入、删除和改指向。

9.1.3 从文件偏移到块号

inode 的 block[15] 是一棵最多四层的树（图 9.3）：前 12 项直接指向数据块，第 13 项指向一个装满块号的间接块，第 14、15 项分别多套一层和两层。4 KiB 块时每个间接块能放 $n = 1024$ 个块号，所以一个文件最多有

$$12 + n + n^2 + n^3 = 12 + 1024 + 1\,048\,576 + 1\,073\,741\,824 \text{ 块} \approx 4 \text{ TiB}.$$

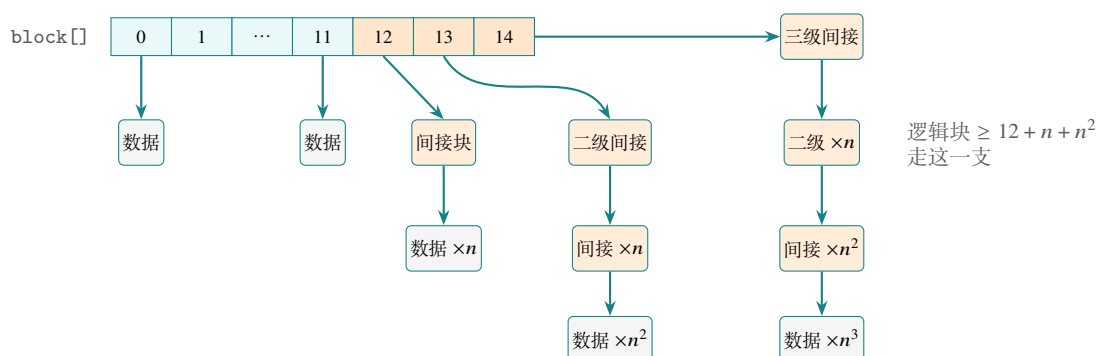


图 9.3: ext2 inode 的块指针树。块号为 0 表示空洞：读时返回全 0，写时才分配

fileblock() 按逻辑块号依次减去每一层能覆盖的块数，落到哪一层就沿哪一支往下读：

```
static uint32 fileblock(struct inode *ip, uint64 logical) {
    uint64 n = e2.block_size / 4;
    if(logical < 12) return ip->block[logical];
    logical -= 12;
    if(logical < n){
        // 一级间接
        if(ip->block[12] == 0 || readblock(ip->block[12], ptrbuf[0]) < 0) return 0;
        return ptrbuf[0][logical];
    }
    logical -= n;
    if(logical < n*n){ ... 读 block[13], 再读 ptrbuf[0][logical / n] ... }
    logical -= n*n;
    // 三级间接
    ... 读 block[14], 再依次取 logical / n^2, (logical % n^2) / n, logical % n ...
}
```

三层用三个独立的 4 KiB 缓冲 ptrbuf[0..2]，它们是全局变量而不是局部数组——内核栈只有 4 KiB，放不下其中任何一个（第 7 章的摄像头调试记录里有过同样的教训）。所有 ext2 操作都持着同一把锁，所以全局缓冲是安全的。

写的方向由 set_fileblock() 和 get_or_alloc_fileblock() 完成：路径上缺哪一层间接块就分配哪一层。截断时 clear_fileblock() 释放数据块，再检查它所在的间接块是否已经全为 0 (ptr_empty())，是就把间接块也释放掉，逐层向上。waltest large 在偏移 5 GiB 处写一个字节：5 GiB/4 KiB = 1 310 720 > 12 + n + n^2，正好走到三级间接这一支，文件只占 4 个块。

9.1.4 分配与释放

分配一个数据块要改三处：块位图里的一位、组描述符里的空闲块数、superblock 里的空闲块数。alloc_block() 从首选的组开始找第一个空闲位：

```
for(uint32 bit = 0; bit < count; bit++){
    if((blockbuf[bit >> 3] & (1U << (bit & 7))) != 0) continue;
    if(freed_in_tx(first + bit)) continue;           // 本事务刚释放的块，暂不复用
    blockbuf[bit >> 3] |= 1U << (bit & 7);
    writeblock(bitmap, blockbuf);                    // 位图
    put16(gd + 12, le16(gd + 12) - 1);
    writegd(group, gd);                              // 组描述符
    adjust_super_count(12, -1);                      // superblock
    *out = first + bit;
    memset(blockbuf, 0, e2.block_size);
    writeblock_data(*out, blockbuf);                  // 新块清零，直接写
    return 0;
}
```

两处细节和后面的日志有关。第一，freed_in_tx()：一个块如果在同一个事务里刚被释放，就不能立刻再分配出去并原地改写——事务一旦中止或者断电，释放会被撤销，这个块又回到原主人手里，内容却已经是别人的了。第二，新块直接清零而不经日志：在事务提交之前，没有任何 inode 指向它，写什么都不会被看见。

inode 的分配 (alloc_inode()) 和释放 (free_inode()) 是同样的三步，只是改的是 inode 位图和空闲 inode 数；目录还要维护组描述符里的目录计数。

9.1.5 接入 VFS：按 inode 打开

第 6.6 节的 VFS 最初是“按路径”的：每次 read、write 都把路径传给后端重新查找。对 ext2 这会带来两个问题：每次读写都要从根目录走一遍；更糟的是，文件被重命名或删除以后，已经打开的描述符会突然指向别的文件，或者失效。Unix 的语义是：打开的是 inode，而不是名字。

为此 vnode_ops 增加了一组可选的句柄操作：

```
struct vnode_ops {
    ... // 原来的按路径操作
    int (*open)(char*, uint64*);           // 解析一次路径，返回句柄
    void (*release)(uint64);               // 最后一次 close
    int (*hstat)(uint64, struct stat*);
    int (*hread)(uint64, uint64, void*, int); // 句柄、偏移、缓冲、长度
    int (*hwrite)(uint64, uint64, int, uint64, int);
    int (*htruncate)(uint64, uint64);
};
```

vfsopen() 打开普通文件时，如果后端提供了 open，就调用它得到句柄并记在 vnode 里，之后的 read/write/fstat/ftruncate 都走句柄版本。ext2 的句柄就是 inode 号；ext2open() 还在一个小表 e2.open[] 里给这个 inode 记一个引用计数。

有了引用计数，就能实现“删除仍被打开的文件”：unlink 或被 rename 覆盖时，目录项立即消失；如果 inode 的链接数降到 0 而它还被打开着，drop_link() 只把它标成孤儿，块不释放。旧描述符照常读写原来的数据，最后一次 close 时 ext2release() 才在一个事务里截断并释放 inode。

rename 也补全了 POSIX 语义：目标已存在时原子地替换它（类型必须相同，目录必须为空）；把目录移进

它自己的子树会被 `is_ancestor()` 拒绝——否则这棵子树会从根上脱落，形成一个环。

锁。 所有 ext2 操作由睡眠锁 `e2.lock` 串行化。最初用的是自旋锁，但 SD 卡是轮询 PIO，一次 `fsync` 最长可能等 5 秒，持自旋锁期间 CPU 一直关着中断，键盘和网络都会卡住。换成睡眠锁后，等待的进程会让出 CPU。

9.1.6 持久化：sdflush() 写屏障

`sdsector(lba, buf, 1)` 返回时，CMD24 的数据已经传给卡，但卡可能还在内部把它编程进 NAND。这时断电，这个扇区可能是新的、旧的，甚至是坏的。要建立“之前的写都已落盘”这样一个边界，需要单独的屏障：

```
int sdflush(void) {                                // kernel/sd.c
    acquire(&sdlock);                               // 与所有传输互斥
    asm volatile("dsb sy" ::: "memory");
    uint64 deadline = r_cntvct_el0() + 5ULL * r_cntfrq_el0();
    for(;;){
        uint32 status = 0;
        if(send_command(CMD_INDEX(13) | CMD_RSPNS_48 | CMD_CRCCHK_EN | CMD_IXCHK_EN,
                        sd_rca << 16, &status) == 0 &&
           (status & (1U << 8)) && ((status >> 9) & 0xf) == 4) // READY_FOR_DATA, TRAN 态
            return 0;                                         // （省略解锁与超时）
        ...
    }
}
```

CMD13 读卡的状态寄存器。第 8 位 `READY_FOR_DATA` 置位、第 9~12 位的状态回到 `TRAN` (4)，说明卡已经处理完之前的写，可以接受新命令。因为 `sdflush()` 持有块设备锁，屏障之后的写也不可能越过它。SD 6.x 规范还定义了一个可选的写缓存，需要专门的 `Flush Cache` 命令；本驱动没有启用它，所以 CMD13 就是当前支持的卡的持久化边界。

xv6 原生日志（第 6 章）也在四个位置用上了这个屏障：日志内容 → flush → 提交头 → flush → 写回原位 → flush → 清提交头 → flush。下面的 `xjournal` 用的是同一个思路。

9.1.7 外置事务日志 xjournal

要解决的问题。 一次 `rename` 要改新目录的数据块、旧目录的数据块，覆盖已有文件时还要改目标 `inode`、位图和计数，一共可能十几个扇区。这些写落在卡上的不同位置，断电可能发生在任意两次之间，留下“两个名字指向同一个 `inode`”或者“块已分配但没人用”的状态。ext2 本身对此无能为力，只能事后让 `e2fsck` 修。

`xjournal` 的思路与 `xv6` 原生日志相同：先把一次操作要写的所有元数据扇区完整地写到一块独立的区域，写一个“提交”标记，然后才写回原位。断电后只要看到提交标记就重放一遍，没看到就当这次操作没发生。区别在于它工作在 512 字节扇区一层，不依赖 `xv6` 的块缓存，也不知道 ext2 的格式，所以叫“外置”。

接口。 客户（这里是 ext2）在每个修改型操作前后调用 `xj_begin()` 和 `xj_commit()`；中间的写调用 `xj_write()`，它不写卡，而是把扇区存进事务缓冲；读调用 `xj_read()`，命中缓冲时返回未提交的新内容，这样同一个操作后面的步骤能看到前面的修改。缓冲是 128 个物理页（1024 个扇区），启动时分配。ext2 只在两个函数里接入它：

```
static int e2_rsector(uint32 lba, void *buf) {
    if(e2.journal && xj_read(lba, buf))                // 先查事务缓冲
        return 0;
    return sdsector(lba, buf, 0);
}
```

```
static int e2_wsector(uint32 lba, const void *buf) {
    if(e2.journal && xj_active() && (!e2.direct || xj_contains(lba)))
        return xj_write(lba, buf);           // 元数据：进事务
    return sdsector(lba, (void*)buf, 1);      // 文件数据：直接写
}
```

ordered 模式。 文件内容不进日志，由 `writeblock_data()` 置 `e2.direct` 后直接写回原位。这借鉴的是 ext3 的 `data=ordered`：数据先落盘，引用它的元数据（inode 里的块号、文件大小）后提交。新分配的块在提交前没人能看到，所以永远不会出现“文件里有一段垃圾”；覆盖已有数据则不保证原子——这与 Linux 上的语义一致，需要原子更新的程序应该自己用 WAL（见下一小节的 `waltest`）。这样做让日志只承载元数据，一个事务 1024 个扇区的容量对绝大多数操作都绰绰有余。

磁盘格式与提交协议。 日志区分三部分（图 9.1）：9 个扇区的描述符（魔数 XJD1、序号、扇区数和每个扇区的原位 LBA， $16 + 4 \times 1024$ 字节恰好需要 9 个扇区）、最多 1024 个扇区的载荷，以及 1 个扇区的提交记录（魔数 XJC1、序号、扇区数、整个事务的校验和，再加一个对这 16 字节自身的校验和）。校验和用 32 位 FNV-1a，覆盖描述符和全部载荷。提交分四步，每步后面都有一次 `sdflush()`（图 9.4）：

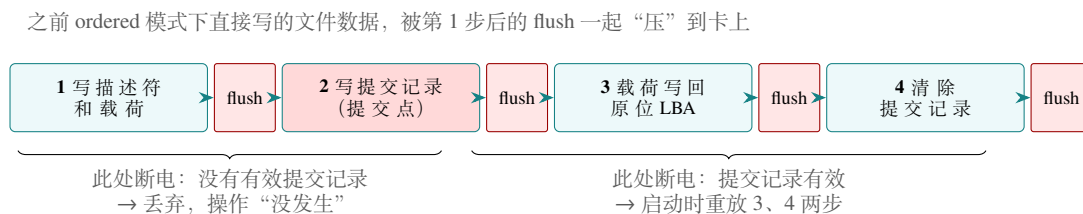


图 9.4: xjournal 的提交协议。四次 `sdflush()` 把四步严格排序；提交点之前和之后的断电有两种确定的结局

```
int xj_commit(void) {                                     // kernel/xjournal.c, 节选
    ...
    uint32 sum = txsum(count);                           // 描述符 + 载荷的 FNV
    // 1. 描述符和载荷；这次 flush 也保证此前直接写的已经落盘
    for(uint32 i = 0; i < XJ_DESC; i++) sdsector(xj.start + i, desc + i*512, 1);
    for(uint32 i = 0; i < count; i++) sdsector(xj.start + XJ_DESC + i, slot(i), 1);
    if(sdflush() < 0) goto fail;
    // 2. 提交记录：这个扇区落盘的那一刻，事务就生效了
    make_commit(seq, count, sum);
    if(sdsector(xj.start + XJ_DESC + XJ_CAP, xj.commit, 1) < 0 || sdflush() < 0)
        goto fail;
    // 3./4. 写回原位并清除提交记录；失败也没关系，下次启动会重放
    if(write_home(count) < 0 || clear_commit() < 0)
        printf("xjournal: checkpoint of seq=%d incomplete; replay at boot\n", seq);
    return 0;
fail:
    printf("xjournal: commit of seq=%d failed before commit point\n", seq);
    return -1;
}
```

提交点是一个扇区的写入。SD 卡保证单个扇区的写要么完成、要么不生效吗？并不完全保证，所以提交记录自带一个校验和：写坏的提交记录校验不过，等同于没有。

崩溃恢复。 `recover()` 在 `xj_init()` 里执行，顺序是：读提交记录并校验它自身；读描述符，确认魔数、序号和扇区数与提交记录一致；读载荷，重算整个事务的校验和并与提交记录比较；全部通过才把载荷写回原位、清除提交记录。任何一步不符都丢弃——描述符或载荷写到一半时断电，校验和必然对不上。表 9.2 列出了各个断电时刻的结局。

表 9.2: 断电时刻与恢复结果

断电发生在	磁盘状态	启动时
第 1 步期间或之后	原位置数据未动；日志里有半个或一整个事务，但没有提交记录	无事可做，操作没发生
第 2 步期间	提交记录可能是旧的 0 扇区或写坏的扇区	校验失败，丢弃
第 3 步期间	原位一部分是新的、一部分是旧的	重放：xjournal: replayed seq=N
第 4 步期间	原位已全部更新，提交记录还在	再重放一次（幂等），结果相同
第 4 步之后	一切都是新的	无事可做

重放是幂等的：载荷是扇区的完整新内容，而不是“加 1”“置位”这样的增量，写一次和写两次结果一样。这让恢复本身被打断也没有关系，下次启动再来一遍即可。

ext2 怎样使用事务。 每个修改型操作的外形都一样，以 `rename` 为例：

```
int ext2rename(char *oldpath, char *newpath) {
    acquiresleep(&e2.lock);
    tx_begin();                                // xj_begin(), 清空 freed[]
    int r = tx_end(rename_locked(oldpath, newpath) == 0); // 成功则提交，失败则中止
    releasesleep(&e2.lock);
    return r;
}
```

`tx_end(0)` 调用 `xj_abort()`，事务缓冲被直接丢掉，这次操作对磁盘的元数据没有留下任何痕迹。以前 `rename` 中途失败（比如目标目录满了又分配不到新块）会留下“新名字加上了、旧名字没删”的状态，有了事务就不会了。

少数操作会超过一个事务的容量：截断一个几 GiB 的文件要改成千上万个位图扇区。`tx_checkpoint()` 在剩余空间不足 96 个扇区时，先把 inode 以当前已截断到的大小写入事务并提交，再开一个新事务继续。每个分段的终点都是一个一致的状态：inode 的大小与它还引用的块相符，释放掉的块已经回到位图，只是文件还没截到最终长度。

最后，第一次写 ext2 时，`mark_write_session_dirty()` 绕过日志直接清除 superblock 的 `VALID_FS` 位并 flush。驱动还没有 `umount`，从来不会把它置回去，所以开发机上的 `e2fsck` 总会做一次完整检查。

9.1.8 测试：WAL 程序与随机断电

`user/waltest.c` 是一个极小的“数据库”：数据文件 `data.db` 里存一个值，修改时先写一条 redo 记录到 `redo.wal`。每条记录带魔数、事务号和校验和，提交顺序是：

```
写 redo.wal -> fdatsync -> 写 data.db -> fsync -> 清空 redo.wal -> fsync
```

它同时检验了两件事：`fsync` 是否真的建立了持久化边界，以及文件系统操作的语义是否正确。子命令如表 9.3。

断电测试在 QEMU 里做：准备一个与真机分区布局相同的 256 MiB SD 镜像，运行 `waltest jstress`，在随机时刻直接杀掉 QEMU 进程，再从同一个镜像启动。每次重启后，把 p3 从镜像里取出来，在宿主机上运行 `e2fsck -fn`。

```
dd if=sd.img of=p3.img bs=512 skip=215040 count=309248
```

表 9.3: waltest 的子命令

命令	检查什么
init / commit 42 / show	正常提交与读取
prepare 123, 断电, recover	只持久化 WAL 后断电, 重启必须输出 REDO tx=...value=123
fsops	mkdir、rename、unlink、ftruncate、pread、pwrite
semantics	rename 覆盖已有文件; 目录不能移入自己的子树; 删除仍被打开的文件后, 旧描述符仍可读写
large	5 GiB 处写一个字节, 走三级间接块
jstress	无限循环: 写文件、rename 覆盖、unlink, 供随机断电使用

```
e2fsck -fn p3.img          # 退出码 0 表示没有任何错误
```

结果: 第一轮随机断电 14 次, 其中 3 次正好落在“已提交、未写回”的窗口——这时如果不重放, e2fsck 会报错 (退出码 4); 三次启动都打印了 xjournal: replayed seq=..., 重放后全部无错误。其余 11 次断电前后都没有错误。拆分出 rootdev.c 以后又断电 4 次, 1 次重放 (seq=85, 27 个扇区), e2fsck 全部通过。

```
xjournal: replayed seq=85 (27 sectors)
xjournal: ready lba=198656 capacity=1024 sectors
ext2: p2 has no ext superblock
ext2: p3 ready rw start=215040 sectors=309248 block=4096 groups=2
```

(QEMU 镜像的 p2 较小, 日志起点是 198656; 真机上应为 274432。)

9.1.9 在真机上使用

p3 要格式化成不带 ext4 特性的 ext2。比第 6 章“烧录 SD 卡”一节里的命令多指定几个特性开关更稳妥:

```
diskutil unmount /dev/disk4s3
sudo mke2fs -F -t ext2 -b 4096 \
  -O filetype,sparse_super,large_file,^has_journal,^extent,^64bit,^metadata_csum \
  -L xv6-linux /dev/rdisk4s3
sudo e2fsck -fn /dev/rdisk4s3
```

/etc/fstab 里 ext2 一行要从 ro 改成 rw (init 不会覆盖已有的 fstab)。正常的启动日志:

```
xjournal: ready lba=274432 capacity=1024 sectors
ext2: p3 ready rw start=1030144 sectors=... block=4096 groups=...
vfs: mounted ext2 at /mnt/ext2 read-write
```

9.1.10 当前边界

这仍是一个小型的 ext2 后端, 与 Linux 的实现相比还缺不少:

- 不支持硬链接、符号链接、扩展属性、配额、HTree 索引目录和 GPT 分区表;
- 不支持 ext3 日志、ext4 extent、64 位块号和元数据校验和;
- 外置日志只有本内核认识。如果日志里还有已提交未写回的事务, 要先在 xv6 里启动一次让它重放, 再把卡插到 Linux 上;
- 孤儿 inode 只记在内存里, 没有 ext3 那样的磁盘孤儿链表。“删除仍被打开的文件”之后断电, 这个 inode 要靠 e2fsck 回收;
- 还没有 umount, 接回开发机时应运行 e2fsck -f。

这些都是不错的后续练习。其中最有价值的大概是磁盘孤儿链表和 umount: 前者只需要在 superblock 的

`s_last_orphan` 和 `inode` 的 `dttime` 字段里串一条链，在同一个事务里维护；后者让 `VALID_FS` 能在干净卸载时置回去，开发机就不必每次都做完整检查。

参考文献

- [1] Arm Limited. *Arm Architecture Reference Manual for A-profile architecture*. ARM DDI 0487. Arm Limited. 2024.
- [2] Arm Limited. *Arm Cortex-A53 MPCore Processor Technical Reference Manual*. ARM DDI 0500. Arm Limited. 2018.
- [3] Daniel J. Bernstein et al. *TweetNaCl: a crypto library in 100 tweets*. 2014. URL: <https://tweetnacl.cr.yp.to/>.
- [4] Broadcom Corporation. *BCM2835 ARM Peripherals*. Broadcom. 2012.
- [5] Broadcom Corporation. *BCM2836 ARM-local peripherals (Quad-A7 control)*. Broadcom. 2014.
- [6] Russ Cox, Frans Kaashoek, and Robert Morris. *xv6: a simple, Unix-like teaching operating system*. MIT PDOS, 2024. URL: <https://pdos.csail.mit.edu/6.1810/xv6/book-riscv-rev4.pdf>.
- [7] Sergey Matyukevich. *raspberrypi-os: Learning operating system development using Linux kernel and Raspberry Pi*. 2018. URL: <https://github.com/s-matyukevich/raspberrypi-os>.
- [8] Microsoft Corporation. *Microsoft Extensible Firmware Initiative FAT32 File System Specification*. Microsoft. 2000.
- [9] MIPI Alliance. *MIPI Alliance Specification for Camera Serial Interface 2 (CSI-2)*. MIPI Alliance. 2017.
- [10] Raspberry Pi Ltd. *Raspberry Pi firmware: boot files, config.txt and the mailbox property interface*. 2026. URL: <https://github.com/raspberrypi/firmware/wiki>.
- [11] Raspberry Pi Ltd and Linux kernel developers. *Raspberry Pi Linux kernel source (drivers: brcmfmac, mt7601u, dwc2, bcm2835-unicam, ov5647, i2c-bcm2835)*. 2026. URL: <https://github.com/raspberrypi/linux>.
- [12] rockytriton. *LLD: Low Level Devel – Raspberry Pi bare metal series*. 2021. URL: <https://github.com/rockytriton/LLD>.
- [13] SD Association. *SD Specifications Part 1: Physical Layer Simplified Specification*. SD Association. 2020.
- [14] SD Association. *SD Specifications Part E1: SDIO Simplified Specification*. SD Association. 2020.
- [15] Rene Stange. *Circle: a C++ bare metal programming environment for the Raspberry Pi*. 2026. URL: <https://github.com/rsta2/circle>.
- [16] USB Implementers Forum. *Universal Serial Bus Specification, Revision 2.0*. USB-IF. 2000.